

The Kafka Handbook

Volume I — Foundations

For the Java backend developer with zero Kafka knowledge.
Built for deep understanding, interview readiness, and production work.

- Part 1 Introduction: the problem Kafka solves
- Part 2 Messaging systems compared
- Part 3 Event-driven architecture
- Part 4 Kafka architecture from first principles
- Part 5 Installation
- Part 6 Running Kafka & the CLI toolbox

Targets Apache Kafka 3.7 – 4.x (KRaft) · Java 17+ · Spring Boot 3.2+
Every chapter ends with Summary · Interview Questions · Mini Quiz · Practice Exercises · Common Mistakes · Further Reading

Contents

Part 1 — Introduction: The Problem Kafka Solves	6
1.1 Start with the pain, not the tool	6
1.2 The first fix: put a buffer in the middle	7
1.3 So what is Kafka?	8
1.4 Traditional communication styles	9
1.5 Message Broker vs Message Queue	11
1.6 Why Kafka was created	12
1.7 Real-world examples you can picture	12
Summary	13
Interview Questions	14
Mini Quiz	14
Practice Exercises	15
Common Mistakes	15
Further Reading	16
Part 2 — Messaging Systems Compared	17
2.1 The one distinction that explains everything	17
2.2 RabbitMQ	18
2.3 Apache ActiveMQ (Classic and Artemis)	19
2.4 IBM MQ (formerly MQSeries / WebSphere MQ)	21
2.5 Amazon SQS (and SNS, and Kinesis)	22
2.6 Apache Kafka	23
2.7 Apache Pulsar (worth 30 seconds, because interviewers ask)	23
2.8 The comparison table	24
2.9 Decision flowchart	25
2.10 Using both is normal	25
Summary	26
Interview Questions	26
Mini Quiz	27
Practice Exercises	28
Common Mistakes	28
Further Reading	29
Part 3 — Event-Driven Architecture	30
3.1 What is an event?	30
3.2 Events vs Commands vs Notifications	31
3.3 Producer and Consumer	33
3.4 REST vs Events	34
3.5 Why large companies prefer events	37
3.6 Fat events vs thin events (the classic design argument)	37
3.7 Choreography, orchestration, and sagas	38
3.8 Event sourcing ≠ event-driven (don't conflate these)	39
Summary	39
Interview Questions	40
Mini Quiz	41
Practice Exercises	42
Common Mistakes	42
Further Reading	43

Part 4 — Kafka Architecture From First Principles	44
4.1 A five-minute history (interviewers do ask)	44
4.2 Building Kafka from scratch (derive the architecture)	44
4.3 The nouns, precisely	46
4.4 Putting it together: full write path	50
4.5 Retention: how data actually leaves	50
4.6 Leader, follower, and ISR	51
4.7 Why ZooKeeper existed	52
4.8 KRaft: Kafka Raft	53
4.9 Controller: what it actually does	55
4.10 Metadata: what clients cache and why it matters	56
Summary	57
Interview Questions	58
Mini Quiz	59
Practice Exercises	60
Common Mistakes	61
Further Reading	61
Part 5 — Installation	63
5.1 Prerequisites	63
5.2 Manual installation (Linux / macOS) — KRaft mode	63
5.3 macOS	67
5.4 Windows	67
5.5 Docker — the recommended dev setup	67
5.6 ZooKeeper mode (legacy, for reference only)	70
5.7 The configuration files, annotated	70
5.8 OS-level production tuning	72
Summary	73
Interview Questions	73
Mini Quiz	74
Practice Exercises	75
Common Mistakes	75
Further Reading	76

Part 6 — Running Kafka & the CLI Toolbox	77
6.1 Starting and stopping	77
6.2 Topics	79
6.3 Console producer	81
6.4 Console consumer	81
6.5 Consumer groups — the most important tool you own	82
6.6 Offsets and lag	85
6.7 Inspecting the physical log (great for interviews)	85
6.8 Performance testing	86
6.9 Partition reassignment and leader balancing	86
6.10 Other tools worth knowing	87
6.11 A 10-minute hands-on walkthrough	87
Summary	88
Interview Questions	89
Mini Quiz	90
Practice Exercises	91
Common Mistakes	91
Further Reading	92

Part 1 — Introduction: The Problem Kafka Solves

1.1 Start with the pain, not the tool

Most Kafka tutorials open with "Kafka is a distributed streaming platform." That sentence teaches you nothing. Let's earn it instead.

You are a Java backend developer at an e-commerce company. You own `OrderService`. It has one job: accept an order.

Week 1. The code is beautiful.

```
@PostMapping("/orders")
public OrderResponse create(@RequestBody OrderRequest req) {
    Order order = orderRepo.save(new Order(req));
    return new OrderResponse(order.getId());
}
```

Week 2. Payments team: "call us when an order is placed."

```
Order order = orderRepo.save(new Order(req));
paymentClient.charge(order);           // +1 HTTP call
```

Week 6. Inventory, shipping, email, and analytics all want in.

```
Order order = orderRepo.save(new Order(req));
paymentClient.charge(order);           // 120 ms
inventoryClient.reserve(order);        // 90 ms
shippingClient.schedule(order);        // 200 ms
emailClient.sendConfirmation(order);   // 300 ms (SMTP is slow)
analyticsClient.track(order);          // 40 ms
fraudClient.score(order);              // 150 ms
```

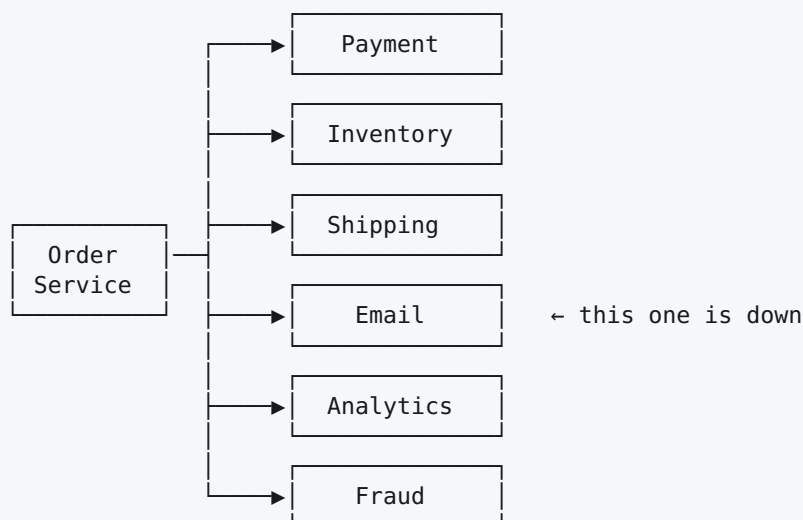
Your endpoint now takes **900 ms**. And it has six new ways to fail.

Week 7, 2 a.m. The email service is down for 10 minutes. Because `emailClient.sendConfirmation()` throws, your order endpoint returns 500. The company stops selling shoes because a *mail server* is unhappy. Nobody can explain to the CEO why an email outage stopped revenue.

Week 9. You add retries. Now duplicate payments appear. You add idempotency keys. Now you have a distributed transaction problem you didn't sign up for.

Week 12. A new "Loyalty Points" team asks you to add a seventh call. You now have to redeploy `OrderService` — a service that has nothing to do with loyalty points — to add a feature to loyalty points.

The shape of the problem



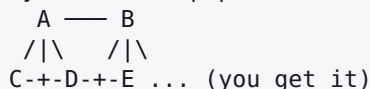
Order Service must know: WHO exists, WHERE they are, WHETHER they are up, WHAT their API looks like, and HOW to retry each of them.

This is the **N×M integration problem**. With 6 services it's annoying. LinkedIn in 2010 had dozens of data systems and hundreds of pipelines between them, and each new system meant writing new bespoke pipelines to every other system. The number of connections grows like N^2 .

3 systems → 3 pipelines



6 systems → 15 pipelines



The five diseases

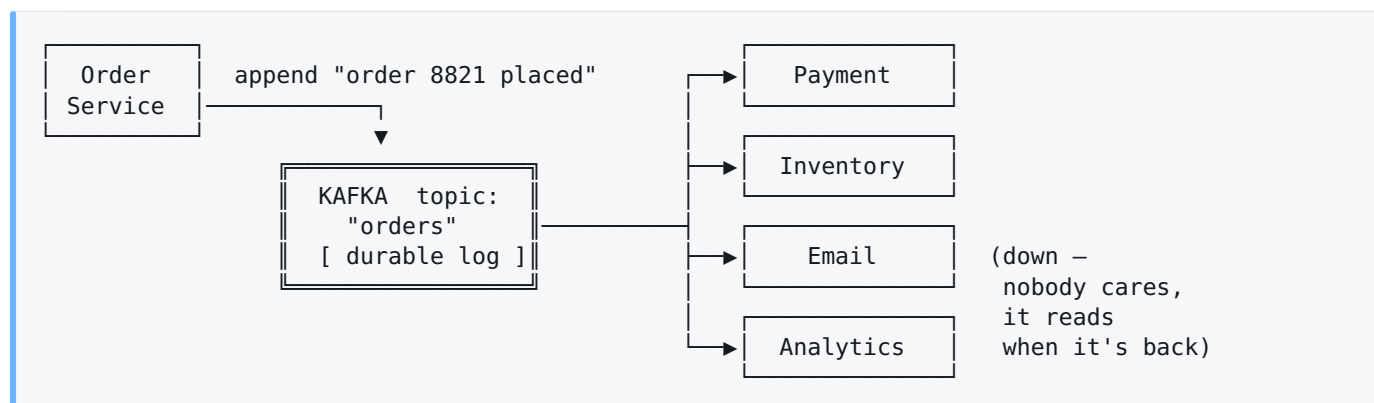
Disease	Symptom
Tight coupling	Order Service imports 6 client SDKs and knows 6 hostnames
Cascading failure	One slow downstream kills the upstream request
Latency accumulation	Response time = sum of everything, not max
No replay	If Analytics had a bug, yesterday's orders are gone forever
Deploy coupling	New consumer of order data ⇒ redeploy the producer

Everything Kafka does is a cure for one of these five.

1.2 The first fix: put a buffer in the middle

Instead of calling six services, `OrderService` writes one fact to a durable log: *"order 8821 was placed."* Then it returns. Done in 5 ms.

The six other services read that log at their own pace.



Notice what changed:

- Order Service knows about **one** system, not six.
- Email being down affects **email only**.
- A seventh consumer (Loyalty) can be added with **zero changes** to Order Service.
- Analytics can re-read the last 7 days after fixing a bug — the log is still there.

That "durable log in the middle" is Kafka. Everything else in this handbook is detail about how to make that log fast, replicated, ordered, and safe.

1.3 So what *is* Kafka?

Apache Kafka is a distributed, replicated, append-only commit log that multiple independent readers can read at their own speed and position.

Read that again slowly, because each word is load-bearing:

Word	Meaning	Consequence
append-only	you can only add to the end, never modify	writes are sequential ⇒ extremely fast, even on spinning disks
log	ordered sequence of records with numeric positions	ordering is a first-class guarantee, replay is trivial
commit log	same idea as a database WAL / redo log	Kafka is essentially a database's most reliable component, extracted and scaled
distributed	split across many machines	one topic can be bigger than one machine
replicated	every record exists on N machines	a machine can die without data loss
readers at their own position	each consumer tracks its own offset	one slow reader doesn't slow the others; readers can rewind

The single most important mental model

Most Java developers arrive with a **queue** mental model from JMS/RabbitMQ: *"a message arrives, a consumer takes it, the message is gone."*

Kafka is not that. Delete that model now.


```

QUEUE model (RabbitMQ, SQS, JMS):
  [ m1 m2 m3 ] → consumer takes m1 → [ m2 m3 ]      (m1 destroyed)

LOG model (Kafka):
  offset:  0    1    2    3    4    5
           [m1] [m2] [m3] [m4] [m5] [m6] ...
                ↑           ↑
            consumerA reads here  consumerB reads here

Nothing is destroyed by reading. Reading = "move my bookmark forward".

```

In Kafka, **consuming does not delete**. Records are deleted by a *retention policy* (time or size), completely independently of whether anyone read them. This one difference explains:

- why multiple teams can consume the same data,
- why you can replay history,
- why Kafka scales reads so well (no per-message bookkeeping on the broker),
- and why "my consumer read the message, why is it still there?" is not a bug.

Analogy that actually holds up

Kafka is a **newspaper archive**, not a **mailbox**.

- A mailbox: the letter is for you; when you take it, it's gone.
- A newspaper archive: every issue is filed in order by date. You, your neighbour, and a historian in 2050 can all read issue #4,182. Your "offset" is just which issue you've read up to. The library discards issues older than its retention policy — not because someone read them.

1.4 Traditional communication styles

Before we go further, we need shared vocabulary. Four styles, in order of increasing decoupling.

(a) Synchronous request/response (REST, gRPC, SOAP)

```

Client — request —> Server
Client <- response — Server      (client BLOCKS and waits)

```

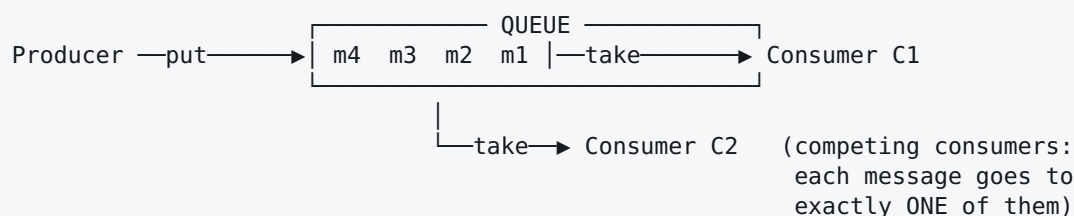
Analogy: a phone call. Both parties must be available at the same moment.

- ✓ Simple, immediate answer, easy to debug, natural for queries.
- ✗ Caller must know the callee. Callee must be up. Caller pays the latency. Fan-out is $O(N)$ calls.

Use it when you need an answer to continue: "is this coupon valid?", "give me the user profile", "authorise this card".

(b) Point-to-point messaging (a queue)

Exactly one consumer processes each message.



Analogy: a bank with one queue and three tellers. Each customer is served by exactly one teller. Add tellers ⇒ the line moves faster.

- ✓ Load balancing (work distribution), decoupled in time, natural back-pressure.
- ✗ One logical consumer only. If Analytics *also* wants the data, it has to either steal from Payments or you need a second queue and the producer must know to write to both.

Use it for: work/task distribution — image resizing, PDF generation, sending SMS.

(c) Publish/Subscribe (a topic)

Every subscriber gets **every** message.



Analogy: a radio broadcast. The DJ doesn't know or care who is listening.

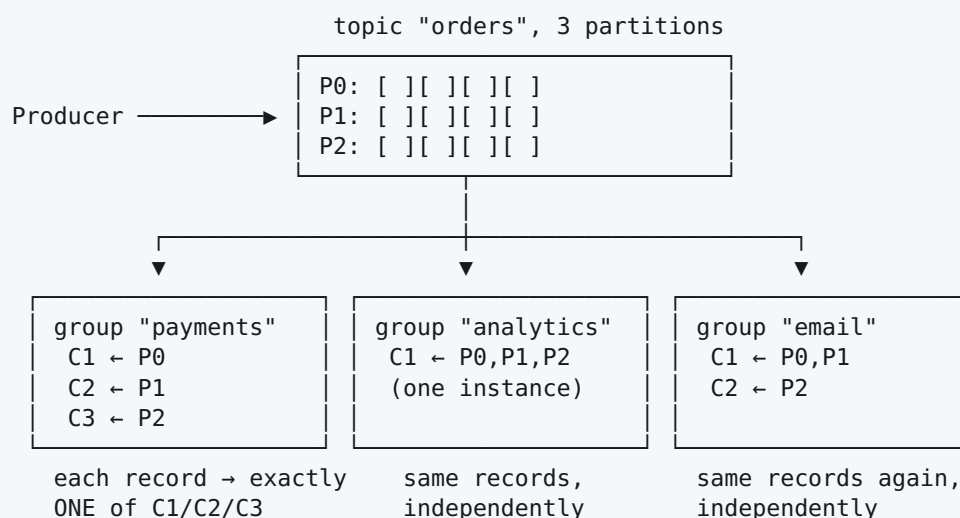
- ✓ Publisher is completely unaware of subscribers. Add subscriber #4 with zero producer changes.
- ✗ Classic (JMS-style) pub/sub is *ephemeral*: if a subscriber is offline when the message is published, it misses it forever — unless it has a "durable subscription".

Use it for: broadcasting events — "price changed", "user logged out", "config updated".

(d) Kafka: both at once

Here is the trick that makes Kafka special, and it's a favourite interview question.

Kafka is pub/sub *between* consumer groups, and point-to-point *within* a consumer group.



ACROSS groups = publish/subscribe (everyone sees everything)
 WITHIN a group = point-to-point (each record handled once)

You get work distribution *and* broadcast from one primitive. RabbitMQ needs an exchange + per-consumer queues to emulate this; Kafka gets it for free from the log + per-group offsets.

Comparison table

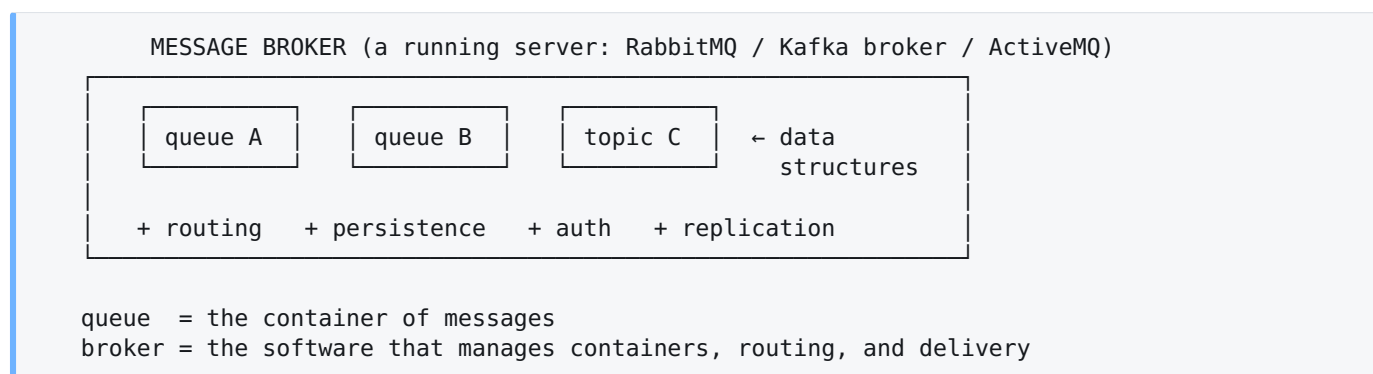
	Sync REST	Queue (P2P)	Classic Pub/Sub	Kafka
Caller knows callee	yes	no	no	no
Callee must be up	yes	no	no (if durable)	no
Message consumed by	1	1	all subscribers	1 per group, all groups
Message survives read	n/a	no	no	yes
Replay history	no	no	rarely	yes
Ordering guarantee	n/a	usually FIFO-ish	weak	per partition, strict
Typical throughput	1k-10k rps	10k-50k msg/s	10k-50k msg/s	100k-1M+ msg/s
Immediate answer	yes	no	no	no

1.5 Message Broker vs Message Queue

Interviewers love this because candidates use the words interchangeably.

Message Queue = a *data structure*. An ordered buffer that holds messages. It's a noun describing storage: FIFO in, FIFO out.

Message Broker = a *server/middleware process* that receives messages from producers and routes them to consumers. A broker *contains* queues (and topics, exchanges, routing rules, ACLs, persistence, monitoring...).



One-line answer for an interview:

"A queue is a data structure that stores messages in order. A broker is the middleware server that hosts queues and topics and is responsible for routing, persistence, and delivery. RabbitMQ is a broker; a RabbitMQ queue is a queue. Kafka is a broker, but its storage abstraction is a partitioned log rather than a queue — which is why reads are non-destructive."

Where does Kafka sit?

Kafka is a broker, but calling it "a message queue" undersells it. It's really three products in one:

Role	What it means	Competes with
Messaging system	decoupled async communication	RabbitMQ, SQS, ActiveMQ
Storage system	durable, replicated, retained log (days-forever)	HDFS-lite, a database WAL
Stream processing platform	Kafka Streams / ksqldb compute over the log	Flink, Spark Streaming

That "storage" row is the unusual one. Kafka is comfortable holding terabytes for weeks. That's what makes replay, backfill, and "Kafka as source of truth" (event sourcing) possible.

1.6 Why Kafka was created

Concrete history, because interviewers ask "why was Kafka built?"

Where: LinkedIn, 2010. **Who:** Jay Kreps, Neha Narkhede, Jun Rao. **Open sourced:** 2011. **Apache top-level project:** 2012. **Named after:** Franz Kafka — Kreps liked the writer and it was "a system optimised for writing."

The actual problem: LinkedIn had two categories of data plumbing, both broken.

1. **User activity tracking** — page views, clicks, searches. Volume was enormous and the existing solution was batched XML-over-HTTP flushed every few minutes to a scraper. Latency was minutes, and it fell over regularly.
2. **Operational metrics** — a separate, also-custom pipeline.

And then a proliferation of destinations: the data warehouse, Hadoop, search indexes, the recommendation engine, monitoring dashboards, newsfeed. Each new system meant writing new point-to-point pipelines. The team was spending its life writing plumbing.

What they evaluated first: existing message brokers (ActiveMQ and friends). They failed for specific, technical reasons — worth memorising:

Requirement	Why existing brokers failed
Hundreds of MB/s throughput	brokers tracked per-message state and did random I/O
Persist for days, not seconds	queues assumed messages are transient; large backlogs degraded them badly
Many independent consumers of the same feed	destructive reads meant duplicating data per consumer
Horizontal scale-out of a single feed	a queue lived on one broker; no partitioning of one logical stream
Replay/backfill	once consumed, gone
Handle a slow/offline consumer without hurting others	slow consumers caused broker memory blowups

The insight: stop trying to be a queue. Be a *log*. A log is the simplest possible data structure that gives you ordering and durability, and — crucially — it can be written and read **sequentially**, which lets you use the disk at near-memory speeds and hand pages straight to the network card.

We'll make that concrete in Part 7 (sequential I/O, page cache, zero-copy). For now, hold this: **Kafka is fast because it does less, not because it does something clever.** No per-message acknowledgement bookkeeping, no random-access index over messages, no in-broker routing decisions per message. Append bytes. Serve byte ranges. That's it.

1.7 Real-world examples you can picture

Scenario	The event	Who reads it
Uber ride request	<code>ride.requested</code>	matching engine, pricing/surge, fraud, ETA, analytics

Scenario	The event	Who reads it
Netflix "you watched 12 min of X"	<code>playback.heartbeat</code>	recommendations, billing/quality, A/B analysis, CDN health
Bank card swipe	<code>transaction.authorised</code>	ledger, fraud scoring, SMS alerts, monthly statement, regulator feed
Swiggy order	<code>order.placed</code>	restaurant app, delivery-partner assignment, payments, ETA, notifications
Log aggregation	<code>app.log.line</code>	Elasticsearch, S3 archive, alerting, security (SIEM)
IoT / vehicle telemetry	<code>sensor.reading</code>	real-time dashboard, anomaly detection, cold storage
Database CDC	<code>mysql.customers.updated</code>	search index, cache invalidation, data warehouse

The pattern is always the same: **one fact, many independent interpretations, each at its own speed.** If your problem has that shape, Kafka fits. If your problem is "I need an answer right now to continue," it does not — use REST.

When Kafka is the WRONG choice

Be able to say this in an interview; it signals maturity.

- **You need a synchronous reply.** Kafka has no request/response. (You can fake it with a reply topic + correlation ID; it's usually a mistake.)
- **You need per-message priority.** Kafka has no priority queues. A log has one order.
- **You need per-message delayed delivery / scheduling.** No native TTL-per-message or "deliver at 5 pm". RabbitMQ does this natively.
- **You need complex routing rules** (topic patterns, header-based routing to different queues). That's RabbitMQ's exchanges.
- **Very low volume, small team.** Kafka's operational cost (brokers, monitoring, rebalancing knowledge) is not worth it for 100 messages/day. Use SQS or a DB table.
- **You need to delete or modify one specific message** (GDPR "forget user X"). Possible via compaction + tombstones, but painful; plan for it upfront.

Summary

- Kafka exists to cure five diseases of direct service-to-service calls: tight coupling, cascading failure, latency accumulation, no replay, deploy coupling.
- Kafka is a **distributed, replicated, append-only commit log**, not a queue. **Reading does not delete.** Retention deletes.
- Consumers track their own **offset** (bookmark), so many independent readers can read the same data at different speeds and positions.
- Kafka is **pub/sub across consumer groups** and **point-to-point within a consumer group** — one primitive, both patterns.
- A **queue** is a data structure; a **broker** is the server that hosts queues and topics and handles routing, persistence, and delivery.
- Built at LinkedIn (2010, Kreps/Narkhede/Rao) because existing brokers could not do high throughput + long retention + many independent consumers + horizontal scale + replay.
- It is fast because it does *less*: sequential appends, no per-message broker bookkeeping, byte-range reads.

Interview Questions

Q1. What is Kafka in one sentence? A distributed, replicated, fault-tolerant append-only commit log that lets many independent consumers read the same ordered stream of records at their own position and pace.

Q2. Why not just use REST calls between microservices? REST is synchronous, so the caller must know and reach the callee, pays the sum of downstream latencies, and fails when any downstream fails. Fan-out means N calls and N failure modes, and adding a new consumer requires changing the producer. Kafka inverts this: the producer writes one fact once; consumers are added, removed, or replayed without the producer knowing. The trade-off is that you give up an immediate answer and accept eventual consistency.

Q3. Is Kafka a message queue? It's a broker, but its storage model is a partitioned log rather than a queue. The practical difference: consuming doesn't remove data, so multiple consumer groups can read the same records and any group can rewind. Deletion is driven by retention policy, not consumption.

Q4. Difference between a message queue and a message broker? Queue = data structure holding ordered messages. Broker = middleware server that hosts queues/topics and provides routing, persistence, security and delivery semantics.

Q5. Difference between point-to-point and publish/subscribe? Which is Kafka? P2P: each message goes to exactly one of the competing consumers (work distribution). Pub/sub: every subscriber receives every message (broadcast). Kafka does both — pub/sub across consumer groups, P2P within a group.

Q6. Why was Kafka created at LinkedIn? Their activity-tracking and metrics pipelines were bespoke, high-latency and fragile, and every new downstream system (Hadoop, warehouse, search, recs) required a new point-to-point pipeline — an N^2 integration problem. Existing brokers couldn't handle the throughput, couldn't retain data for days, degraded with large backlogs, and couldn't serve many independent consumers of one feed or scale a single feed across machines.

Q7. When would you NOT use Kafka? When you need a synchronous response, per-message priority, per-message delayed delivery, complex header/pattern-based routing, or when volume is so low the operational overhead isn't justified.

Q8. Kafka is described as fast. Why? (high level) Because it minimises work: append-only sequential disk writes, no per-message state tracking on the broker, batching and compression on the client, reliance on the OS page cache, and zero-copy transfer from page cache to socket. Details in Part 7.

Q9. What does "Kafka retains data" buy you that a queue doesn't? Replay (reprocess after fixing a consumer bug), backfill (bootstrap a brand-new service from history), debugging (inspect exactly what was sent), and multiple independent consumers without duplicating storage.

Q10. Your Kafka consumer processed a message but you still see it in the topic. Bug? No. Consuming in Kafka moves the consumer's offset; it doesn't delete the record. Records disappear only when retention (`retention.ms` / `retention.bytes`) or compaction removes them.

Mini Quiz

1. True/false: in Kafka, when a consumer reads a record, the record is removed from the broker.
2. Two consumer groups, `A` and `B`, subscribe to topic `orders`. A record is produced. How many groups see it? How many *consumers within* group A see it?
3. What deletes data in Kafka?
4. Name three of the five "diseases" of direct synchronous fan-out.
5. You need to know whether a coupon code is valid before responding to the user. Kafka or REST?

6. Queue or broker: RabbitMQ. Queue or broker: `payment-requests`.
7. Why did LinkedIn reject ActiveMQ-style brokers? Give two technical reasons.
8. What does "append-only" buy you at the disk level?

ANSWERS

1. **False.** Reading moves the consumer's offset; the record stays.
2. Both groups see it (pub/sub across groups). Exactly **one** consumer inside group A sees it (P2P within a group).
3. Retention policy — time (`retention.ms`) or size (`retention.bytes`) — or log compaction. Not consumption.
4. Tight coupling, cascading failure, latency accumulation, no replay, deploy coupling.
5. **REST.** You need a synchronous answer to continue.
6. RabbitMQ = broker. `payment-requests` = a queue (a data structure/destination).
7. Couldn't sustain the throughput (per-message bookkeeping + random I/O); degraded badly with large retained backlogs; destructive reads forced data duplication per consumer; a single feed couldn't be partitioned across brokers; no replay.
8. Sequential writes — the disk head never seeks, so throughput approaches the drive's streaming limit, and the OS can do large readahead and write-behind.

Practice Exercises

1. **Draw the disease.** Take a system you've actually worked on. Draw the synchronous fan-out. Circle every place a downstream failure becomes an upstream 500.
2. **Redraw it with a log.** Same system, Kafka in the middle. For each downstream, write one sentence: what happens now if it's down for 30 minutes?
3. **Classify 10 interactions.** List 10 interactions in that system. Label each `SYNC` (needs an answer now) or `EVENT` (a fact others react to). Be honest — most people find 60–80% are events masquerading as REST calls.
4. **The replay question.** Name one bug you shipped where "we could have replayed the last 24 hours of input" would have saved you. This is the argument you'll use to sell Kafka to a skeptical team.
5. **Anti-Kafka.** Find one requirement in your system that Kafka is *bad* at (priority, scheduling, request/reply). Write down what you'd use instead.

Common Mistakes

- **Bringing the queue mental model.** Expecting messages to disappear on read. Every early Kafka confusion traces back to this.
- **Using Kafka for request/response.** "Send request on topic A, wait for reply on topic B" — you've built a slow, hard-to-debug RPC. Use REST/gRPC.
- **Replacing every REST call with an event.** Some calls genuinely need an answer. Eventual consistency is a cost, not a free upgrade.
- **Treating Kafka as a database.** It has no queries, no secondary indexes, no updates by key (compaction is not an UPDATE). It's a log; project it into a database if you need to query it.
- **Adopting Kafka for 100 messages/day.** The operational surface (brokers, monitoring, rebalance debugging, schema evolution) is real. Match the tool to the volume.

- **Saying "Kafka is a message queue" in an interview** without immediately qualifying it. It's the fastest way to signal shallow knowledge.
-

Further Reading

- Jay Kreps, "*The Log: What every software engineer should know about real-time data's unifying abstraction*" (2013) — the single most important thing to read about Kafka. Non-optional.
- Jay Kreps, "*Kafka: a Distributed Messaging System for Log Processing*" (NetDB 2011) — the original paper; short and readable.
- Martin Kleppmann, *Designing Data-Intensive Applications*, Ch. 11 ("Stream Processing") — the best textbook treatment of log-based messaging vs traditional messaging.
- Apache Kafka docs, section 1.1 "Introduction" and 4.1 "Persistence".

Part 2 — Messaging Systems Compared

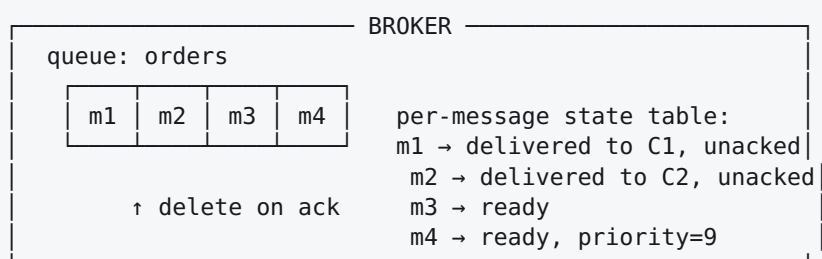
You will be asked "why Kafka and not RabbitMQ?" in almost every interview where Kafka appears on your CV. A vague answer ("Kafka is faster") gets you nowhere. A good answer names the *architectural* difference and then derives the trade-offs from it.

2.1 The one distinction that explains everything

There are exactly two families of messaging systems.

Family A — Smart broker, dumb consumer (queue-based)

The broker owns per-message state. It knows which messages are unacknowledged, which consumer has which message, when to redeliver, what priority each has, and when to route to a dead-letter queue.



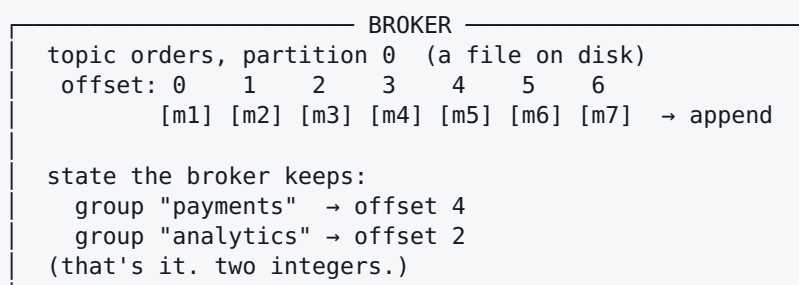
Members: RabbitMQ, ActiveMQ, IBM MQ, Amazon SQS, JMS in general.

What this buys you: priority, per-message TTL, delayed delivery, header-based routing, competing consumers at message granularity, native DLQ, selective ack.

What it costs you: the broker does bookkeeping per message. That's random I/O, memory, and lock contention. Throughput plateaus in the tens of thousands per second. Deep backlogs hurt because the state table no longer fits in RAM. And when a message is acked, it's gone — replay is not a feature.

Family B — Dumb broker, smart consumer (log-based)

The broker stores an immutable ordered log and serves byte ranges. It keeps **no per-message state**. The only state is one number per (group, partition): the committed offset.



Members: Kafka, Apache Pulsar (hybrid), AWS Kinesis, Redpanda.

What this buys you: enormous throughput (sequential I/O, zero-copy), retention independent of consumption, replay, many independent consumer groups, and horizontal scale of a single logical stream via partitions.

What it costs you: no priority, no per-message TTL, no delayed delivery, no selective ack (a slow message head-of-line-blocks its partition), coarser routing.

The interview line: "RabbitMQ tracks state per message, which is why it can do priority, delay and per-message redelivery — and why it caps out at tens of thousands of messages per second. Kafka tracks one offset per consumer group per partition, which is why it does millions per second with long retention and replay — and why it can't do priority or per-message delay. Pick based on whether you need per-message control or per-stream throughput."

Everything below is elaboration on that paragraph.

2.2 RabbitMQ

Analogy: a post office with a very clever sorting clerk. You hand over an envelope with routing instructions on it; the clerk decides which of the many pigeonholes it belongs in, handles registered mail, priority mail, and "deliver on Tuesday."

Technically

- Erlang/OTP implementation of **AMQP 0-9-1** (also supports MQTT, STOMP, AMQP 1.0).
- Core model: **producer → exchange → (binding rules) → queue → consumer**.
- Four exchange types:

DIRECT	routing key must match binding key exactly "payment.usd" → queue bound with "payment.usd"
TOPIC	wildcard patterns: * = one word, # = zero or more "order.us.created" → queue bound with "order.*.created" → queue bound with "order.#"
FANOUT	ignore routing key, copy to every bound queue (broadcast)
HEADERS	route on AMQP header key/values instead of routing key

- Consumer **acks** each message (`basic.ack`); unacked messages are redelivered after the consumer dies or `consumer_timeout` fires.
- `prefetch` (QoS) controls how many unacked messages a consumer may hold — this is RabbitMQ's back-pressure knob.

Internally

- Messages live in queue processes (Erlang lightweight processes) with an index + message store. Small messages are embedded in the queue index; large ones go to a shared message store.
- Queues are memory-first with paging to disk when memory pressure hits. A deep backlog triggers paging, and throughput drops noticeably. RabbitMQ is happiest when queues are near-empty.
- **Classic queues** live on one node (mirroring was deprecated). **Quorum queues** (3.8+) use Raft for replication and are the modern choice for durability.
- **Streams** (3.9+) were added as an append-only, replayable log — RabbitMQ copying Kafka's model because customers wanted it.

Example

```
// RabbitMQ producer (amqp-client)
ConnectionFactory f = new ConnectionFactory();
f.setHost("localhost");
try (Connection conn = f.newConnection(); Channel ch = conn.createChannel()) {
    ch.exchangeDeclare("orders", BuiltinExchangeType.TOPIC, true);
    ch.basicPublish("orders", "order.us.created",
        MessageProperties.PERSISTENT_TEXT_PLAIN,
        "{\"orderId\":\"8821\"}.getBytes(StandardCharsets.UTF_8));
}

// RabbitMQ consumer – note the explicit per-message ack
ch.basicQos(10); // prefetch
ch.basicConsume("us-orders", false, (tag, delivery) -> {
    process(delivery.getBody());
    ch.basicAck(delivery.getEnvelope().getDeliveryTag(), false); // ← per message
}, tag -> {});
```

Compare with Kafka, where there is no per-message ack — only "my group has processed up to offset N."

Why it exists / when to pick it

Pick RabbitMQ when the *unit of work* matters more than the *stream*:

- **Task queues** with priority: "premium customers' PDFs first."
- **Delayed jobs**: "retry this payment in 15 minutes" (delayed-message plugin, or TTL + dead-letter exchange).
- **Complex routing**: one publish, routed by pattern to five different teams' queues, with rules living in the broker instead of in code.
- **Per-message retry and DLQ** semantics out of the box.
- **RPC over messaging** (it has first-class `reply_to` + `correlation_id`).
- Low-to-moderate volume (< ~50k msg/s) where operational simplicity wins.

Strengths / weaknesses

✓	✗
Richest routing model of any broker	Throughput ceiling ~10k-50k msg/s per queue
Priority queues, per-message TTL, delayed delivery	Not designed to retain data for days
Native DLQ + retry semantics	No replay (ack destroys)
Mature management UI, easy to start	A single queue doesn't scale across nodes (partition manually)
Many protocols (AMQP, MQTT, STOMP)	Deep backlogs degrade performance (memory paging)
Per-message ack = fine-grained failure handling	Erlang expertise needed for deep debugging

2.3 Apache ActiveMQ (Classic and Artemis)

Analogy: the reliable old company mailroom. It's been there for 20 years, it follows the corporate standard to the letter, and everyone's legacy Java app already knows how to talk to it.

Technically

- The reference **JMS 1.1 / 2.0** broker. Java-native. If your codebase uses `javax.jms` / `jakarta.jms`, ActiveMQ is the default target.
- Two products, and interviewers appreciate you knowing the difference:
- **ActiveMQ Classic (5.x)** — the original. Thread-per-connection, KahaDB store, "Network of Brokers" for scale.
- **ActiveMQ Artemis** — based on the donated HornetQ codebase, non-blocking asynchronous architecture, much higher throughput, and the basis of **Red Hat AMQ 7**. This is the future of the project.
- JMS concepts map like this:

JMS	Meaning	Kafka's nearest thing
Queue	point-to-point, one consumer wins	a consumer group with N consumers
Topic	pub/sub, non-durable by default	topic (but Kafka is always durable)
Durable subscription	topic subscriber that survives disconnect	consumer group + committed offsets
Message selector (SQL-92 on headers)	broker-side filtering	✗ no equivalent; filter in the consumer
MessageListener	push model	✗ Kafka is pull (<code>poll()</code>)

Internally

- **Classic**: KahaDB — a journal (append-only) plus a B-tree index of message locations. Also supports LevelDB (deprecated) and JDBC stores.
- **Artemis**: append-only journal with either NIO or Linux **libaio** (async direct I/O), plus paging to disk for large backlogs. Journal + in-memory index is architecturally much closer to Kafka than Classic ever was.
- **Message selectors** are evaluated on the broker. Powerful, but it means the broker is doing per-message CPU work — the opposite of Kafka's philosophy.

Example

```
// JMS 2.0 – note how the API hides everything about the broker
try (JMSContext ctx = factory.createContext()) {
    Queue q = ctx.createQueue("ORDERS");
    ctx.createProducer().send(q, "{\"orderId\":8821}");

    // broker-side filtering – Kafka cannot do this
    JMSConsumer c = ctx.createConsumer(q, "region = 'US' AND amount > 100");
    String msg = c.receiveBody(String.class);
}
```

When to pick it

- You're in a Java/JEE shop and the requirement is literally "JMS-compliant."
- You need **broker-side message selectors** (filtering without shipping data).
- You need XA / two-phase-commit distributed transactions with a database. (Kafka's transactions are Kafka-only — they cannot span your JDBC datasource.)
- Moderate volume, existing operational familiarity.

Strengths / weaknesses

✓	✗
JMS standard compliance, huge Java ecosystem	Classic 5.x throughput is low (~a few thousand/s)
Broker-side selectors, XA transactions	Two divergent products confuses adoption
Both push (<code>MessageListener</code>) and pull	"Network of Brokers" is complex and dated vs partitioning
Artemis is genuinely fast	Mindshare has shifted to Kafka/RabbitMQ
Embeddable in a JVM for tests	No replay, no long retention

2.4 IBM MQ (formerly MQSeries / WebSphere MQ)

Analogy: an armoured bank courier with a signed chain of custody. Slower and expensive, but the message *will* arrive exactly once, and there's a legally auditable paper trail.

Technically

- Commercial, since 1993. Runs everywhere including **z/OS mainframes** — which is the real reason it still matters.
- Concepts: **queue manager** (the broker), **local/remote/alias/transmission queues**, **channels** between queue managers, **MQI** API (plus JMS and AMQP).
- **Store-and-forward** across queue managers: a message hops from QM to QM, each hop persisted, so it survives network partitions between data centres.
- Best-in-class **XA / two-phase commit** integration with CICS, DB2, IMS.

Internally

- Persistent messages are written to a **recovery log** (WAL) before ack, then into queue files — the same durability discipline as a database.
- Clustering distributes messages across queue managers; **uniform clusters** and **native HA** (Raft-based, in newer versions) provide failover.
- Message-level transactional integrity is the design centre. Everything else is subordinate to it.

When to pick it

You usually don't get to pick — it's already there. It's correct when:

- Mainframe integration is required (`z/OS`, CICS, DB2).
- Regulatory/audit requirements demand vendor support with an SLA and a phone number.
- Cross-organisation, cross-datacentre guaranteed delivery over unreliable links.
- Banking/insurance/government core systems where the message *is* the money.

Strengths / weaknesses

✓	✗
Extremely mature, battle-tested since 1993	Expensive licensing
Mainframe + legacy integration nobody else does	Low throughput vs Kafka (thousands/s)
Best XA/2PC transaction support	Heavy, specialist administration (MQ admins are a job title)

✓	✗
Vendor support & compliance story	Not designed for streaming/analytics/replay
Store-and-forward across unreliable WAN	Poor fit for high-volume event data

A very common real architecture: IBM MQ carries the transactions, and a Kafka Connect / MQ-source bridge copies them into Kafka so the analytics, fraud, and ML teams can consume them without touching the core banking system. Knowing this pattern impresses interviewers at banks.

2.5 Amazon SQS (and SNS, and Kinesis)

Analogy: renting mailbox space instead of building a mailroom. You never see the building, you never fix the roof, and you pay per letter.

Technically — SQS

- Fully managed, HTTP-based, **no broker to run**. Effectively infinite scale.
- Two flavours:

	Standard queue	FIFO queue
Ordering	best-effort, not guaranteed	strict, per message group ID
Delivery	at-least-once (duplicates happen)	exactly-once <i>processing</i> within a 5-min dedup window
Throughput	nearly unlimited	300 msg/s (3,000 batched); high-throughput mode raises this

- **Visibility timeout** instead of an ack-connection: a consumer receives a message, the message becomes invisible for N seconds, and the consumer must `DeleteMessage` before the timer expires or it reappears.

```

receive() → message invisible for 30s → DeleteMessage() ✓ gone
      |
      └─ consumer crashed / too slow
          ▼
      message becomes visible again → redelivered
      (after maxReceiveCount → Dead Letter Queue)

```

- **Long polling** (`WaitTimeSeconds=20`) avoids the cost and latency of tight polling loops.
- Max message size 256 KB; max retention **14 days**; delay queues up to 15 min.

SNS vs SQS vs Kinesis vs Kafka — the AWS mapping

Need	AWS service	Kafka equivalent
Work queue, one consumer per message	SQS	consumer group
Fan-out notification to many endpoints	SNS	topic with many groups
Durable fan-out with replay, ordered shards	Kinesis Data Streams	Kafka topic + partitions
Managed Kafka itself	MSK / Confluent Cloud	Kafka

The classic AWS pattern is SNS → several SQS queues ("fan-out pattern"), which manually reconstructs what a Kafka topic with multiple consumer groups gives you natively.

When to pick SQS

- Small/medium team with no ops appetite, already on AWS.
- Decoupling Lambda functions or ECS tasks; native Lambda event-source mapping.
- Bursty, unpredictable volume — SQS absorbs spikes without capacity planning.
- You need per-message delay, and DLQ configuration with two clicks.

Strengths / weaknesses

✓	✗
Zero operations, zero capacity planning	AWS lock-in
Effectively infinite scale (standard)	Max 14-day retention; no replay after delete
Built-in DLQ, delay, visibility timeout	No true pub/sub (needs SNS in front)
Pay-per-request	FIFO throughput is limited
Very high availability by default	Per-message cost gets painful at Kafka volumes
	Higher per-message latency (HTTP) than a local broker

2.6 Apache Kafka

Covered in depth in Parts 4–13, but for the comparison table:

- **Model:** partitioned, replicated, append-only log. Pull-based consumers.
- **Ordering:** strict within a partition; none across partitions.
- **Delivery:** at-least-once by default; exactly-once *within Kafka* via idempotent producer + transactions; at-most-once if you commit before processing.
- **Retention:** time or size based, independent of consumption. Days, weeks, or forever (tiered storage in 3.6+ pushes old segments to S3).
- **Scale:** partitions are the unit of parallelism; add partitions and consumers together. Single clusters at large companies handle trillions of messages/day.
- **Throughput:** hundreds of thousands to millions of messages/s per cluster; ~100 MB/s+ per broker is unremarkable.
- **Consumer parallelism ceiling:** one partition is consumed by at most one consumer per group.
Partitions cap your parallelism — this has no analogue in RabbitMQ/SQS, where you just add consumers.

That last point is the most important operational difference and a very common interview question. See Part 11.

2.7 Apache Pulsar (worth 30 seconds, because interviewers ask)

Pulsar deserves a mention because "why not Pulsar?" is a senior-level question.

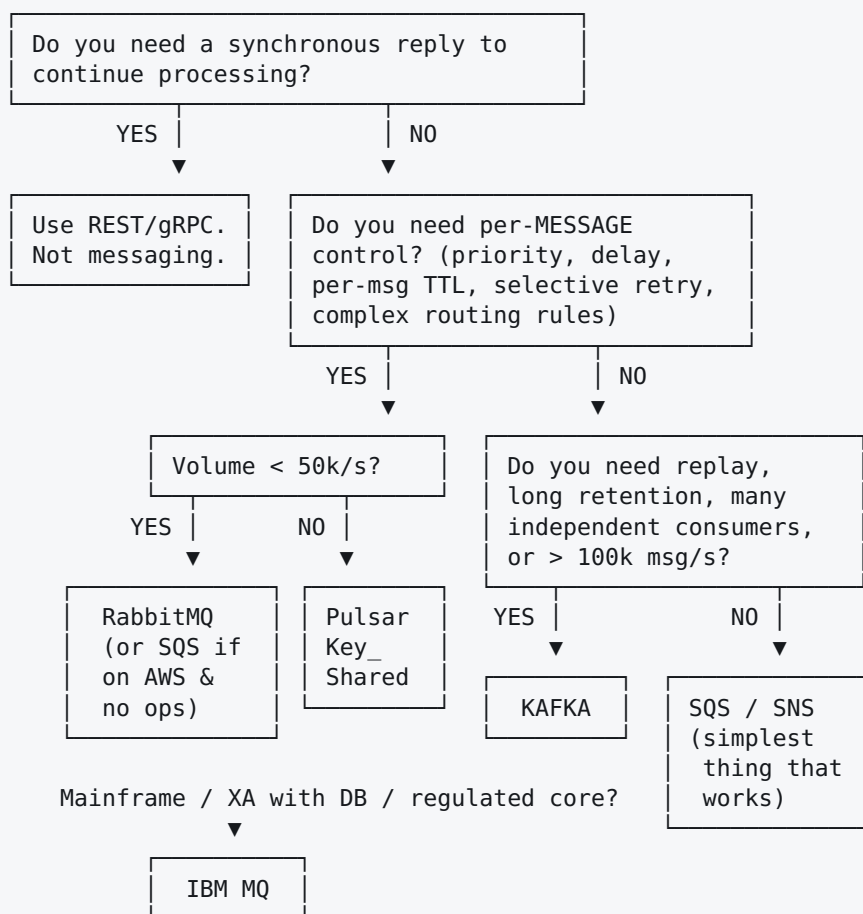
- **Segregated storage and serving:** brokers are stateless; storage is Apache BookKeeper. This makes scaling and rebalancing cheaper (no data movement when adding a broker) but adds a whole second distributed system to operate.
- **Multi-tenancy and geo-replication** are first-class, not bolted on.

- Supports **both** subscription models: `Exclusive / Failover` (log-like) and `Shared / Key_Shared` (queue-like, per-message ack). So Pulsar can do *per-message ack* and *unlimited consumers per partition* — genuinely a capability Kafka lacks.
- **Why Kafka usually still wins:** ecosystem. Kafka Connect, Kafka Streams, ksqldb, Debezium, every cloud vendor, every monitoring tool, and every candidate's CV. Operational familiarity beats architectural elegance.

2.8 The comparison table

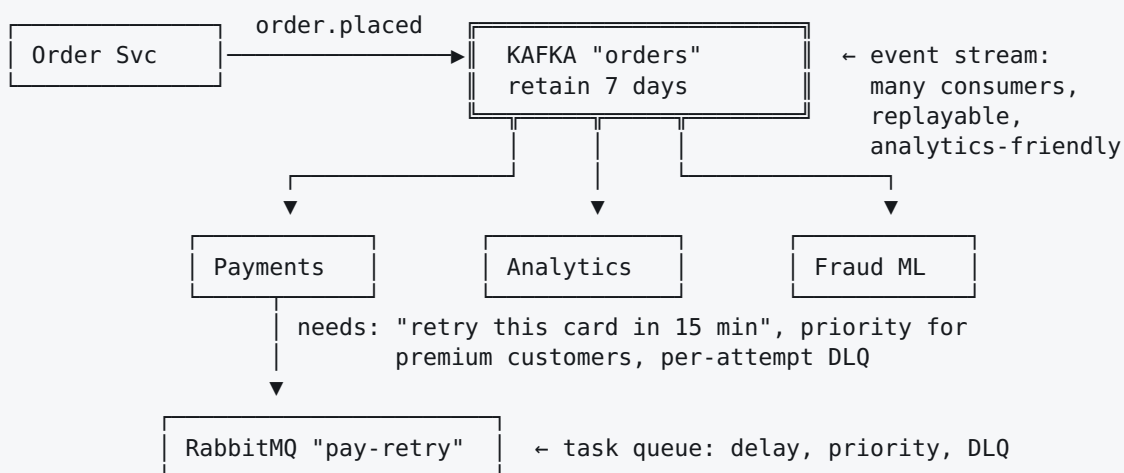
	Kafka	RabbitMQ	ActiveMQ (Artemis)	IBM MQ	Amazon SQS
Model	partitioned log	queue + exchange routing	JMS queue/topic	queue manager	managed queue
Storage	append-only log files	memory-first + disk paging	journal + paging	recovery log + queue files	AWS-managed
Broker state per message	none	yes	yes	yes	yes
Consuming deletes?	no	yes (on ack)	yes	yes	yes (on delete)
Replay history	yes	no (Streams: yes)	no	no	no
Retention	days/weeks/∞	until consumed	until consumed	until consumed	≤ 14 days
Throughput	very high (100k–1M+/s)	medium (10–50k/s)	medium	low-medium	high (standard)
Latency (p99)	2–20 ms	1–5 ms (lowest)	1–10 ms	5–50 ms	20–100 ms
Ordering	strict per partition	per queue (single consumer)	per queue	per queue	FIFO queues only
Priority messages	✗	✓	✓	✓	✗
Per-message TTL	✗ (per-topic only)	✓	✓	✓	✓
Delayed delivery	✗	✓ (plugin)	✓	✓	✓ (15 min)
Complex routing	✗ (topic only)	✓✓ (exchanges)	✓ (selectors)	✓	✗
Broker-side filtering	✗	headers exchange	✓ SQL selectors	✓	✗
Native DLQ	✗ (do it in code / Spring)	✓	✓	✓	✓
Push or pull	pull	push	both	both	pull (HTTP)
Fan-out to many readers	✓✓ consumer groups	needs N queues	durable subs	needs config	needs SNS
Exactly-once	✓ within Kafka (txns)	✗ (at-least-once)	via XA	✓ XA	FIFO, 5-min window
XA / 2PC with a DB	✗	✗	✓	✓✓	✗
Stream processing	✓ Streams/ksqlDB	✗	✗	✗	✗ (use Kinesis/Flink)
Ops burden	high	medium	medium	high (specialists)	none
Consumer scale limit	# partitions	unlimited	unlimited	unlimited	unlimited
Protocol	custom binary TCP	AMQP/MQTT/STOMP	AMQP/OpenWire/JMS	MQI/JMS/AMQP	HTTPS
Written in	Java/Scala	Erlang	Java	C/C++	—

2.9 Decision flowchart



2.10 Using both is normal

Do not present this as a religious war. Large systems run several.



Rule of thumb: Kafka for **events** (facts about the past, many readers). RabbitMQ/SQS for **commands/tasks** (work for one worker, with scheduling and priority). This distinction is developed properly in Part 3.

Summary

- Two families: **smart broker / dumb consumer** (queue-based: RabbitMQ, ActiveMQ, IBM MQ, SQS) and **dumb broker / smart consumer** (log-based: Kafka, Pulsar, Kinesis).
- Per-message broker state is the source of both RabbitMQ's power (priority, delay, selective retry) and its throughput ceiling.
- One offset per group per partition is the source of both Kafka's throughput and replay, and its inability to do priority or per-message delay.
- **RabbitMQ**: richest routing, lowest latency, task queues. **ActiveMQ**: JMS standard, broker-side selectors, XA. **IBM MQ**: mainframe, regulated, guaranteed delivery. **SQS**: zero ops, infinite scale, 14-day cap. **Kafka**: throughput, retention, replay, many consumers, stream processing.
- Kafka's unique operational constraint: **consumer parallelism is capped by partition count**.
- Kafka for events, queues for tasks. Running both is a sign of maturity, not indecision.

Interview Questions

Q1. Kafka vs RabbitMQ — the real answer. They have different broker architectures. RabbitMQ keeps per-message state, so it can do priority, per-message TTL, delayed delivery, selective ack and rich exchange-based routing — and pays for it with a throughput ceiling around tens of thousands of messages/second and degradation on deep backlogs, plus no replay since ack destroys the message. Kafka keeps only a committed offset per consumer group per partition, so it does sequential I/O and zero-copy reads at hundreds of thousands to millions of messages/second with days of retention and full replay — and gives up priority, per-message delay and fine-grained routing. Choose RabbitMQ for task queues needing per-message control; Kafka for high-volume event streams with multiple independent consumers and replay.

Q2. Why can't Kafka do message priority? Priority requires reordering, and Kafka's storage *is* the order — an immutable offset-indexed append-only file. Reordering would break offsets, which every consumer's position depends on. The usual workaround is separate topics per priority tier and a consumer that polls the high-priority topic more aggressively — which is approximate, not real priority.

Q3. Can RabbitMQ replay messages? Classic queues, no — ack deletes. RabbitMQ **Streams** (3.9+) added an append-only replayable log with offset-based consumption, explicitly Kafka-like. For a greenfield high-volume streaming use case Kafka is still the safer choice because of ecosystem maturity.

Q4. In Kafka, why can't I just add 100 consumers to speed up a 10-partition topic? Within a consumer group, each partition is assigned to at most one consumer. With 10 partitions, consumers 11–100 sit idle. Partition count is your parallelism ceiling — plan it up front, because you can add partitions but never remove them, and adding them breaks key→partition affinity for existing keys.

Q5. SQS vs Kafka? SQS is a managed queue: zero ops, effectively infinite scale, per-message delay and native DLQ, but max 14-day retention, no replay after deletion, no true pub/sub without SNS, and per-message pricing that becomes expensive at Kafka volumes. Kafka gives replay, long retention, multiple consumer groups, ordering per partition and a stream-processing ecosystem, at the cost of running a cluster (or paying MSK/Confluent). Pick SQS for simple decoupling on AWS; Kafka for event streaming and analytics.

Q6. What is a visibility timeout and what's the Kafka equivalent? In SQS, a received message becomes invisible for N seconds; if the consumer doesn't delete it in time it reappears. Kafka has no equivalent — the nearest analogue is `max.poll.interval.ms`: if you don't call `poll()` again within that window, the broker assumes you're dead, kicks you out of the group and rebalances your partitions to someone else, who reprocesses from the last committed offset.

Q7. Which system would you use for "send the invoice email, retry with backoff up to 5 times, then give up"? RabbitMQ or SQS — native retry/delay/DLQ. In Kafka you build it: a retry topic per delay tier

plus a DLT, or Spring Kafka's `DefaultErrorHandler` + `DeadLetterPublishingRecoverer`. Doable, but it's application code, not broker config.

Q8. Why does RabbitMQ degrade with a large backlog while Kafka doesn't care? RabbitMQ is memory-first: queue indexes and message bodies want to be in RAM. A deep backlog forces paging to disk plus random I/O to serve it, and the per-message state table stops fitting in memory. Kafka's backlog *is* its normal storage format — reading old data is a sequential read of an older segment file, the same operation as always, just missing the page-cache hit.

Q9. Does Kafka support exactly-once? Exactly-once *within Kafka*: yes — idempotent producer (dedup by producer ID + sequence number per partition) plus transactions (atomic writes across partitions including the offset commit) plus `isolation.level=read_committed` on the consumer. This makes read→process→write pipelines exactly-once. It does **not** extend to external systems: writing to Kafka and to PostgreSQL is not atomic. For that you need an idempotent sink or the transactional-outbox pattern.

Q10. When would you choose IBM MQ over Kafka in 2026? Mainframe integration (z/OS, CICS, DB2), XA two-phase commit spanning the broker and a database, or regulatory requirements demanding commercial support with an SLA. Common real design: IBM MQ for the core transactional path, bridged into Kafka via Kafka Connect so analytics and ML teams consume without touching the core system.

Q11. What's Pulsar's architectural advantage and why is Kafka still dominant? Pulsar separates stateless brokers from BookKeeper storage, so scaling doesn't move data, and it offers `Key_Shared` subscriptions giving per-message ack and consumer counts beyond the partition count. Kafka wins on ecosystem — Connect, Streams, Debezium, every cloud managed service, every monitoring tool, and hiring pool — and on operating one distributed system instead of two.

Q12. Push vs pull: why did Kafka choose pull? Pull lets the consumer control its own rate (natural back-pressure — a slow consumer just polls less, it can't be overwhelmed), enables efficient batching (fetch whatever has accumulated), and means the broker keeps no delivery state. The cost is potential busy-waiting on an empty topic, which Kafka solves with `fetch.min.bytes` + `fetch.max.wait.ms` long-polling.

Mini Quiz

1. Which broker keeps per-message state, and what capability does that enable?
2. Your Kafka topic has 6 partitions and your consumer group has 10 instances. How many are doing work?
3. You need "deliver this message 2 hours from now." Kafka or RabbitMQ?
4. Which RabbitMQ exchange type broadcasts to all bound queues?
5. Max retention in SQS?
6. Which two systems support XA transactions with a relational database?
7. Kafka: push or pull? Name one advantage of that choice.
8. Which system has the *lowest* p99 latency, and why isn't that Kafka?
9. What is the Kafka equivalent of an SQS visibility timeout?
10. Name two things a RabbitMQ Stream and a Kafka topic have in common.

ANSWERS

1. RabbitMQ (also ActiveMQ, IBM MQ, SQS). It enables priority, per-message TTL, delayed delivery, selective ack/redelivery, and broker-side DLQ.
2. **6**. The other 4 are idle — one partition per consumer per group, max.
3. RabbitMQ (delayed-message plugin or TTL + DLX). SQS also works for ≤ 15 min.
4. `fanout`.
5. 14 days.

6. ActiveMQ and IBM MQ. (Kafka transactions are Kafka-only.)
7. Pull. Advantages: consumer-controlled rate = natural back-pressure; efficient batching; no delivery state on the broker.
8. RabbitMQ, ~1-5 ms p99. Kafka optimises for *throughput* via batching (`linger.ms`), which trades a little latency for a lot of throughput; it's also durability-first (replication acks).
9. `max.poll.interval.ms` — exceed it and you're evicted from the group and your partitions are reassigned.
10. Both are append-only, offset-addressed, replayable logs with retention independent of consumption.

Practice Exercises

1. **Run both.** `docker compose up` a RabbitMQ and a Kafka. Publish 1,000 messages to each; consume them; then consume *again from the beginning*. Observe that only Kafka can.
2. **Feel the backlog.** Publish 5 million small messages to a RabbitMQ queue with no consumer, watching memory. Do the same to Kafka, watching disk. Note which one gets unhappy.
3. **Emulate Kafka in RabbitMQ.** Make one publish reach three independent teams, each processing at its own speed, using only RabbitMQ. Count the moving parts versus three Kafka consumer groups.
4. **Emulate RabbitMQ in Kafka.** Implement priority: two topics (`tasks.high`, `tasks.low`) and a consumer that drains high first. Write down why this isn't real priority (hint: what if `high` is never empty?).
5. **Write the decision doc.** One page: "why we chose X" for a system you know. Include throughput numbers, retention needs, consumer count, and ordering requirements. This is the actual artefact senior engineers produce.
6. **Cost model.** For 1 billion messages/month, price SQS (per-request) against MSK/Confluent (per-broker-hour + storage). The crossover point is a real number you should know.

Common Mistakes

- **"Kafka is just a faster RabbitMQ."** They're different data models. Saying this in an interview reveals you've only used one.
- **Choosing Kafka for a low-volume task queue** and then reimplementing priority, delay, and DLQ in application code. You've built a worse RabbitMQ.
- **Choosing RabbitMQ for high-volume analytics events** and discovering at 3 a.m. that a 40-million-message backlog has pushed the broker into paging.
- **Ignoring the partition ceiling.** Autoscaling a Kafka consumer deployment to 50 pods on a 6-partition topic. 44 pods burn money doing nothing, and every scale event triggers a rebalance.
- **Expecting Kafka transactions to cover your database.** They're Kafka-only. Use the outbox pattern.
- **Using SQS standard queues and assuming ordering.** Standard queues are explicitly best-effort ordering *and* at-least-once — you will see out-of-order duplicates in production.
- **Comparing throughput without message size.** "1M msg/s" for 100-byte records and "50k msg/s" for 100 KB records are the same MB/s. Always ask for MB/s.

Further Reading

- *RabbitMQ vs Kafka* — Pivotal/VMware's own comparison series; refreshingly honest about where each loses.
- RabbitMQ docs: "Quorum Queues" and "Streams" — see them adopting Raft and logs.
- Apache ActiveMQ Artemis documentation: "Persistence" chapter (journal + paging).
- AWS docs: "SQS FIFO queues" and the SNS→SQS fan-out pattern.
- Apache Pulsar docs: "Concepts and Architecture" — especially subscription types.
- Martin Kleppmann, *DDIA* Ch. 11, "Message brokers compared to databases".

Part 3 — Event-Driven Architecture

Kafka is a tool. Event-driven architecture (EDA) is the way of thinking that makes the tool worth having. Teams that adopt Kafka without adopting this mindset end up with "REST over Kafka" — all the operational cost, none of the benefit.

3.1 What is an event?

Analogy

An event is a **receipt**, not an **order slip**.

Walk into a coffee shop:

- You say "*one espresso, please.*" → that's a **command**. It's an instruction, directed at a specific person, and it might be refused.
- The barista hands you a **receipt**: "*Espresso, ₹180, 09:42, till 3.*" → that's an **event**. It's a statement of fact about something that already happened. It's past tense. It cannot be refused, argued with, or rejected — it can only be *recorded*. Anyone who reads it learns the same thing.

The receipt has no opinion about what should happen next. Accounting uses it for revenue. Inventory uses it to decrement beans. The loyalty program uses it for points. The barista didn't know or care about any of them.

Technically

An **event** is an immutable record of a fact that occurred at a point in time.

Four properties, all load-bearing:

Property	Meaning	Consequence
Immutable	it happened; you can't un-happen it	corrections are new events, never edits
Past tense	<code>OrderPlaced</code> , not <code>PlaceOrder</code>	naming discipline that prevents coupling
Fact, not instruction	describes what <i>is</i> , not what <i>should be</i>	producer doesn't know or care who reacts
Timestamped	has an occurrence time	enables replay, windowing, time-travel

Internally (in Kafka)

An event is one **record** in a partition:

RECORD (what Kafka actually stores)		
offset	4182	(assigned by broker)
timestamp	1754239320123	(CreateTime or LogAppendTime)
key	"customer-991"	(bytes; drives partitioning)
value	{"orderId":8821,...}(bytes; the payload)	
headers	traceId=abc, schemaVersion=3, eventType=OrderPlaced	
partition	2	

Note what Kafka *doesn't* know: it has no idea this is an "OrderPlaced". To Kafka, key and value are opaque byte arrays. All semantics live in your serialization format and headers. That's why Part 10 (schemas) matters so much — Kafka won't protect you from a producer that changes the shape of the JSON.

Example: a well-formed event

```
{
  "eventId":      "b3f1c2e4-9a7d-4f1e-8c3b-2d5a6e7f8091",
  "eventType":    "OrderPlaced",
  "eventVersion": 3,
  "occurredAt":   "2026-08-03T09:42:00.123Z",
  "producer":     "order-service@2.14.1",
  "traceId":      "4bf92f3577b34da6a3ce929d0e0e4736",

  "data": {
    "orderId":     "8821",
    "customerId":  "customer-991",
    "items": [
      { "sku": "SH0E-42-BLK", "qty": 1, "unitPriceMinor": 899900 }
    ],
    "totalMinor":  899900,
    "currency":    "INR",
    "shippingAddressId": "addr-5512"
  }
}
```

Design notes that separate juniors from seniors:

- **eventId** — a UUID so consumers can deduplicate. At-least-once delivery means you *will* see this record twice; the consumer needs a way to know.
- **eventVersion** — schema evolution without breaking old consumers.
- **occurredAt** vs Kafka's timestamp — business time vs broker time. They differ when the producer was offline or retrying. Keep both.
- **traceId** — distributed tracing across async hops, otherwise debugging is archaeology. Put it in a **header**, not just the body, so infrastructure can read it without deserializing.
- **totalMinor in minor units** — integers, not floats. `899900` paise, not `8999.00`. Never put currency in a `double`.
- **The event carries enough data to be useful** but references IDs (`shippingAddressId`) for things that are large or change independently. More on this trade-off in §3.6.

Why events exist

Because **facts are shareable and instructions are not**.

If `OrderService` sends the command "charge this card", it has decided that payment must happen, chosen who does it, and coupled itself to that decision. If it publishes the fact "order 8821 was placed", it has decided nothing. Six teams can independently decide what that fact means to them, and a seventh can join next year and replay history to catch up.

Facts are append-only, and append-only things scale.

3.2 Events vs Commands vs Notifications

This is a genuinely important distinction and a strong interview differentiator.

	COMMAND	EVENT	NOTIFICATION	QUERY
Intent	do this	this happened	FYI, go look	tell me
Tense	imperative	past	past	present
Naming	PlaceOrder	OrderPlaced	OrderChanged	GetOrder
Recipients	exactly one	0..N	0..N	one
Sender knows receiver?	YES – a specific one	NO	NO	YES
Can be refused?	YES (reject, validate)	NO – it already happened	NO	NO
Payload	full input	full state change	just an ID + event type	criteria
Coupling	HIGH	LOW	MEDIUM (reader must call back)	HIGH
Typical transport	RabbitMQ / REST / gRPC	KAFKA	Kafka / SNS	REST/gRPC

Command

```
// topic: payment-commands – ONE consumer group is expected to act
{ "command": "CapturePayment",
  "orderId": "8821", "amountMinor": 899900, "currency": "INR",
  "idempotencyKey": "8821-capture-1" }
```

A command has a specific intended handler and expects it to *do something*. Commands can fail validation. They usually need a reply or a status event.

You *can* put commands on Kafka (a `*-commands` topic with one consumer group), and it's a legitimate pattern. But recognise that you've reintroduced coupling: the sender knows a handler exists and expects execution.

Event

```
// topic: orders – N consumer groups, producer knows none of them
{ "eventType": "OrderPlaced", "orderId": "8821", ... }
```

Notification (a.k.a. "event-carried skinny event")

```
// topic: order-notifications
{ "eventType": "OrderUpdated", "orderId": "8821", "version": 7 }
```

Interested parties call `GET /orders/8821` to find out what changed.

- ✓ Tiny payload, no data duplication, no schema-evolution pain, no risk of leaking internal fields.
- ✗ Every consumer must call back → the "thundering herd" problem (10 consumers × 1000 events/s = 10,000 rps on your API), you lose replayability (the API returns *current* state, not state at event time), and you've re-coupled at runtime.

Rule: the notification pattern trades storage for runtime coupling. Use it when the payload is huge (a video file, a 5 MB document) or highly sensitive. Otherwise prefer a fat event.

The naming rule that prevents 80% of design mistakes

If you can't name it in the past tense, it's not an event.

✗ Wrong (command dressed as event)	✓ Right
SendEmail	OrderPlaced (email service decides to send)
UpdateInventory	OrderPlaced / ItemShipped
ProcessPayment	PaymentAuthorised, PaymentFailed
NotifyUser	PasswordChanged
order-service-to-email-service	orders

The left column bakes *your* assumptions about the consumer into the message. The moment a second consumer appears, the name is a lie. That last row is the worst: a topic named after two services can never have a third.

3.3 Producer and Consumer

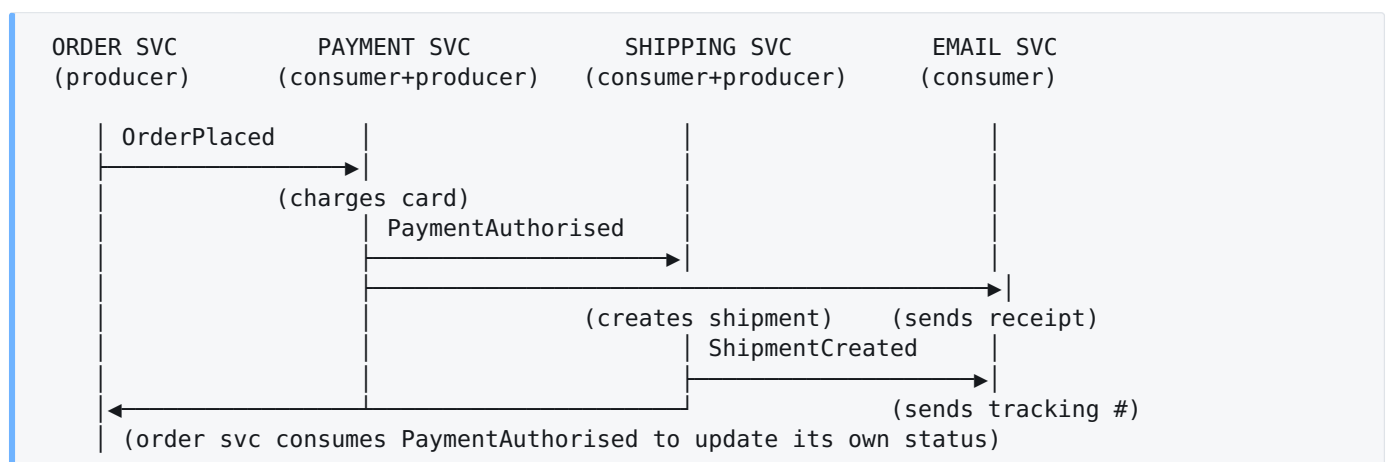
Analogy

A **producer** is a journalist filing a story to the newspaper. A **consumer** is a reader. The journalist doesn't know who reads, how fast they read, whether they read at all, or whether someone reads it 5 years later in the archive.

Technically

- **Producer:** any process that appends records to a Kafka topic. It chooses the topic, the key, the value, and optionally the partition. It gets back an acknowledgement containing the assigned partition and offset.
- **Consumer:** any process that reads records from one or more topic partitions, starting at a chosen offset, and periodically records its progress ("committing an offset").

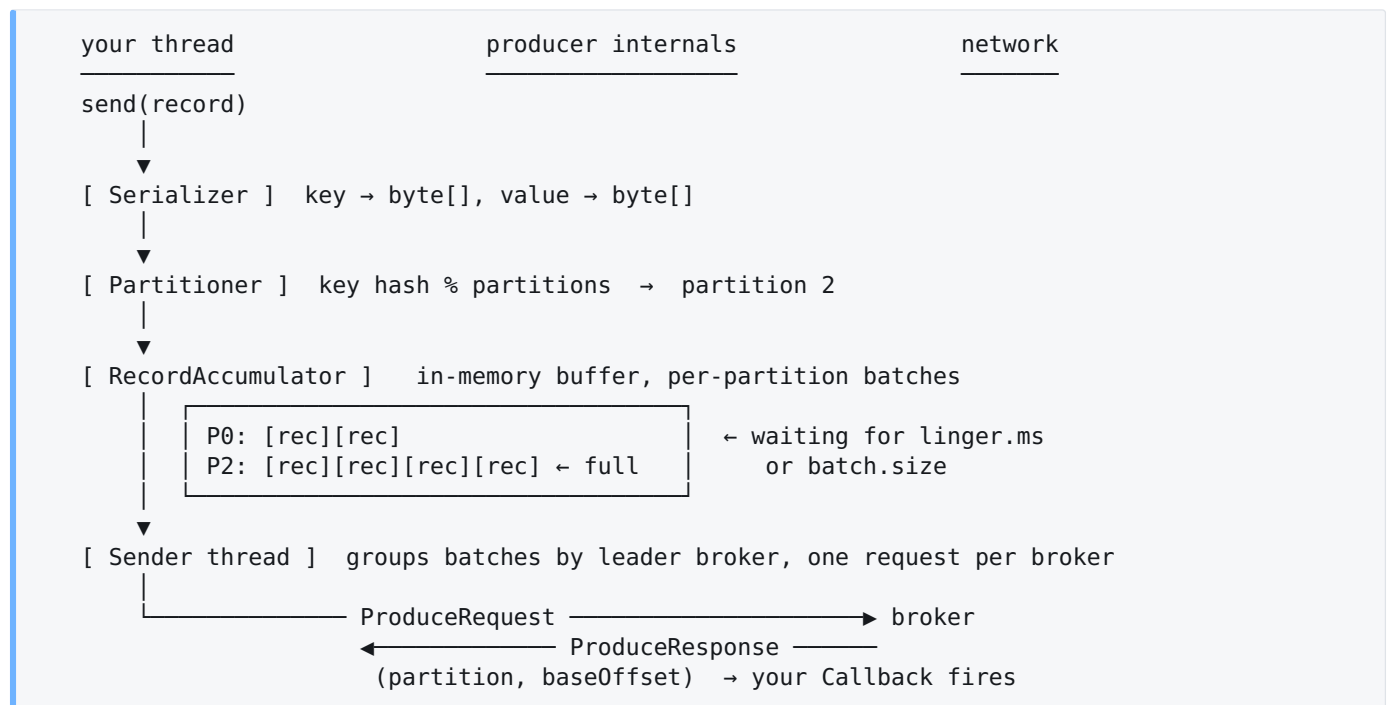
Critically: **a service is usually both**. That's how event chains form.



Every arrow is a topic. Nobody calls anybody. This is a **choreography** — no central coordinator. Compare with **orchestration** in §3.7.

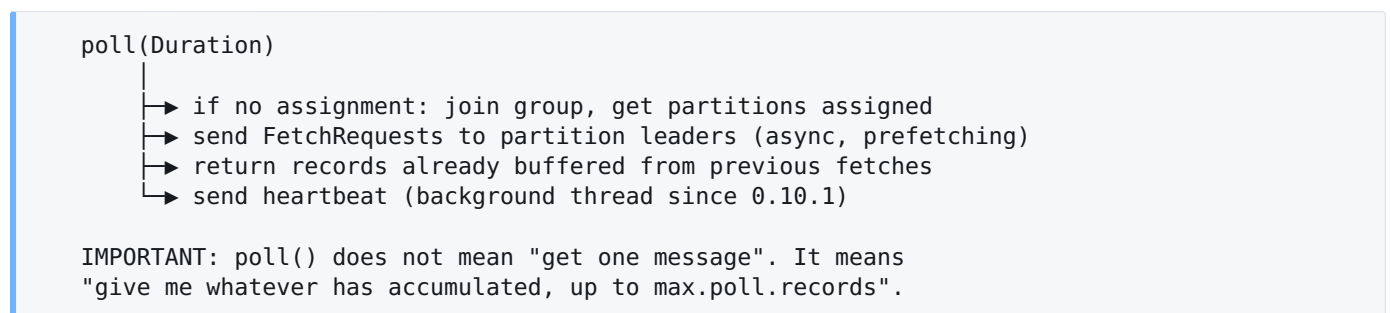
Internally (preview of Parts 8-9)

The producer is not a thin wrapper around a socket:



Two things to internalise now: **`send()` is asynchronous** (it returns a `Future` after buffering, not after the broker stores it), and **records are sent in batches**, which is where most of Kafka's throughput comes from.

The consumer side:



3.4 REST vs Events

The comparison

	REST / gRPC (synchronous)	Events (Kafka)
Coupling	caller knows callee's address & API	both know only the topic + schema
Availability	callee must be up	callee can be down for hours
Latency of the caller	sum of all downstream latencies	one local buffer append (~µs)
Failure	propagates upward as 5xx	isolated to the failing consumer
Adding a consumer	change the producer & redeploy	zero producer changes
Ordering	n/a	strict per partition
Replay / audit	no (unless you log it)	yes, built in

	REST / gRPC (synchronous)	Events (Kafka)
Debugging	easy — one stack trace	hard — distributed, async
Consistency	can be strong (read-after-write)	eventual
Getting an answer	immediate	never (or via a response event)
Backpressure	429/503, caller must retry	consumer lag; producer unaffected
Team autonomy	low	high

The same feature, both ways

Requirement: when an order is placed, reserve inventory and email the user.

REST version:

```
@Transactional
public OrderResponse place(OrderRequest req) {
    Order order = repo.save(new Order(req));           // 5 ms
    inventoryClient.reserve(order);                     // 90 ms — can 500
    emailClient.sendConfirmation(order);                 // 300 ms — can 500
    return new OrderResponse(order.getId());           // total ~400 ms
}
```

Problems, in order of severity:

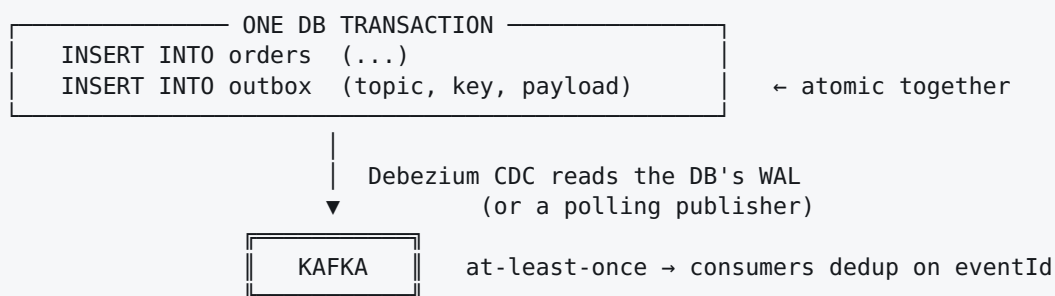
1. **@Transactional around network calls.** Your DB transaction is open for 400 ms holding row locks while you wait for an SMTP server. Under load this exhausts the connection pool and takes down the database.
2. **Email failure fails the order.** A non-critical side effect has veto power over revenue.
3. **If inventory succeeds and email fails, and you roll back** — inventory is now inconsistent. If you *don't* roll back, the customer has an order and no email. Either way you've built a distributed transaction with no coordinator.
4. Latency is the sum, and it grows every time someone adds a call.

Event version:

```
@Transactional
public OrderResponse place(OrderRequest req) {
    Order order = repo.save(new Order(req));
    outbox.save(new OutboxEvent("orders", order.getId(),
                                toJson(new OrderPlaced(order)))); // same txn!
    return new OrderResponse(order.getId());                       // ~6 ms
}
// A separate poller (or Debezium CDC) publishes outbox rows to Kafka.
```

- Order endpoint: **6 ms**, one failure mode (the DB).
- Inventory reserves when it can. Email sends when it can.
- Email down for 2 hours ⇒ 2 hours of emails send late. Orders never stop.
- Loyalty team joins next quarter ⇒ new consumer group, no changes here.

Note the **transactional outbox** pattern in that snippet. It's the standard answer to "*how do you atomically update your database and publish to Kafka?*" You can't — those are two systems with no shared transaction. So you write the event into a table **in the same DB transaction** as the state change, and a separate process ships outbox rows to Kafka. This is the single most valuable pattern in this Part; expect it in senior interviews.

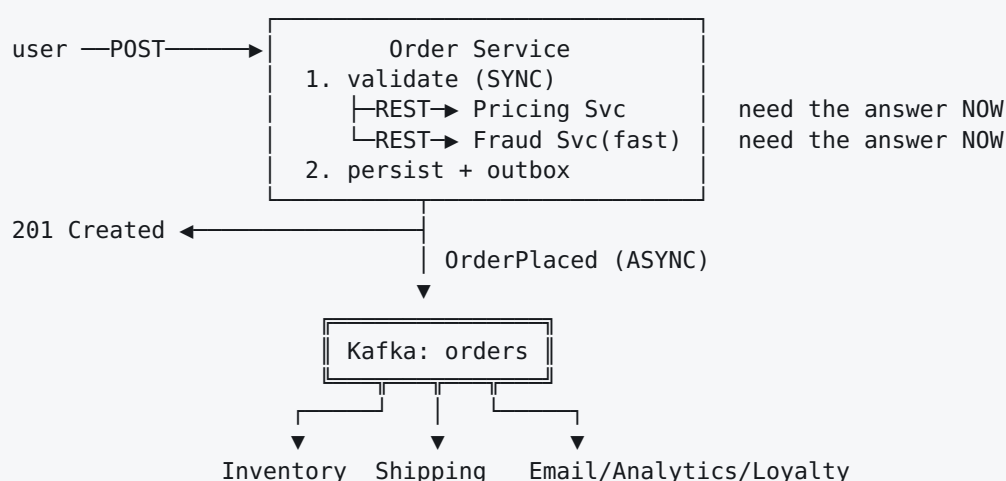


What events cost you — be honest about this

An interviewer will respect you far more for volunteering these than for evangelising.

1. **Eventual consistency.** `POST /orders` returns 201, then the user hits `GET /orders/8821` on a read model that hasn't caught up and sees nothing. You must design for this: return the full object from the write, use read-your-writes routing, or show an explicit "processing" state.
2. **Debugging is harder.** No stack trace crosses a topic. Without correlation IDs propagated through headers and distributed tracing, "why didn't this user get their email?" becomes a multi-hour log excavation.
3. **No transactions across services.** Two-phase commit is off the table. You need **sagas** with compensating events (§3.7).
4. **Duplicates are guaranteed, not hypothetical.** At-least-once delivery means every consumer must be idempotent. This is real work.
5. **Schema evolution becomes a contract problem.** Ten teams consume your event. Renaming a field is now a migration project. (Part 10.)
6. **Ordering is only per-partition.** If two related events land in different partitions, they can be processed out of order. (Part 11.)
7. **Cognitive load.** Every developer on the team must understand offsets, rebalancing, lag, and idempotency. That's a real hiring and training cost.

The right answer: use both



Heuristic: "Do I need the answer to decide what to do next?" Yes → synchronous call. No → publish an event.

Validation, pricing, auth, and inventory *availability checks* need answers. Notifications, analytics, projections, downstream workflows, and audit do not.

3.5 Why large companies prefer events

Six reasons, roughly in order of how much they actually matter.

1. Organisational scaling (Conway's Law), not technical scaling. This is the real reason, and the one candidates rarely say. With REST, adding a consumer of order data requires a conversation, a ticket, an API change, a code review, and a deploy in *someone else's* repository. With events, a team subscribes to a topic and ships on Monday. Events let 200 teams move independently — that's worth more than any throughput number.

2. Failure isolation. A synchronous call graph fails together. In Netflix's or Uber's architecture, a non-critical service outage must not stop the critical path. Events make "degraded" a first-class state: emails queue up, analytics falls behind, recommendations get stale — and nothing customer-critical breaks.

3. Replay = a new superpower. - Your fraud model had a bug for 3 days → fix, reset the consumer group offset to 3 days ago, reprocess. - A brand-new "Customer Lifetime Value" service needs 2 years of history → read the compacted topic from offset 0. - QA wants a realistic load test → replay yesterday's production traffic into staging. None of this is possible with REST unless you separately built an event log — at which point you've built Kafka.

4. One integration point instead of N^2 . LinkedIn's original motivation. Each system integrates with Kafka once, not with every other system.

5. Real-time analytics for free. Because operational data flows through Kafka anyway, the analytics/ML/monitoring consumers are the *same* stream, seconds behind — not a nightly batch ETL job that reads a production database replica.

6. Audit and compliance. The log *is* the audit trail: what happened, in what order, when, produced by whom. In finance and healthcare this converts from "nice" to "required."

The honest counter-argument

Events are also how you get a system nobody understands. If every service both produces and consumes and there's no catalogue of topics, no schema registry, and no tracing, you have a **distributed monolith with worse debugging**. The companies that succeed with EDA invest heavily in:

- a **schema registry** with enforced compatibility (Part 10),
- an **event catalogue** — who produces what, who consumes it,
- **distributed tracing** through headers,
- strong **naming and versioning conventions**,
- **idempotency by default** in every consumer.

Mentioning these in an interview signals that you've actually operated one.

3.6 Fat events vs thin events (the classic design argument)

THIN / NOTIFICATION
`{ "orderId": "8821" }`

consumer must call back
 — runtime coupling —

FAT / EVENT-CARRIED STATE TRANSFER
`{ "orderId":"8821", "customerId":"991",
 "items":[...], "totalMinor":899900,
 "shippingAddress":{"...} }`

consumer is self-sufficient
 — schema coupling —

	Thin	Fat
Payload size	tiny	larger
Consumer needs an API call	yes	no
Works if producer is down	✗	✓
Replay gives historical state	✗ (API returns <i>now</i>)	✓
Schema evolution pain	low	higher
Risk of leaking internal model	low	high
Load on producer's API	N consumers × rate	zero

Default to fat events, with discipline:

- Publish a **deliberate public contract**, not your JPA entity. Never `objectMapper.writeValueAsString(orderEntity)` — you've just made your database schema a public API, and adding a column becomes a breaking change for ten teams.
- Include everything a consumer plausibly needs to act *without calling back*.
- Reference by ID only for large blobs or PII you don't want copied.
- Version the schema and use a registry.

3.7 Choreography, orchestration, and sagas

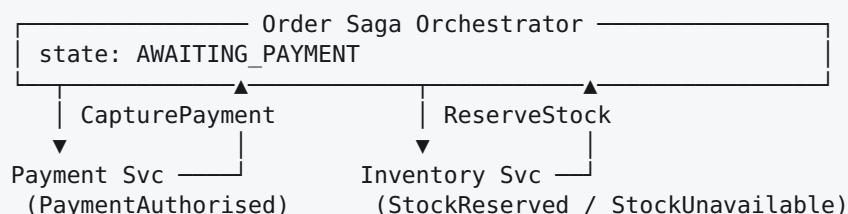
Since there are no distributed transactions, how do you handle "order placed but payment failed"?

Choreography (each service reacts; no coordinator)

```
OrderPlaced → Payment —PaymentFailed→ Order Svc (marks CANCELLED)
                                     ↳ Inventory (releases reservation)
```

- ✓ Maximum decoupling, no single point of failure, easy to add participants.
- ✗ No single place shows the workflow. Understanding "what happens after an order" requires reading eight repositories. Cyclic dependencies sneak in.

Orchestration (a coordinator sends commands and listens for events)



- ✓ The workflow is explicit, in one place, testable, and observable. Timeouts and compensation are easy to reason about.
- ✗ The orchestrator is a new service to run, and it can become a god object.

Compensating events (the "undo")

You cannot roll back. You can only emit a new fact that reverses the effect:

Forward	Compensation
StockReserved	StockReleased
PaymentAuthorised	PaymentRefunded
ShipmentCreated	ShipmentCancelled
PointsAwarded	PointsRevoked

Note these are *business* operations, not technical rollbacks — a refund is a real event with its own audit trail, which is usually what the business actually wants anyway.

Practical guidance: use choreography for simple, 2-3 step flows (most of them). Use orchestration when the flow has ≥ 4 steps, timeouts, or compensations — i.e. checkout, KYC onboarding, loan approval. Mixing both in one system is normal.

3.8 Event sourcing $\neq$ event-driven (don't conflate these)

A very common interview trap.

	Event-driven architecture	Event sourcing
What it is	services communicate via events	events are the only source of truth
State storage	services keep normal DB tables	state is <i>derived</i> by replaying events
Scope	integration between services	persistence inside one service
Difficulty	moderate	high
Kafka's role	the transport	the store (needs infinite retention/compaction)

You can absolutely do event-driven architecture with plain old CRUD databases inside each service — and 95% of teams should. Event sourcing means `OrderService` has no `orders` table; it rebuilds order state from `OrderPlaced`, `ItemAdded`, `OrderCancelled`. Powerful (perfect audit, time-travel queries), but you inherit snapshotting, schema versioning of historical events *forever*, and the fact that you can never query "all orders over ₹10,000" without a projection.

Interview line: "Event-driven architecture is about how services communicate. Event sourcing is about how one service persists state. Kafka enables both, but they're independent decisions, and most systems should do the first without the second."

Summary

- An **event** is an immutable, past-tense, timestamped record of a fact. Commands are instructions (one recipient, reusable); events are facts (N recipients, not reusable); notifications are IDs that force a callback.
- If you can't name it in the past tense, it's not an event.
- To Kafka, a record is just key/value bytes + headers + offset + timestamp. All meaning lives in your schema — hence the schema registry.
- REST when you **need the answer to proceed**; events when you don't.
- Events buy decoupling, failure isolation, replay, and organisational autonomy. They cost eventual consistency, harder debugging, mandatory idempotency, saga complexity, and schema governance.

- You cannot atomically write to your DB and Kafka. Use the **transactional outbox** (+ Debezium CDC), then dedup downstream on `eventId`.
- Prefer **fat events** carrying a deliberate public contract — never your ORM entity.
- No 2PC ⇒ **sagas** with compensating events. Choreography for simple flows, orchestration for complex ones.
- **Event-driven ≠ event sourcing.**

Interview Questions

Q1. Difference between an event and a command? A command is an imperative instruction with one intended handler that can reject it (`PlaceOrder`); the sender knows a handler exists and expects execution. An event is a past-tense immutable fact with zero-to-many interested parties (`OrderPlaced`); the producer knows nothing about consumers and the fact cannot be refused. Commands couple sender to receiver; events don't.

Q2. How do you atomically update your database and publish an event? You can't — two systems, no shared transaction. Options: (a) **transactional outbox** — write the event to an outbox table in the same DB transaction, then a poller or Debezium CDC publishes it to Kafka; (b) **CDC directly** off the WAL, deriving events from row changes; (c) **event sourcing** — Kafka *is* the store, so there's only one write. Outbox is the pragmatic default. All are at-least-once, so consumers must dedup on `eventId`.

Q3. Why not just use REST between microservices? Covered in Part 1 Q2 — coupling, cascading failure, latency accumulation, no replay, deploy coupling. Add: a `@Transactional` method that makes HTTP calls holds DB locks for the duration of the network call, which is how you take down the database under load.

Q4. What are the downsides of EDA? Eventual consistency (read-after-write anomalies visible to users), hard debugging without correlation IDs and tracing, no cross-service transactions (sagas + compensating events instead), guaranteed duplicates requiring idempotent consumers, schema evolution as a multi-team contract problem, only per-partition ordering, and higher cognitive load for every developer.

Q5. Fat events or thin events? Fat (event-carried state transfer) by default: consumers are self-sufficient, work when the producer is down, and replay reproduces historical state. Thin notifications force a callback, which re-couples at runtime, creates a thundering herd against the producer's API, and breaks replay because the API returns current state. Use thin only for huge payloads or PII you don't want copied. Either way, publish a deliberate public contract, not your ORM entity.

Q6. Choreography vs orchestration? Choreography: each service reacts to events, no coordinator — maximally decoupled, but the workflow exists nowhere and is hard to understand or change. Orchestration: a saga coordinator issues commands and tracks state — explicit, testable, easy to add timeouts and compensation, but it's another service and can become a god object. Choreography for 2–3 step flows; orchestration for long flows with timeouts and compensations.

Q7. Event sourcing vs event-driven architecture? EDA is an integration style (services talk via events). Event sourcing is a persistence style (state is derived by replaying an event log; there is no current-state table). They're orthogonal — most teams should do EDA with normal databases and skip event sourcing.

Q8. How do you handle duplicate events? Make consumers idempotent. Practically: (a) natural idempotency — `SET status = 'PAID'` is safe to repeat; (b) a dedup table keyed on `eventId` with a unique constraint, checked in the same transaction as the effect; (c) upserts keyed on business ID; (d) version/sequence checks — ignore events with a version $\leq$ the one already applied. Kafka's exactly-once semantics only help for Kafka-to-Kafka; anything touching an external system needs application-level idempotency.

Q9. How do you debug "the user never got their email" in an EDA? Propagate a `traceId` in Kafka **headers** from the entry point onward, and use distributed tracing (OpenTelemetry) that understands

producer/consumer spans. Then: confirm the event exists in the topic at the expected offset (console consumer with the key), check the email consumer group's lag and last committed offset, check the DLT for that key, and check for a poison-pill deserialization error stalling the partition. Without headers-based correlation you're reading logs by timestamp, which doesn't scale.

Q10. Your event schema needs a breaking change and ten teams consume it. What do you do?

Don't break it. Add the new field as optional with a default (backward compatible) and let consumers migrate. If it's truly breaking: publish a new topic version (`orders.v2`), dual-write both for a deprecation window, migrate consumers one at a time, then retire `v1`. Enforce compatibility mechanically in the Schema Registry so this can't happen accidentally. (Part 10.)

Q11. Should the event contain the full object or the ID? See Q5 — and add: if the object contains PII, consider what it means to have it replicated into ten consumers' databases and Kafka's 7-day retention when a GDPR deletion request arrives. Sometimes thin events are a compliance decision, not a performance one.

Q12. A user places an order and immediately refreshes; the order isn't there. Fix? That's eventual consistency between the write path and a read model. Options: return the created resource directly from the `POST` response; route read-your-own-writes to the primary or pass a version/timestamp the read model must have caught up to; show an explicit `PROCESSING` state so the UI is honest; or keep this particular read synchronous against the owning service. The wrong answer is "add a sleep."

Mini Quiz

1. Which of these are events? `PlaceOrder`, `OrderPlaced`, `SendEmail`, `PaymentFailed`, `UpdateInventory`.
2. Sender knows the receiver: command or event?
3. What pattern solves "write to DB and publish to Kafka atomically"?
4. Name the two downsides of thin/notification events.
5. Event sourcing or EDA: "services communicate asynchronously via Kafka"?
6. Why is `@Transactional` around an HTTP call dangerous?
7. What replaces a distributed transaction rollback in EDA?
8. Where should a `traceId` live — event body, Kafka header, or both? Why?
9. Fat events couple on ; **thin events couple on** .
10. Give the one-sentence heuristic for REST vs event.

ANSWERS

1. Events: `OrderPlaced`, `PaymentFailed`. Commands: `PlaceOrder`, `SendEmail`, `UpdateInventory`.
2. Command.
3. Transactional outbox (optionally with Debezium CDC).
4. Runtime coupling / thundering herd against the producer's API, and loss of replayability (the API returns current state, not state at event time).
5. EDA.
6. The DB transaction — and its row locks and pooled connection — stays open for the whole network round trip, so a slow downstream exhausts the connection pool and stalls the database.
7. Sagas with compensating events (`StockReleased`, `PaymentRefunded`).
8. Both, but the header is the important one: infrastructure, interceptors and tracing agents can read headers without deserializing (or even understanding) the payload.
9. Fat couple on **schema**; thin couple on **runtime availability**.

10. "Do I need the answer to decide what to do next?" Yes → REST. No → event.

Practice Exercises

1. **Rename everything.** Take 10 message/queue names from a system you know. Rewrite each as a past-tense event. Note which ones resist — those are genuinely commands, and that's fine, but now you know.
2. **Draw both.** Pick one user action. Draw the synchronous call graph and the event flow side by side. Annotate each with total latency and the number of ways it can fail.
3. **Design one event properly.** Write the full JSON for `PaymentAuthorised`: `eventId`, `eventType`, `eventVersion`, `occurredAt`, `traceId`, plus a `data` block. Money in minor units. Then list every consumer you'd expect and check each has what it needs *without a callback*.
4. **Build the outbox.** In Spring Boot: an `outbox` table, a `@Transactional` service that writes both rows, and a `@Scheduled` publisher that ships unsent rows to Kafka and marks them sent. Then kill the app between the DB commit and the Kafka send and confirm the event still gets published.
5. **Make a consumer idempotent.** Process the same event twice deliberately. First, observe the double-charge. Then add a `processed_events(event_id)` table with a unique constraint, inserted in the same transaction as the effect, and observe it becoming safe.
6. **Write a saga.** Order → Payment → Inventory → Shipping, where inventory fails. Write out every event including compensations, in order. Then write the same flow as an orchestrator with explicit states and timeouts. Compare which you'd rather debug at 3 a.m.
7. **Find the eventual-consistency bug.** In any EDA you have access to, find one screen where a user can observe stale state right after an action. Propose a fix that isn't a sleep.

Common Mistakes

- **Naming events as commands** (`SendEmailEvent`). It bakes one consumer's purpose into the contract and guarantees the name lies as soon as a second consumer appears.
- **Naming topics after service pairs** (`order-service-to-email-service`). A third consumer can never join without the name being absurd.
- **Publishing your JPA entity.** Your DB schema becomes a public API; adding a column becomes a breaking change for ten teams; you leak internal fields and PII. Always map to an explicit event DTO.
- **Publishing to Kafka inside a DB transaction** (or worse, before committing). If the transaction rolls back you've announced something that never happened — a "phantom event," and impossible to retract. Use the outbox.
- **Assuming exactly-once and skipping idempotency.** You will get duplicates. Design for them from day one; retrofitting idempotency into a payment service is not fun.
- **Doing request/response over Kafka.** Reply topics with correlation IDs to emulate RPC gives you a slow, hard-to-debug RPC. Use REST/gRPC.
- **Events with no versioning and no registry.** Works beautifully for six months, then one producer deploy breaks four consumers in production.
- **Not propagating correlation/trace IDs.** The single biggest predictor of whether an EDA is operable.
- **Conflating event-driven with event sourcing** and accidentally signing up for a much harder project than intended.
- **Making everything asynchronous.** Validation and authorisation need answers. Eventual consistency is a cost you pay per use case, not a global upgrade.

Further Reading

- Martin Fowler, "*What do you mean by 'Event-Driven'?*" — the four patterns (notification, event-carried state transfer, event sourcing, CQRS) untangled. Read this before any interview involving EDA.
- Martin Fowler, "*Event Sourcing*" and Greg Young's talks — for the persistence side.
- Chris Richardson, *Microservices Patterns* — Ch. 3-5 cover messaging, sagas (both flavours) and the transactional outbox with real code. Also microservices.io/patterns.
- Ben Stopford, *Designing Event-Driven Systems* (O'Reilly, free from Confluent) — EDA specifically through a Kafka lens.
- Debezium documentation, "Outbox Event Router" — the production implementation of §3.4.
- Vaughn Vernon, *Implementing Domain-Driven Design*, Ch. 8 ("Domain Events") — for where events come from in the first place.

Part 4 — Kafka Architecture From First Principles

This is the most important chapter in Volume I. Everything in Parts 7–22 assumes you have these ten nouns loaded in memory: **broker, cluster, topic, partition, offset, record, producer, consumer, consumer group, replica**.

We'll build them up in the order a designer would invent them.

4.1 A five-minute history (interviewers do ask)

Year	Event
2010	Built at LinkedIn by Jay Kreps, Neha Narkhede, Jun Rao
2011	Open sourced; NetDB paper published
2012	Apache top-level project
2014	Founders leave to start Confluent
2015	Kafka Connect (0.9), new consumer with server-side group coordination
2016	Kafka Streams (0.10), record timestamps
2017	Exactly-once semantics — idempotent producer + transactions (0.11); the message format v2 that everything since depends on
2018	Confluent's Schema Registry / ksqldb ecosystem matures
2020	KIP-500 published: remove ZooKeeper
2021	KRaft early access (2.8)
2022	KRaft production-ready for new clusters (3.3)
2023–24	Tiered storage (3.6), ZK deprecated (3.5), KRaft-only migration paths mature
2025	Kafka 4.0 — ZooKeeper removed entirely . KRaft is the only mode.

The two dates that matter for interviews: **0.11 (2017)** gave us exactly-once, and **4.0 (2025)** killed ZooKeeper.

4.2 Building Kafka from scratch (derive the architecture)

Step 1: a file

You need a durable, ordered record of facts. The simplest thing that works is a file you only append to.

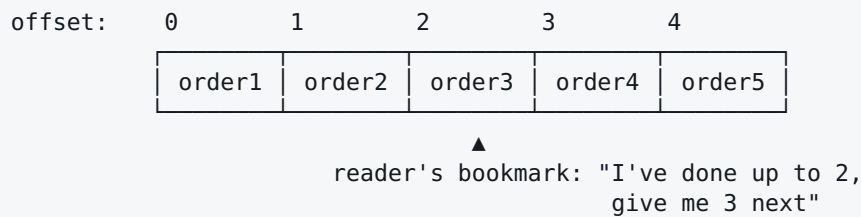
```
orders.log
```

```
{order 1} {order 2} {order 3} {order 4} → append here
```

Appending to a file is *fast* — the disk head never seeks, so you get streaming throughput (hundreds of MB/s even on spinning rust). Reading sequentially is also fast. Good start.

Step 2: addressable positions → the OFFSET

To let a reader resume where it left off, give each record a position number.



Offset = a monotonically increasing, immutable integer identifying a record's position **within one partition**. Assigned by the broker at append time. Never reused, never reordered. `long`, so effectively unbounded.

Crucial: offsets are **per-partition**, not per-topic. "Offset 4182" is meaningless without saying which partition.

Step 3: the file is too big for one machine → PARTITIONS

One file lives on one disk on one machine. That caps you at one machine's throughput and storage. So split the topic into multiple logs.

TOPIC "orders"

Partition 0	offsets: 0 1 2 3 4 5 ... (a file on broker 1)
Partition 1	offsets: 0 1 2 3 ... (a file on broker 2)
Partition 2	offsets: 0 1 2 3 4 5 6 ... (a file on broker 3)

Immediate consequences — memorise these three:

1. **Ordering is per-partition only.** There is *no* global order across a topic. Partition 0's offset 5 and partition 1's offset 5 have no defined relative order. This is the #1 source of production bugs.
2. **Partitions are the unit of parallelism.** N partitions ⇒ at most N consumers doing work in a group.
3. **Each partition has independent offsets**, all starting at 0.

Step 4: keeping related records ordered → the KEY

If ordering is per-partition and you need events for customer 991 in order, they must all land in the *same* partition. That's what the key is for.

```
record.key = "customer-991"
  |
  ▼
partition = murmur2(serialize(key)) % numPartitions
  |
  ▼
always partition 2 → all customer-991 events are strictly ordered ✓
```

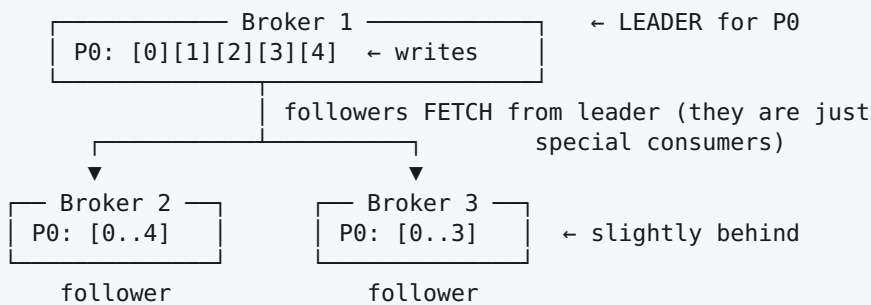
Same key ⇒ same partition ⇒ guaranteed order. No key ⇒ spread across partitions ⇒ no order guarantee between records.

Choosing the key is the single most consequential design decision you make. Wrong key = broken ordering or a hot partition. Part 11 is entirely about this.

Step 5: the machine can die → REPLICATION

One copy is not durable. Keep N copies on N brokers. One is the **leader** (all reads and writes go to it); the rest are **followers** that copy from the leader.

TOPIC "orders", partition 0, replication.factor = 3



If broker 1 dies, one of the in-sync followers is promoted to leader. Clients discover the new leader through metadata and continue. Part 12 covers the mechanics; §4.8 below covers ISR.

Step 6: many machines need coordination → CLUSTER + CONTROLLER

Who decides which broker leads which partition? Who notices broker 1 died? Who stores the list of topics? You need a **control plane**: originally ZooKeeper, now KRaft. §4.9–4.11.

That's the whole architecture. Everything else is optimisation.

4.3 The nouns, precisely

Record (a.k.a. message, event)

RECORD		
key	byte[]	optional. Drives partitioning + compaction. the payload. `null` value = tombstone (deletes the key in a compacted topic).
value	byte[]	
headers	Map	optional metadata: traceId, eventType, __TypeId__
timestamp	long	
		assigned by the broker on append: topic, partition, offset

To Kafka these are **opaque bytes**. It does not know or care that your value is JSON. This is why the broker cannot filter, transform, or validate your data (with the sole exception of compaction, which needs the key).

Topic

Analogy: a folder name; or a table name in a database — a *category* of events.

Technically: a named, ordered-per-partition, append-only stream. Topics are multi-subscriber: 0..N consumer groups may read the same topic independently.

Internally: a topic is not a thing on disk. It's a *logical name* over a set of partition directories:

```
/var/lib/kafka/data/
├── orders-0/           ← topic "orders", partition 0
│   ├── 00000000000000000000.log      ← the actual records
│   ├── 00000000000000000000.index   ← offset → byte position
│   ├── 00000000000000000000.timeindex
│   └── leader-epoch-checkpoint
├── orders-1/
├── payments-0/
└── __consumer_offsets-31/           ← internal topic
```

Naming conventions (matter more than you'd think, because topics are forever):

```
<domain>.<entity>.<event-or-type>[.<version>]
```

```
✓ commerce.orders.placed
✓ commerce.orders           (all order events, discriminated by header)
✓ payments.transactions.v2
✗ orders_topic_final        (why is anything "final")
✗ order-service-to-email    (a third consumer can never join)
✗ Orders                    (case sensitivity + '.'/'_' collision issues)
```

△ Kafka treats `.` and `_` as colliding for metric names — `my.topic` and `my_topic` produce ambiguous metrics. Pick one separator and stick with it.

Partition

Analogy: lanes on a highway. Same road (topic), traffic split across lanes. Cars in one lane stay in order relative to each other; there's no defined order *across* lanes.

Technically: the unit of ordering, parallelism, replication, and storage. An ordered immutable sequence of records with monotonically increasing offsets.

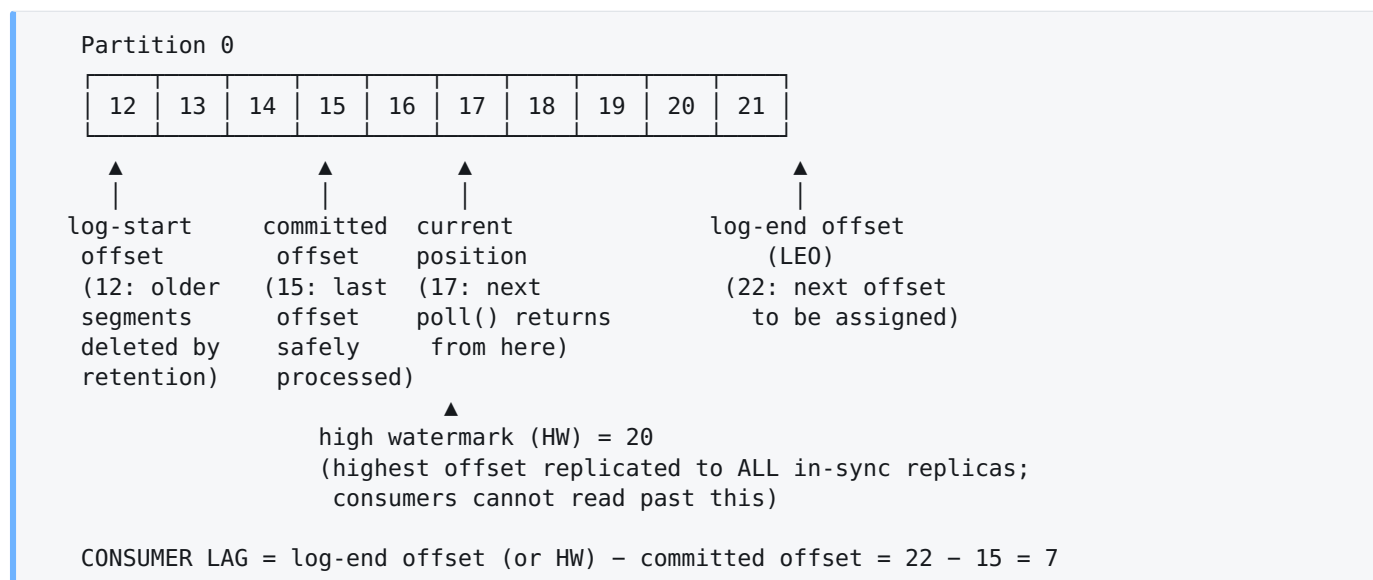
Internally: a directory of **log segments** (Part 7).

The three hard rules:

1. You can **increase** partitions but **never decrease** them.
2. Increasing partitions **changes the key→partition mapping** for future records, so existing keys may move — breaking per-key ordering across the change. (Old records don't move; new records for the same key may land elsewhere, so a consumer may see them out of order.)
3. Partitions have a real cost: file handles, memory, and, most importantly, controller metadata and failover time. Modern KRaft clusters handle hundreds of thousands of partitions, but per-broker guidance of a few thousand is still sane.

Offset — five different offsets you must distinguish

This distinction is asked constantly and answered badly.



Offset	Meaning	Why you care
log-start offset	oldest still-retained offset	if your committed offset < this, you get <code>OffsetOutOfRangeException</code> and <code>auto.offset.reset</code> kicks in — you've lost data
committed offset	last offset this group durably recorded as processed	where a new consumer resumes after a rebalance/restart
current position	next offset <code>poll()</code> will return, in memory	may be far ahead of committed (this is where duplicates come from)
high watermark (HW)	highest offset replicated to all ISR	consumers can only read up to HW — this is what makes committed data visible
log-end offset (LEO)	next offset the leader will assign	leader may have data past HW that isn't yet safe to read

The HW is why "the producer got an ack but my consumer can't see the record yet" is possible for a few milliseconds: the leader has it, but it isn't replicated to all ISR yet, so it isn't yet visible.

Producer

The client that appends. Covered in Part 8. Key facts for now: `send()` is async and buffered; records are batched per partition; the producer caches cluster metadata so it knows which broker leads which partition and sends directly there (no proxying).

Consumer and Consumer Group

Analogy: a consumer group is a **team sharing a workload**. The manager (group coordinator) assigns each team member a set of lanes. If someone quits, their lanes get reassigned. Different teams (groups) each get the whole workload.

Technically: a `group.id` string identifies the group. Kafka guarantees each partition is assigned to **at most one consumer within a group**.

topic "orders" with 4 partitions

— group "payments" with 2 consumers —
C1 ← P0, P1
C2 ← P2, P3

— group "payments" scaled to 4 —
C1←P0 C2←P1 C3←P2 C4←P3
(perfect)

— group "analytics", 1 consumer —
C1 ← P0, P1, P2, P3

— group "payments" scaled to 6 —
C1←P0 C2←P1 C3←P2 C4←P3
C5, C6 IDLE ← wasted money

The rules:

- consumers $\leq$ partitions $\rightarrow$ all consumers busy
- consumers = partitions $\rightarrow$ optimal
- consumers $>$ partitions $\rightarrow$ **extras idle forever**

This is why partition count is a capacity-planning decision, not an afterthought.

Broker

Analogy: one server in the warehouse. Holds some shelves (partitions), leads some of them, and copies others as backup.

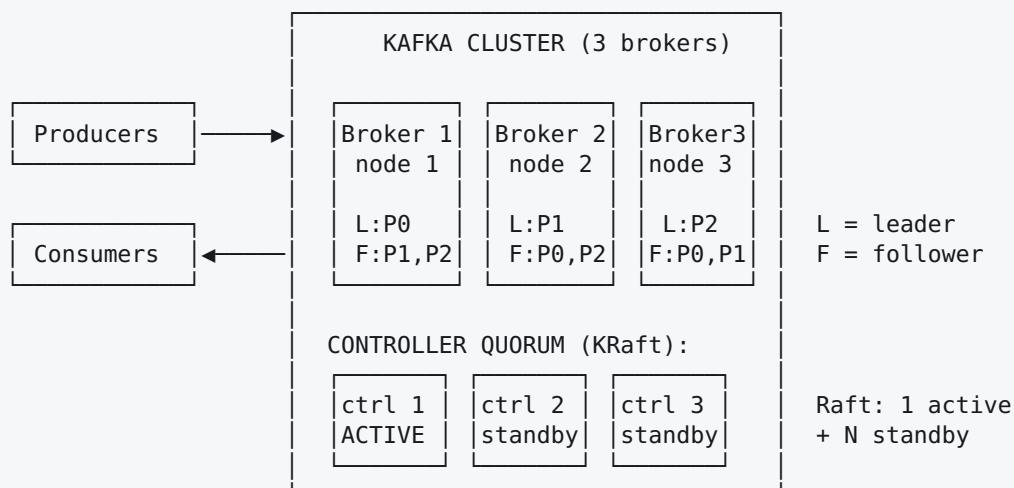
Technically: one Kafka server process. Identified by an integer `node.id` (formerly `broker.id`).

Responsibilities:

- host partition replicas (leader for some, follower for others)
- serve `Produce` and `Fetch` requests
- replicate from leaders for its follower partitions
- serve metadata to clients
- act as **group coordinator** for some consumer groups
- expire old segments per retention policy

A broker is not special. There's no "master broker" holding your data. Any broker can be a leader for some partitions and a follower for others, and every broker can answer metadata requests. (In KRaft, some nodes additionally take the *controller* role — that's a separate concern; see §4.10.)

Cluster



Balanced leadership matters: if all leaders land on one broker, that broker does all the work. Kafka tries to spread leadership; `auto.leader.rebalance.enable` (default true) periodically restores "preferred" leaders.

4.4 Putting it together: full write path

1. `Producer.send(new ProducerRecord<>("orders", "customer-991", json))`
2.
 - ↳ serialize key & value to `byte[]`
 - ↳ partitioner: `murmur2("customer-991") % 3 = 2`
 - ↳ append to in-memory batch for `orders-2`
 - ↳ return `Future` immediately ← your thread continues here (sender thread, later:)
3.
 - ↳ metadata says leader of `orders-2` is broker 3
 - ↳ `ProduceRequest(acks=all)` → Broker 3
4. Broker 3 (leader of `orders-2`):
 - ↳ validate, assign offsets (say 4182)
 - ↳ append to active log segment (page cache; `fsync` is `async` by default)
 - ↳ hold the request in "purgatory" until replication satisfies `acks`
5. Followers (brokers 1, 2) send `FetchRequest`s, pull the new records, append to their own logs, and report their fetch position back.
6. When all in-sync replicas have offset 4182:
 - ↳ leader advances the `HIGH WATERMARK` to 4183
 - ↳ `ProduceResponse(partition=2, baseOffset=4182)` → producer
 - ↳ your `Callback` fires with `RecordMetadata`
7. Consumers in group "payments" whose assignment includes `orders-2` fetch from their position; records up to HW are visible.
 - ↳ process → commit offset 4183 to `__consumer_offsets`

Read that flow three times. Almost every Kafka interview question is "what happens at step X if Y fails?"

4.5 Retention: how data actually leaves

Because consumption doesn't delete, something else must.

Policy	Config	Behaviour
delete (default)	<code>cleanup.policy=delete</code>	drop whole segments older than <code>retention.ms</code> (default 7 days) or once the partition exceeds <code>retention.bytes</code> (default <code>-1</code> = unlimited)
compact	<code>cleanup.policy=compact</code>	keep at least the latest value for each key , forever. Older values for the same key are removed.
both	<code>cleanup.policy=delete,compact</code>	compact, and also apply time/size limits

BEFORE compaction (offsets shown):

0:(A,v1) 1:(B,v1) 2:(A,v2) 3:(C,v1) 4:(B,v2) 5:(A,v3) 6:(C,null)

AFTER compaction:

4:(B,v2) 5:(A,v3) 6:(C,null←tombstone, retained briefly then removed)

Note: offsets are NOT renumbered. Gaps are normal and legal.

Compacted topics are how you store *state* rather than *events*: a topic where the key is a user ID and the value is the current profile is effectively a replicated key-value store with a changelog. `__consumer_offsets` is compacted. Kafka Streams state store changelogs are compacted.

△ Compaction is **not** an UPDATE and **not** immediate. It runs in the background on inactive segments, so duplicates for a key persist for a while. A tombstone (`value=null`) marks a key for deletion, and is itself retained for `delete.retention.ms` (default 24 h) so that consumers get a chance to see it.

4.6 Leader, follower, and ISR

Analogy

A team writing a shared document. One person (the **leader**) is the only one allowed to type. The others (**followers**) continuously copy what the leader typed. The **ISR** — in-sync replicas — is the set of people who are currently caught up "recently enough" to be trusted to take over.

Technically

- Every partition has exactly one **leader replica** at a time. All producer writes and (by default) all consumer reads go to the leader.
- **Followers** replicate by issuing `FetchRequest`s to the leader — they are literally consumers with special privileges.
- **ISR (In-Sync Replica set)** = the replicas that have fetched up to the leader's log end offset within `replica.lag.time.max.ms` (default **30 s**). The leader is always in its own ISR.
- **AR (Assigned Replicas)** = all replicas. **OSR (Out-of-Sync Replicas)** = AR – ISR.

```
replication.factor = 3, min.insync.replicas = 2

AR = {B1(leader), B2, B3}
ISR = {B1, B2}           ← B3 fell behind >30s (GC pause, slow disk, network)
OSR = {B3}

acks=all means "acknowledged by all replicas in the ISR",
NOT "all replicas in AR". ← THE most misunderstood point in Kafka
```

That distinction is critical: with `acks=all` and ISR shrunk to just the leader, "all" means "one". Which is why `min.insync.replicas` exists.

`min.insync.replicas` — the safety net

A **broker/topic** config (not a producer config) meaning: with `acks=all`, if the ISR has fewer than this many replicas, **reject the write** with `NotEnoughReplicasException`.

```
RF=3, min.insync.replicas=2, acks=all

ISR = {L, F1, F2} → write needs all 3 ISR members → tolerates 1 failure ✓
ISR = {L, F1}     → write needs 2 → still accepted ✓
ISR = {L}         → 1 < 2 → WRITE REJECTED ✗ (fail fast rather than
                                     silently risk data loss)
```

The production durability triple: `RF=3 + min.insync.replicas=2 + acks=all`. Memorise it. It tolerates one broker failure with no data loss and no availability loss. Setting `min.insync.replicas=3` with `RF=3` means *any* single broker failure stops writes — a classic misconfiguration.

Why ISR exists at all

Two naive alternatives, both bad:

- **Wait for all replicas always** (like a strict quorum of everyone): one slow disk anywhere makes the whole partition unavailable. Latency = the slowest replica, always.
- **Wait for a majority** (Raft-style, e.g. 2 of 3): tolerates f failures with $2f+1$ replicas. Kafka's ISR approach tolerates f failures with $f+1$ replicas — cheaper in storage for the same fault tolerance, at the cost of needing the ISR bookkeeping.

ISR is the compromise: **dynamic membership**. A slow replica is quietly evicted so it stops holding up writes, and rejoins automatically when it catches up. Availability degrades gracefully instead of stopping.

Unclean leader election

If **all** ISR members die, you have a choice:

<code>unclean.leader.election.enable</code>	Behaviour	Trade-off
<code>false</code> (default since 0.11)	partition stays offline until an ISR member returns	consistency — no data loss
<code>true</code>	elect an out-of-sync replica as leader	availability — silently loses the records the dead leader had but the new leader didn't

This is Kafka's CAP dial, exposed as one boolean. Default is CP (consistency). Turn it on only for data where availability beats correctness (e.g. metrics).

4.7 Why ZooKeeper existed

Analogy

ZooKeeper was the **HR department + notice board** for the Kafka cluster. It didn't do any of the work, but it kept the official record of who works here, who holds which job, and it noticed when someone stopped showing up.

What it actually stored

```
ZooKeeper znode tree (the old world)
/
├── /brokers
│   ├── /ids/1      ← EPHEMERAL node: {"host":"b1","port":9092,...}
│   ├── /ids/2      ← disappears automatically if b2's session expires
│   └── /topics/orders {"partitions":{"0":[1,2,3],"1":[2,3,1]}}
├── /controller      ← EPHEMERAL: {"brokerid":1,"epoch":42}
├── /controller_epoch ← 42 (fencing counter)
├── /config/topics/orders
├── /admin/delete_topics
└── /isr_change_notification
```

Four jobs:

1. **Membership & failure detection.** Each broker created an *ephemeral* znode. If its session (heartbeat) lapsed, ZooKeeper deleted the node and notified watchers. That's how Kafka learned a broker died.
2. **Controller election.** Whoever successfully created the ephemeral `/controller` node became the controller. First writer wins — a distributed lock.

3. **Metadata store.** Topics, partitions, replica assignments, configs, ACLs, quotas.
4. **Fencing via epochs.** `controller_epoch` increments on each election, so a zombie ex-controller's requests are rejected as stale.

Historically ZooKeeper also stored **consumer offsets** (pre-0.9). That was moved into the `__consumer_offsets` Kafka topic precisely because ZooKeeper couldn't handle the write rate. That migration was the first proof that ZooKeeper was the bottleneck.

Why it had to go — KIP-500

Six concrete problems. Being able to list these is a strong senior signal.

1. **Two distributed systems to operate.** Two sets of config, JVMs, monitoring, security models, upgrade procedures, and failure modes. Most Kafka outages involved ZooKeeper somehow.
2. **Metadata scalability.** Metadata was stored as znodes and each broker watched them. Cluster-wide metadata size and change propagation limited partition counts to roughly the low tens of thousands.
3. **Catastrophic controller failover.** On failover, the new controller had to **read the entire cluster metadata out of ZooKeeper** before it could do anything. With 100k partitions this took *minutes* — during which no leader elections could happen, so partition failures went unhealed. This was the killer.
4. **Push-based, per-broker metadata propagation.** The controller sent `UpdateMetadataRequest` to every broker on every change — $O(\text{brokers} \times \text{partitions})$ work and a source of divergence when a message was missed.
5. **Split-brain and divergence risk.** Two sources of truth (ZooKeeper's view and each broker's cached view) drift. "Stale metadata" bugs were endemic.
6. **Operational and cognitive overhead** for users: "how many ZK nodes? do I co-locate? what's a myid file? why did my ZK ensemble lose quorum?"

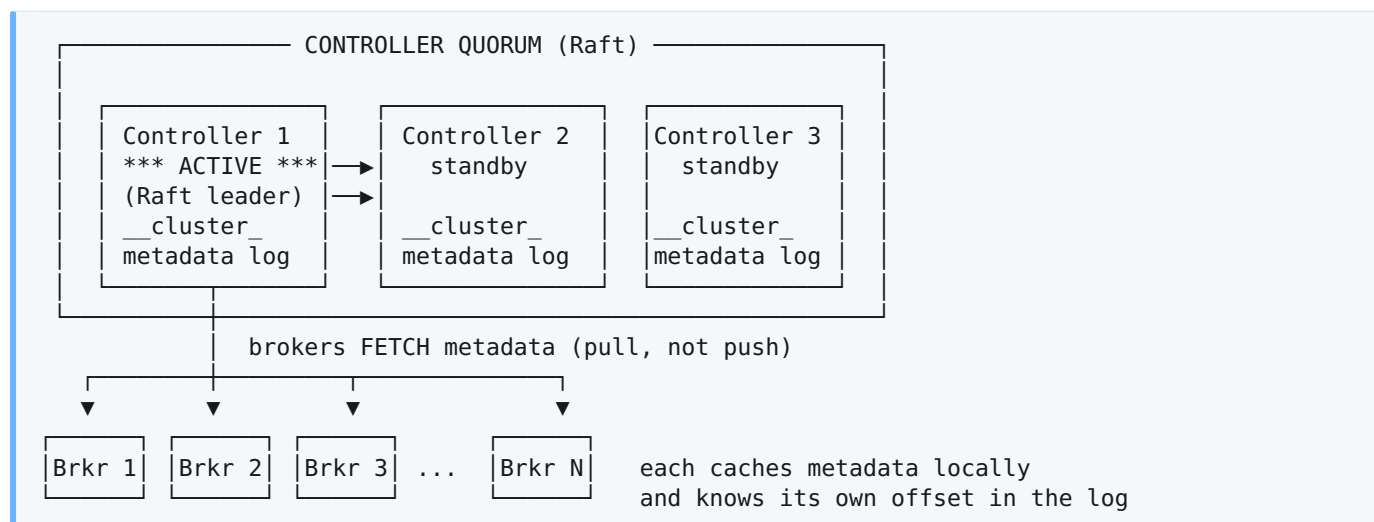
4.8 KRaft: Kafka Raft

Analogy

Instead of outsourcing the notice board to another company, Kafka **wrote the notice board into its own newspaper**. Cluster metadata became just another Kafka log — the thing Kafka is already extremely good at.

Technically

- Cluster metadata lives in an internal, replicated, single-partition topic: `__cluster_metadata`.
- A small set of nodes with `process.roles=controller` form the **controller quorum**, replicating that log using **Raft** (specifically KIP-595's Kafka-flavoured Raft).
- One controller is the **active controller** (Raft leader); the others are hot standbys that already have the full metadata log in memory.
- Brokers are **observers**: they *fetch* metadata records from the quorum, just like consumers. They replay the log into their local state.



Why this is dramatically better

	ZooKeeper mode	KRaft mode
Systems to run	2 (Kafka + ZK)	1
Metadata store	ZK znodes	a Kafka log (<code>_cluster_metadata</code>)
Controller failover	minutes (reload all metadata from ZK)	milliseconds (standby already has it in memory)
Metadata propagation	push (<code>UpdateMetadataRequest</code>)	pull (brokers fetch, track their own offset)
Partition ceiling	~tens of thousands	millions (2M+ demonstrated)
Recovery after failure	full reload	resume from last fetched offset — incremental
Source of truth	ZK + broker caches (drift possible)	one ordered log; every node converges
Config files	<code>server.properties</code> + <code>zookeeper.properties</code>	<code>server.properties</code> only
Security	two models to configure	one

The reason failover became milliseconds instead of minutes is the single most important thing to say: **standbys already have the metadata log applied in memory, so a new active controller has nothing to load.** Replication of metadata replaced reloading of metadata.

Node roles

```
# Combined mode – dev/small clusters only
process.roles=broker,controller

# Production – separate the roles
process.roles=broker           # on data nodes
process.roles=controller       # on 3 (or 5) dedicated controller nodes
```

```
# Every node needs to know the quorum
controller.quorum.voters=1@ctrl1:9093,2@ctrl2:9093,3@ctrl3:9093
node.id=1
```

Controller quorum sizing: **3** tolerates 1 failure, **5** tolerates 2. Always odd (Raft needs a majority). More than 5 just slows down consensus.

Migration and versions

Kafka version	ZooKeeper	KRaft
≤ 2.7	required	—
2.8	default	early access
3.3	default	production-ready (new clusters)
3.5	deprecated	recommended; ZK→KRaft migration available
3.6–3.9	supported for migration	default for new
4.0+	removed	only mode

Migrating an existing ZK cluster: bring up a KRaft quorum in migration mode, the controllers copy ZK metadata into `__cluster_metadata`, brokers roll into KRaft mode one at a time, then you finalise and decommission ZK. Doable online, but a project.

4.9 Controller: what it actually does

Analogy

The **shift manager** of the warehouse. Doesn't move boxes (no data flows through it). Decides who works which aisle, notices when someone doesn't clock in, and reassigns their aisle.

Responsibilities

1. **Leader election** for every partition when a broker joins/leaves.
2. **Maintaining ISR** — processing shrink/expand requests from leaders.
3. **Topic lifecycle** — create/delete topics, add partitions.
4. **Broker registration & liveness** — brokers heartbeat to the controller in KRaft (they registered with ZK before).
5. **Propagating metadata** to all brokers.
6. **Partition reassignment** driven by admin commands.
7. **Fencing** stale brokers/controllers via epochs.

The controller does not touch your data. No produce or fetch request goes through it. If the controller is unavailable, existing leaders keep serving reads and writes — you just can't do administrative operations or heal failures. This distinction is a great interview answer.

Controller election

ZooKeeper mode:

1. Every broker tries to create the EPHEMERAL znode /controller
2. ZooKeeper guarantees only one succeeds → that broker is controller
3. controller_epoch++ (stored in ZK)
4. Losers register a WATCH on /controller
5. Controller dies → session expires → znode auto-deleted
→ all watchers notified → race again → new controller
6. New controller reads ALL metadata from ZK ← the slow part
7. Old controller wakes up, sends a request with epoch 41,
brokers see current epoch is 42 → request rejected (fenced)

KRaft mode (Raft leader election):

1. Controllers exchange votes for a term; a candidate needs a MAJORITY
2. Winner becomes active controller for that epoch/term
3. It already has the full metadata log applied in memory → ready immediately
4. Heartbeats keep followers from starting a new election
5. Active controller dies → followers time out → new election → new epoch
6. Old active controller's writes are rejected: its epoch is stale

Broker "election" — there isn't one

Interviewers sometimes ask about "broker election." Be precise: **brokers are not elected**. They *register*. What gets elected is (a) the controller, and (b) a partition **leader** among that partition's replicas. There is no cluster-wide "master broker."

Partition leader election

When a leader dies, the controller picks a new one:

```
Partition orders-2:  replicas = [3, 1, 2]    (3 is the PREFERRED leader –
                                           first in the assignment list)
Current: leader=3, ISR={3,1,2}

Broker 3 dies:
├─ controller notices (missed heartbeats / session expiry)
├─ picks the first replica in the assignment list that is in the ISR → 1
├─ ISR becomes {1,2}; leader epoch++ (fences stale leader 3)
├─ writes the change to __cluster_metadata
├─ brokers fetch the new metadata; clients get NOT_LEADER_FOR_PARTITION,
  refresh metadata, and retransmit to broker 1 (transparent to your code
  as long as retries are enabled – which they are by default)

Broker 3 comes back:
├─ it becomes a FOLLOWER, truncates any records beyond the leader's HW,
  and catches up
├─ once caught up it rejoins ISR; a later preferred-leader rebalance
  may hand leadership back to 3 (auto.leader.rebalance.enable=true)
```

Leader epoch is a per-partition fencing counter. It exists because high-watermark-only truncation had a genuine data-loss bug (KIP-101); with leader epochs, a returning follower asks "what was the end offset of epoch N?" and truncates precisely.

4.10 Metadata: what clients cache and why it matters

Clients are not dumb. Every producer and consumer holds a local metadata cache:

Client metadata cache

```
brokers: 1 → b1:9092, 2 → b2:9092, 3 → b3:9092
topic "orders":
  partition 0 → leader 1, replicas [1,2,3], isr [1,2,3]
  partition 1 → leader 2, replicas [2,3,1], isr [2,3]
  partition 2 → leader 3, replicas [3,1,2], isr [3,1,2]
```

- `bootstrap.servers` is only used to make the **first** connection and fetch this map. After that the client talks **directly to partition leaders**. This is why you don't need to list every broker in `bootstrap.servers` (2-3 is fine, for redundancy of the bootstrap step) and why **there is no load balancer in front of Kafka** — the client must reach each broker directly by its `advertised.listeners` address.
- Refreshed every `metadata.max.age.ms` (default 5 min) and **immediately** on errors like `NOT_LEADER_OR_FOLLOWER` or `UNKNOWN_TOPIC_OR_PARTITION`.
- ⚠ **The #1 Docker/cloud Kafka problem** is `advertised.listeners`: the broker tells the client "connect to me at `localhost:9092`", but from inside another container or another network, that address is wrong. If your client connects, fetches metadata, then hangs or fails — it's `advertised.listeners`. Part 5 covers this.

Summary

- Kafka's design derives from one decision: **store an append-only log**. Partitions make it scale, keys preserve order, replication makes it durable, a controller coordinates it.
- **Offset** = position within a partition. Learn the five: log-start, committed, current position, high watermark, log-end. **Lag = LEO – committed**.
- **Ordering is per-partition only**. Same key ⇒ same partition ⇒ ordered.
- Partitions can only increase, and increasing them breaks key→partition affinity for future records.
- Within a group, a partition goes to **exactly one** consumer; extra consumers idle. **Partitions cap parallelism**.
- Consumption never deletes. **Retention** (time/size) or **compaction** (latest-value-per-key) does.
- `acks=all` means **all in-sync replicas**, not all replicas. Hence `min.insync.replicas`. Production triple: **RF=3, min.insync.replicas=2, acks=all**.
- **ISR** = replicas caught up within `replica.lag.time.max.ms` (30 s). Dynamic membership is what keeps a slow replica from stalling writes.
- `unclean.leader.election.enable` is Kafka's CAP dial: `false` = consistency (offline partition), `true` = availability (possible silent data loss).
- ZooKeeper did membership, controller election, metadata, and epoch fencing. It was removed (KIP-500, gone in 4.0) mainly because **controller failover required reloading all metadata** and metadata size capped partition counts.
- **KRaft** stores metadata in the `_cluster_metadata` Raft log; standby controllers hold it in memory, so failover is **milliseconds** and partition ceilings are in the millions. One system instead of two.
- The controller coordinates but **carries no data**; brokers register rather than being elected; what's elected is the controller and per-partition leaders.
- Clients cache metadata and talk **directly to partition leaders** — `advertised.listeners` must be an address the client can actually reach.

Interview Questions

Q1. What is a partition and why does it exist? An ordered, immutable, append-only sequence of records with monotonically increasing offsets — the physical unit of a topic. It exists so a topic can exceed one machine's storage and throughput, and so consumption can be parallelised. It's simultaneously the unit of ordering, parallelism, replication, and storage.

Q2. What is an offset? Is it unique across a topic? A monotonically increasing long identifying a record's position **within a single partition**, assigned by the broker at append time. It is *not* unique across a topic — every partition has its own sequence starting at 0. A record is uniquely identified by (topic, partition, offset).

Q3. Explain the high watermark. The highest offset that has been replicated to all in-sync replicas. Consumers cannot read beyond it, which guarantees they only see data that survives a leader failure. The leader's log-end offset can be higher — those records exist but aren't yet safe to expose.

Q4. What is the ISR, and what does `acks=all` really mean? ISR is the set of replicas that have fetched up to the leader's log end offset within `replica.lag.time.max.ms` (default 30 s); the leader is always a member. `acks=all` means "acknowledged by all replicas *currently in the ISR*" — not all assigned replicas. So if the ISR has shrunk to just the leader, `acks=all` gives you single-copy durability. `min.insync.replicas` closes that hole by rejecting writes when the ISR is too small.

Q5. `RF=3`, `min.insync.replicas=3`, `acks=all`. What's wrong? No fault tolerance for writes. Any single broker failure shrinks the ISR to 2, which is below the minimum, so all produces fail with `NotEnoughReplicasException`. You've configured a cluster that can't lose a single node. Use `min.insync.replicas=2` with `RF=3`.

Q6. How is a partition leader elected? The controller detects the leader is gone (missed heartbeats / expired session), then selects the first replica in the partition's assignment list that is currently in the ISR, increments the leader epoch to fence the old leader, and records the change in the metadata log. Brokers pick up the new metadata; clients receive `NOT_LEADER_OR_FOLLOWER`, refresh metadata and retry against the new leader. If no ISR member is available, the partition goes offline unless `unclean.leader.election.enable=true`.

Q7. What is unclean leader election? Electing a leader from replicas that are *not* in the ISR, when all ISR members are unavailable. It restores availability but silently loses records the dead leader had that the new one doesn't. Disabled by default since 0.11 — Kafka chooses consistency. Enable only where availability outweighs correctness.

Q8. Why was ZooKeeper removed? Two distributed systems to operate; metadata stored as znodes with per-broker watches, capping partition counts; **controller failover required reloading all cluster metadata from ZooKeeper, taking minutes at scale** and blocking leader elections during that window; push-based metadata propagation was $O(\text{brokers} \times \text{partitions})$ and could diverge; two sources of truth invited split-brain and stale metadata bugs; and it doubled the operational and security surface.

Q9. How does KRaft fix that? Metadata becomes an ordered, replicated Kafka log (`__cluster_metadata`) managed by a Raft quorum of controllers. Standby controllers replicate and apply it continuously, so failover is milliseconds — nothing to reload. Brokers *pull* metadata and track their own offset, so recovery is incremental and everyone converges on one ordered source of truth. One system, one config, one security model, and partition ceilings in the millions.

Q10. Does the controller handle produce/fetch requests? No. It's purely a control plane: leader elections, ISR changes, topic lifecycle, broker liveness, metadata propagation. Data flows directly between clients and partition leaders. If the controller is down, existing leaders keep serving traffic — you just lose administrative operations and failure healing.

Q11. Difference between broker election and controller election? There is no broker election — brokers register themselves (ephemeral znode in ZK mode; heartbeat registration to the controller quorum

in KRaft). What gets elected is the controller (single ephemeral znode race in ZK; Raft majority vote in KRaft) and, per partition, the leader among its replicas.

Q12. Can you decrease partitions? What happens when you increase them? You cannot decrease — Kafka would have to decide what to do with the existing records and their offsets, and consumers' committed offsets would become meaningless. Increasing is allowed but changes `hash(key) % numPartitions`, so future records for an existing key may land in a different partition than their history. Per-key ordering is broken across that boundary, and any keyed state in consumers or Kafka Streams may be wrong. Over-provision partitions instead.

Q13. 10 partitions, 15 consumers in one group. What happens? 10 consumers get one partition each; 5 sit idle. They're not errors — they're warm standbys that will take over on a rebalance — but they consume no records and cost money.

Q14. What's in `__consumer_offsets`? An internal, compacted, 50-partition-by-default topic storing offset commits keyed by (group, topic, partition), plus consumer group metadata. It replaced storing offsets in ZooKeeper (pre-0.9) because ZooKeeper couldn't sustain the write rate. Compaction keeps only the latest offset per key. A group's coordinator is the broker leading `hash(group.id) % 50`.

Q15. What is a leader epoch and why does it exist? A monotonically increasing counter per partition, bumped on every leader change. It fences stale leaders (a returning ex-leader's requests are rejected) and, per KIP-101, lets a returning follower ask the leader for the end offset of a specific epoch so it truncates its log at exactly the right point. High-watermark-only truncation had a real data-loss/divergence bug that leader epochs fixed.

Q16. What is log compaction and when would you use it? A retention mode (`cleanup.policy=compact`) that keeps at least the latest value per key indefinitely, deleting older values for the same key. Use it when the topic represents *state* rather than *events*: current user profile, latest price, config, consumer offsets, Kafka Streams changelogs. It's not an UPDATE and not immediate — it runs in the background on inactive segments, offsets aren't renumbered, and deletion requires a `null`-valued tombstone.

Q17. My producer got a successful ack but my consumer can't see the record. Why? Most likely the high watermark hasn't advanced past it yet — the leader has it but it isn't replicated to all ISR members, so it isn't consumer-visible. Also possible: your consumer is assigned different partitions, or the consumer is reading with `isolation.level=read_committed` and the record is part of an uncommitted transaction.

Q18. Why don't you put a load balancer in front of Kafka brokers? Because clients must reach *specific* brokers: the leader of each partition, and the coordinator for their group. A load balancer would route requests to the wrong broker, producing endless `NOT_LEADER_OR_FOLLOWER` retries. `bootstrap.servers` is only for the initial metadata fetch; after that clients connect directly using each broker's `advertised.listeners`.

Mini Quiz

1. A topic has 3 partitions. Is there a global order across the topic?
2. Where is offset 4182 — in a topic or in a partition?
3. Formula for consumer lag?
4. `RF=3`, `ISR={leader}`, `acks=all`, `min.insync.replicas=1`. How many copies does your "durable" write have?
5. Which offset can a consumer *not* read past?
6. Two consumers in group `g1`, one partition. How many receive records?
7. What deletes records in a topic with `cleanup.policy=compact`?
8. Name the internal topic that stores cluster metadata in KRaft.
9. Why is KRaft controller failover fast?

10. Can the controller be a broker too?
11. What error do clients get when they contact a former leader, and what do they do about it?
12. You increase partitions from 3 to 6. What breaks?
13. What's the default `replica.lag.time.max.ms`?
14. Odd or even number of controllers, and why?
15. Your client connects to Kafka then times out on produce. First config to check?

ANSWERS

1. No. Order exists only within each partition.
2. In a partition. (topic, partition, offset) identifies a record.
3. `log-end offset - committed offset`, per partition, summed for the group.
4. One. `acks=all` = all *in-sync* replicas, and ISR is just the leader.
5. The high watermark.
6. One. The other is idle.
7. A newer record with the same key (superseding it), or a tombstone (`value=null`) plus the passing of `delete.retention.ms`. Also time/size retention if `cleanup.policy=delete,compact`.
8. `__cluster_metadata`.
9. Standby controllers already replicate and apply the metadata log in memory, so the new active controller has nothing to load. In ZK mode it had to read all metadata from ZooKeeper first.
10. Yes — `process.roles=broker,controller` (combined mode). Fine for dev, discouraged in production; separate the roles.
11. `NOT_LEADER_OR_FOLLOWER`. They refresh metadata and retry against the new leader — transparent if retries are enabled.
12. Key→partition mapping (`hash(key) % n`) changes, so future records for an existing key may land in a new partition, breaking per-key ordering across the change and invalidating keyed state in consumers/Streams.
13. 30,000 ms (30 s).
14. Odd (3 or 5). Raft needs a strict majority; an even count adds a voter without improving fault tolerance.
15. `advertised.listeners`. The broker is advertising an address the client can't reach (classic in Docker/Kubernetes/cloud).

Practice Exercises

1. **See the files.** Create a topic with 3 partitions and produce 10 keyed records. Then `ls` the log directory and confirm three `topic-N` directories and the `.log/.index/.timeindex` files inside. Nothing makes partitions concrete faster.
2. **Prove per-key ordering.** Produce 100 records with key `A` and 100 with key `B` interleaved, to a 3-partition topic. Consume with `--property print.partition=true`. Verify all `A`s share a partition and are in order, and that `A` and `B` records interleave unpredictably across partitions.
3. **Break ordering.** Produce the same records with `key=null`. Observe them spread across partitions and the loss of any cross-record ordering.
4. **Watch a leader election.** 3-broker Docker cluster, RF=3. Run `kafka-topics --describe` and note the leaders. Kill the broker leading partition 0. Re-describe within a second or two. Watch leader and ISR change. Bring it back and watch it rejoin the ISR.

5. **Hit `min.insync.replicas`.** `RF=3`, `min.insync.replicas=2`, producer `acks=all`. Kill two brokers. Confirm produces fail with `NotEnoughReplicasException` rather than silently succeeding. Then set `min.insync.replicas=1` and watch them succeed — and reason about what you just gave up.
6. **Find the five offsets.** For one partition, obtain: log-start (`kafka-get-offsets --time -2`), log-end (`--time -1`), and the group's committed offset (`kafka-consumer-groups --describe`). Compute lag by hand and check it matches the LAG column.
7. **Compaction.** Create a topic with `cleanup.policy=compact`, `segment.ms=1000`, `min.cleanable.dirty.ratio=0.01`. Produce (A,1) (A,2) (A,3) (B,1). Force segment roll, wait, then consume from the beginning and see only the latest per key. Then produce (A,null) and watch A disappear.
8. **Partition-count consequences.** Topic with 2 partitions, consumer group with 4 members. Log which partitions each member owns. Add 2 partitions with `kafka-topics --alter`. Watch the rebalance and the newly-busy consumers.
9. **KRaft internals.** Run `kafka-metadata-shell --snapshot ../__cluster_metadata-0/*.log` and browse the metadata tree. Seeing topics, brokers, and configs as *log records* makes KRaft click.
10. **Deliberately misconfigure.** Set `advertised.listeners` to a hostname the client can't resolve. Observe the exact failure signature so you recognise it forever.

Common Mistakes

- **Assuming a topic is globally ordered.** It isn't, ever, unless it has exactly one partition. Half of all "Kafka reordered my messages" bugs are this.
 - **Producing with no key and expecting order.** No key $\Rightarrow$ any partition $\Rightarrow$ no order.
 - `min.insync.replicas` = **RF**. Zero fault tolerance for writes.
 - **Believing `acks=all` means all replicas.** It means all *in-sync* replicas, which can be one.
 - **Under-provisioning partitions.** You can't reduce them, and increasing them breaks key affinity. Estimate target throughput $\div$ per-consumer throughput, then multiply by 2–3 for headroom.
 - **Massively over-provisioning partitions** (e.g. 1,000 partitions for 100 msg/s). Costs file handles, memory, replication overhead, longer rebalances and slower failover.
 - **Scaling consumers past partition count** and wondering why throughput is flat.
 - **Turning on `unclean.leader.election.enable` to "fix" an offline partition** without understanding you're choosing silent data loss.
 - **Putting Kafka behind a load balancer or single DNS name for data traffic.** Clients must reach specific brokers.
 - **Getting `advertised.listeners` wrong in Docker** and spending an afternoon on it. Every single person does this once.
 - **Treating compaction as UPDATE/DELETE.** It's asynchronous, only runs on inactive segments, keeps offsets as-is with gaps, and needs tombstones.
 - **Naming a topic after two services.** It can never grow a third consumer.
 - **Deploying combined-mode KRaft (`broker,controller`) in production.** Separate the roles so heavy data traffic can't starve metadata consensus.
-

Further Reading

- KIP-500: "Replace ZooKeeper with a Self-Managed Metadata Quorum" — the design document. Read the "Motivation" section at minimum.

- KIP-595: "A Raft Protocol for the Metadata Quorum" — how KRaft actually works.
- KIP-101: "Alter Replication Protocol to use Leader Epoch" — the data-loss bug that leader epochs fixed. Excellent interview material.
- Apache Kafka docs §4 "Design" (especially 4.2 Persistence, 4.6 Message Delivery Semantics, 4.7 Replication) — dense and worth reading twice.
- Jun Rao, "*Intra-cluster Replication in Apache Kafka*" — the ISR design rationale from the person who built it.
- Confluent blog, "*Apache Kafka Made Simple: A First Glimpse of a Kafka Without ZooKeeper*" — clear before/after on controller failover timings.
- Apache Kafka docs, "KRaft Configuration" — the practical setup reference.

Part 5 — Installation

If you only do one thing in this Part: use the Docker Compose file in §5.5. It gives you a 3-broker KRaft cluster plus a UI in one command, which is what you need for every exercise in this handbook. The manual install is here so you understand what Docker is doing for you — and because interviewers ask what's in `server.properties`.

5.1 Prerequisites

Requirement	Detail
Java	JDK 17+ for brokers (Kafka 4.x requires it). Clients work on 11+. Kafka 3.x still runs on Java 11 but 17 is recommended.
RAM	2 GB for a dev single broker; 4–8 GB for a 3-broker Docker cluster
Disk	fast local SSD. Never NFS or network storage for <code>log.dirs</code> .
OS	Linux for production. macOS fine for dev. Windows via WSL2 or Docker.

```
java -version      # need 17+
# Linux
sudo apt install openjdk-17-jdk      # Debian/Ubuntu
sudo dnf install java-17-openjdk     # Fedora/RHEL
# macOS
brew install openjdk@17
```

⚠ **Do not give Kafka a huge JVM heap.** Kafka deliberately keeps very little data in the JVM — it relies on the OS **page cache** for its read/write speed (Part 7). 6 GB heap is plenty even for large brokers; leave the rest of RAM to the OS. A 64 GB heap will make Kafka *slower* (GC pauses) and starve the page cache. This is a favourite interview question.

5.2 Manual installation (Linux / macOS) — KRaft mode

Download and unpack

```
KAFKA_VERSION=4.0.0
SCALA_VERSION=2.13

cd /opt
sudo curl -O https://downloads.apache.org/kafka/${KAFKA_VERSION}/kafka_${SCALA_VERSION}-${KAFKA_VERSION}.tgz
sudo tar -xzf kafka_${SCALA_VERSION}-${KAFKA_VERSION}.tgz
sudo ln -s /opt/kafka_${SCALA_VERSION}-${KAFKA_VERSION} /opt/kafka
```

The `2.13` in the filename is the **Scala** version the broker was compiled with, not the Kafka version. Pick 2.13 unless you have a reason not to.

Folder structure

```

/opt/kafka/
├── bin/                                ← all the CLI tools (.sh) – you'll live here
│   ├── kafka-server-start.sh
│   ├── kafka-server-stop.sh
│   ├── kafka-storage.sh              ← KRaft: format the log directory
│   ├── kafka-topics.sh
│   ├── kafka-console-producer.sh
│   ├── kafka-console-consumer.sh
│   ├── kafka-consumer-groups.sh
│   ├── kafka-configs.sh
│   ├── kafka-get-offsets.sh
│   ├── kafka-reassign-partitions.sh
│   ├── kafka-producer-perf-test.sh
│   ├── kafka-consumer-perf-test.sh
│   ├── kafka-dump-log.sh             ← inspect raw log segments (Part 7)
│   ├── kafka-metadata-shell.sh       ← browse __cluster_metadata
│   ├── kafka-acls.sh
│   ├── connect-distributed.sh
│   └── windows/                     ← .bat equivalents
├── config/
│   ├── server.properties             ← (4.x) the KRaft broker config
│   ├── kraft/                       ← (3.x) broker.properties, controller.properties,
│   │                                       server.properties for combined mode
│   ├── log4j2.yaml                  ← logging (log4j.properties in 3.x)
│   ├── connect-distributed.properties
│   └── producer.properties / consumer.properties ← CLI tool defaults
├── libs/                            ← all the JARs
└── logs/                            ← broker's own application logs (NOT your data!)

```

⚠ **Vocabulary trap that bites everyone:** `/opt/kafka/logs` is where the broker writes its *application logs* (log4j output). Your **data** lives in `log.dirs`, default `/tmp/kafka-logs`. Kafka calls its data "logs" because it is a log. Two completely different things named the same. And yes — the default is under `/tmp`, which many Linux distros clear on reboot. **Always change `log.dirs`.**

Configure a single-node KRaft broker

```
/opt/kafka/config/server.properties:
```

```
##### Roles & identity #####
process.roles=broker,controller      # combined mode: dev only
node.id=1
controller.quorum.voters=1@localhost:9093

##### Listeners #####
listeners=PLAINTEXT://:9092,CONTROLLER://:9093
inter.broker.listener.name=PLAINTEXT
controller.listener.names=CONTROLLER
listener.security.protocol.map=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT

# What clients are TOLD to connect to. Must be reachable BY THE CLIENT.
advertised.listeners=PLAINTEXT://localhost:9092

##### Storage #####
log.dirs=/var/lib/kafka/data          # NOT /tmp. Use a dedicated fast disk.
num.partitions=3                      # default for auto-created topics
num.recovery.threads.per.data.dir=1

##### Durability #####
default.replication.factor=1          # single node, so 1
min.insync.replicas=1
offsets.topic.replication.factor=1    # ← MUST be 1 on a single node
```



```

transaction.state.log.replication.factor=1
transaction.state.log.min.isr=1

##### Retention #####
log.retention.hours=168          # 7 days
log.retention.check.interval.ms=300000
log.segment.bytes=1073741824     # 1 GiB segments

##### Threads #####
num.network.threads=3
num.io.threads=8
socket.send.buffer.bytes=102400
socket.receive.buffer.bytes=102400
socket.request.max.bytes=104857600

##### Groups #####
group.initial.rebalance.delay.ms=0 # 0 for dev (3000 in prod)

##### Safety #####
auto.create.topics.enable=false    # ← turn this OFF. See Common Mistakes.
delete.topic.enable=true

```

The four config lines that cause 90% of first-time failures are marked with comments above. Read them again.

Format the storage directory (KRaft only — this step is new)

ZooKeeper mode didn't need this. KRaft does: the metadata log needs a cluster ID.

```

# 1. Generate a cluster UUID (do this ONCE per cluster, ever)
KAFKA_CLUSTER_ID=$(/opt/kafka/bin/kafka-storage.sh random-uuid)
echo $KAFKA_CLUSTER_ID      # e.g. Mku30EVBNTcwNTJENDM2Qk

# 2. Format each node's log.dirs with that ID
/opt/kafka/bin/kafka-storage.sh format \
  -t $KAFKA_CLUSTER_ID \
  -c /opt/kafka/config/server.properties

```

This writes a `meta.properties` file into `log.dirs` containing the cluster ID and node ID.

Failure modes to recognise:

- **"No `meta.properties` found"** → you forgot to format.
- **"Log dir already formatted"** → you formatted twice; pass `--ignore-formatted` or wipe the directory.
- **"Inconsistent cluster ID"** → different nodes formatted with different UUIDs. All nodes in one cluster share **one** cluster ID.

Start it

```

# Foreground (see the logs – do this the first time)
/opt/kafka/bin/kafka-server-start.sh /opt/kafka/config/server.properties

# Background (daemon)
/opt/kafka/bin/kafka-server-start.sh -daemon /opt/kafka/config/server.properties

# Verify
/opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
# no output + no error = success

# Stop (graceful – allows controlled shutdown & leadership handoff)
/opt/kafka/bin/kafka-server-stop.sh

```

Look for this line in the output — it means you're up:

```
INFO [KafkaRaftServer nodeId=1] Kafka Server started (kafka.server.KafkaRaftServer)
```

Environment variables that matter

Variable	Purpose
KAFKA_HEAP_OPTS	JVM heap. <code>-Xmx6g -Xms6g</code> for prod; <code>-Xmx1g -Xms1g</code> for dev. Set <code>Xmx == Xms</code> .
KAFKA_JVM_PERFORMANCE_OPTS	GC flags. Default uses G1.
KAFKA_OPTS	catch-all extra JVM args (JAAS config for SASL goes here)
KAFKA_JMX_OPTS	JMX settings
JMX_PORT	set this to enable JMX (monitoring, Part 16)
LOG_DIR	where the <i>broker's own</i> log4j output goes
KAFKA_LOG4J_OPTS	points at the log4j config file

```
export KAFKA_HEAP_OPTS="-Xmx1g -Xms1g"
export JMX_PORT=9999
export LOG_DIR=/var/log/kafka
```

Run as a systemd service (production)

```
# /etc/systemd/system/kafka.service
[Unit]
Description=Apache Kafka
Requires=network.target
After=network.target

[Service]
Type=simple
User=kafka
Group=kafka
Environment="JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64"
Environment="KAFKA_HEAP_OPTS=-Xmx6g -Xms6g"
Environment="LOG_DIR=/var/log/kafka"
ExecStart=/opt/kafka/bin/kafka-server-start.sh /opt/kafka/config/server.properties
ExecStop=/opt/kafka/bin/kafka-server-stop.sh
Restart=on-failure
# Give Kafka time to hand off leadership on shutdown
TimeoutStopSec=180
LimitNOFILE=100000

[Install]
WantedBy=multi-user.target
```

```
sudo useradd -r -s /bin/false kafka
sudo mkdir -p /var/lib/kafka/data /var/log/kafka
sudo chown -R kafka:kafka /var/lib/kafka /var/log/kafka /opt/kafka_*
sudo systemctl daemon-reload && sudo systemctl enable --now kafka
sudo journalctl -u kafka -f
```

`LimitNOFILE=100000` is not optional. Kafka opens a file handle per log segment per partition plus every socket; the default 1024 will fail in ways that look mysterious.

5.3 macOS

```
# Homebrew (simplest, but you get whatever version brew has)
brew install kafka
brew services start kafka           # runs in KRaft mode in recent formulae

# Config lives at: /opt/homebrew/etc/kafka/server.properties
# Data lives at:   /opt/homebrew/var/lib/kafka-logs
# CLI tools are on PATH WITHOUT the .sh suffix:
kafka-topics --bootstrap-server localhost:9092 --list
```

For anything beyond "does it start", prefer the manual tarball or Docker — you'll want control over versions and configs.

5.4 Windows

Recommendation: don't run Kafka natively on Windows. The `.bat` scripts lag behind, and there are long-standing bugs where Kafka can't delete log segments because Windows holds file locks — which causes broker crashes on retention or topic deletion. Use one of:

Option A — WSL2 (best native-feeling experience)

```
wsl --install -d Ubuntu
```

Then follow the Linux instructions inside WSL. Kafka is reachable from Windows at `localhost:9092`. Keep `log.dirs` on the **Linux** filesystem (`/var/lib/...`), not under `/mnt/c/` — the 9p filesystem bridge is slow enough to cause timeouts.

Option B — Docker Desktop (recommended overall)

Skip to §5.5.

Option C — native Windows, if you must

```
# Unpack to a SHORT path – long paths break Windows classpath limits
# C:\kafka ✓          C:\Users\you\Downloads\kafka_2.13-4.0.0\ ✗

# Edit config\server.properties:
#   log.dirs=C:/kafka/data      (forward slashes!)

C:\kafka\bin\windows\kafka-storage.bat random-uuid
C:\kafka\bin\windows\kafka-storage.bat format -t <UUID> -c C:\kafka\config\server.properties
C:\kafka\bin\windows\kafka-server-start.bat C:\kafka\config\server.properties
```

5.5 Docker — the recommended dev setup

Single broker, one command

```
docker run -d --name kafka -p 9092:9092 apache/kafka:4.0.0
```

The official `apache/kafka` image auto-configures single-node KRaft. Good for a five-minute smoke test.

A proper 3-broker KRaft cluster with UI

Save as `docker-compose.yml`. This is the environment to use for the rest of the handbook — three brokers means you can actually test replication, ISR shrinkage, and leader elections.

```
name: kafka-lab

services:
  kafka-1: &kafka-template
    image: apache/kafka:4.0.0
    container_name: kafka-1
    ports:
      - "19092:19092"
    environment: &kafka-env
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka-1:9093,2@kafka-2:9093,3@kafka-3:9093
      KAFKA_LISTENERS: INTERNAL://:9092,CONTROLLER://:9093,EXTERNAL://:19092
      KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka-1:9092,EXTERNAL://localhost:19092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: INTERNAL:PLAINTEXT,CONTROLLER:PLAINTEXT,EXTERNAL:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      CLUSTER_ID: 4L6g3nShT-eMCtK--X86sw
      KAFKA_LOG_DIRS: /var/lib/kafka/data
      # 3 brokers → make the internal topics properly replicated
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 3
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 2
      KAFKA_DEFAULT_REPLICATION_FACTOR: 3
      KAFKA_MIN_INSYNC_REPLICAS: 2
      KAFKA_NUM_PARTITIONS: 3
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
      KAFKA_HEAP_OPTS: "-Xmx512m -Xms512m"
    volumes:
      - kafka-1-data:/var/lib/kafka/data

  kafka-2:
    <<: *kafka-template
    container_name: kafka-2
    ports: ["29092:29092"]
    environment:
      <<: *kafka-env
      KAFKA_NODE_ID: 2
      KAFKA_LISTENERS: INTERNAL://:9092,CONTROLLER://:9093,EXTERNAL://:29092
      KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka-2:9092,EXTERNAL://localhost:29092
    volumes:
      - kafka-2-data:/var/lib/kafka/data

  kafka-3:
    <<: *kafka-template
    container_name: kafka-3
    ports: ["39092:39092"]
    environment:
      <<: *kafka-env
      KAFKA_NODE_ID: 3
      KAFKA_LISTENERS: INTERNAL://:9092,CONTROLLER://:9093,EXTERNAL://:39092
      KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka-3:9092,EXTERNAL://localhost:39092
    volumes:
      - kafka-3-data:/var/lib/kafka/data

  kafka-ui:
    image: provectuslabs/kafka-ui:latest
    container_name: kafka-ui
    depends_on: [kafka-1, kafka-2, kafka-3]
    ports:
```

```

- "8080:8080"
environment:
  KAFKA_CLUSTERS_0_NAME: kafka-lab
  KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka-1:9092,kafka-2:9092,kafka-3:9092
  DYNAMIC_CONFIG_ENABLED: "true"

volumes:
  kafka-1-data:
  kafka-2-data:
  kafka-3-data:

```

```

docker compose up -d
docker compose ps
docker compose logs -f kafka-1

# From your host machine:
kafka-topics --bootstrap-server localhost:19092 --list

# Or from inside a container:
docker exec -it kafka-1 /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server kafka-1:9092 --list

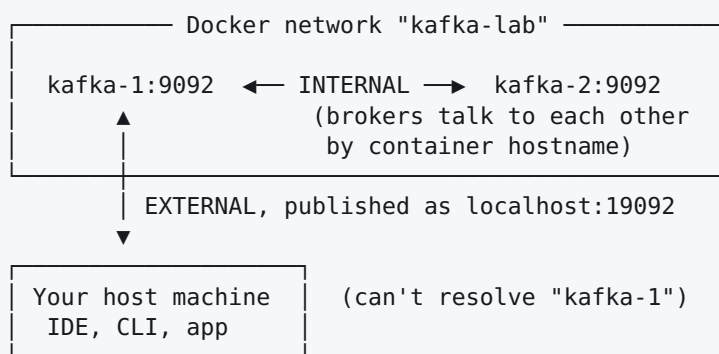
# UI: http://localhost:8080

# Kill a broker to test failover:
docker compose stop kafka-2
docker compose start kafka-2

# Full reset (deletes data):
docker compose down -v

```

Understanding the dual-listener setup (the thing everyone gets wrong)



Two listeners exist because the broker must advertise **different addresses to different clients**:

- Another container asks for metadata → gets `kafka-1:9092` (resolvable inside the Docker network).
- Your laptop asks for metadata → gets `localhost:19092` (resolvable on your host).

If you configure only one listener advertising `kafka-1:9092`, your host app will connect to the bootstrap port, fetch metadata, be told "the leader is at `kafka-1:9092`", fail to resolve `kafka-1`, and hang until timeout — **even though the initial connection succeeded**. That signature (connects, then times out on produce/consume) means `advertised.listeners` every single time.

Confluent images (alternative)

```
kafka:
  image: confluentinc/cp-kafka:7.6.0
  environment:
    KAFKA_NODE_ID: 1
    KAFKA_PROCESS_ROLES: broker,controller
    # ... same variable names
```

Confluent's images add Schema Registry (`cp-schema-registry`), ksqlDB, Control Center, and Connect. Use them when you get to Parts 10 and 17. Note that Confluent's `cp-kafka` versions don't match Apache Kafka versions (7.6 ≈ Kafka 3.6).

5.6 ZooKeeper mode (legacy, for reference only)

You'll meet this in existing systems. Kafka 4.0 removed it, so this only applies to 3.x clusters.

```
# Terminal 1 – ZooKeeper first, always
bin/zookeeper-server-start.sh config/zookeeper.properties

# Terminal 2 – then the broker
bin/kafka-server-start.sh config/server.properties
```

`config/zookeeper.properties`:

```
dataDir=/var/lib/zookeeper
clientPort=2181
maxClientCnxns=0
admin.enableServer=false
```

The ZK-mode broker config differs in these lines:

```
broker.id=1                                # instead of node.id
zookeeper.connect=zk1:2181,zk2:2181,zk3:2181/kafka
zookeeper.connection.timeout.ms=18000
# no process.roles, no controller.quorum.voters, no kafka-storage.sh format
```

Note the `/kafka` suffix (a **chroot**) — it namespaces Kafka's znodes so one ZK ensemble can serve multiple Kafka clusters. Forgetting it and sharing a ZK ensemble between two clusters is a spectacular way to corrupt both.

Production ZK ensembles are 3 or 5 nodes (odd, for quorum), each with a `myid` file and a `server.N=host:2888:3888` list.

5.7 The configuration files, annotated

Broker configs you should actually know

Grouped by what they control. These recur throughout the handbook.

Identity & roles

Config	Default	Notes
<code>node.id</code>	—	unique int per node (KRaft)
<code>process.roles</code>	—	<code>broker</code> , <code>controller</code> , or both
<code>controller.quorum.voters</code>	—	<code>id@host:port</code> list

Networking

Config	Default	Notes
<code>listeners</code>	<code>PLAINTEXT://:9092</code>	what the broker binds
<code>advertised.listeners</code>	<code>= listeners</code>	what clients are told. Get this right.
<code>num.network.threads</code>	3	socket read/write threads
<code>num.io.threads</code>	8	disk I/O threads; $\approx$ number of disks $\times$ 2
<code>socket.request.max.bytes</code>	100 MB	max request size
<code>message.max.bytes</code>	~1 MB	max record batch the broker accepts

Storage & retention

Config	Default	Notes
<code>log.dirs</code>	<code>/tmp/kafka-logs</code>	change this. Comma-separated for multiple disks (JBOD).
<code>log.segment.bytes</code>	1 GiB	segment roll size
<code>log.roll.ms / .hours</code>	168 h	time-based roll
<code>log.retention.ms / .minutes / .hours</code>	168 h	time retention
<code>log.retention.bytes</code>	−1	size retention per partition , not per topic
<code>log.retention.check.interval.ms</code>	5 min	how often retention runs
<code>log.cleanup.policy</code>	<code>delete</code>	or <code>compact</code>
<code>log.index.interval.bytes</code>	4096	sparse index density

Durability & replication

Config	Default	Notes
<code>default.replication.factor</code>	1	set to 3 in production
<code>min.insync.replicas</code>	1	set to 2 with RF=3
<code>unclean.leader.election.enable</code>	false	leave it false
<code>replica.lag.time.max.ms</code>	30 s	ISR eviction threshold
<code>num.replica.fetchers</code>	1	raise to 4–8 on big clusters
<code>auto.leader.rebalance.enable</code>	true	restores preferred leaders

Internal topics — these three cause more failed first installs than anything else, because the defaults are 3 and a single-node cluster can't satisfy that:

Config	Default	Single node	Production
<code>offsets.topic.replication.factor</code>	3	1	3
<code>transaction.state.log.replication.factor</code>	3	1	3
<code>transaction.state.log.min.isr</code>	2	1	2

Safety

Config	Default	Notes
<code>auto.create.topics.enable</code>	true	set false. See below.
<code>delete.topic.enable</code>	true	keep true
<code>group.initial.rebalance.delay.ms</code>	3000	0 in dev for fast startup

Why `auto.create.topics.enable=false`

With it on, a single typo in a consumer's topic name silently creates a brand-new topic with default partitions and RF=1, and your consumer sits there reading nothing while you debug for an hour. Worse, in production you accumulate junk topics with RF=1 (no redundancy) that nobody owns. Create topics deliberately.

Topic-level overrides

Any `log.*` broker config has a topic-level equivalent (without the `log.` prefix) that wins:

```
kafka-configs --bootstrap-server localhost:9092 \
  --entity-type topics --entity-name orders --alter \
  --add-config retention.ms=604800000,cleanup.policy=compact,min.insync.replicas=2
```

Broker config	Topic override
<code>log.retention.ms</code>	<code>retention.ms</code>
<code>log.segment.bytes</code>	<code>segment.bytes</code>
<code>log.cleanup.policy</code>	<code>cleanup.policy</code>
<code>min.insync.replicas</code>	<code>min.insync.replicas</code>
<code>message.max.bytes</code>	<code>max.message.bytes</code>

5.8 OS-level production tuning

Interviewers at infra-heavy companies ask about these.

```
# /etc/sysctl.conf

# File descriptors – Kafka opens one per segment per partition + sockets
fs.file-max = 200000

# Networking: bigger socket buffers for high-throughput replication
net.core.rmem_max = 16777216
net.core.wmem_max = 16777216
net.ipv4.tcp_rmem = 4096 65536 16777216
net.ipv4.tcp_wmem = 4096 65536 16777216

# Let the page cache hold dirty data longer – Kafka relies on the page cache,
# and flushing is the OS's job, not Kafka's
vm.dirty_background_ratio = 5
vm.dirty_ratio = 60

# Do NOT swap the broker
vm.swappiness = 1
```



```
# /etc/security/limits.conf
kafka - nofile 100000
```

Other production notes:

- **Filesystem:** XFS is the usual recommendation (better handling of large files and parallel writes); ext4 is fine with `noatime`.
- **Mount with `noatime`** — Kafka never needs access times, and updating them costs writes.
- **Never use RAID 5/6** for Kafka (write penalty). RAID 10 or plain **JBOD** (multiple entries in `log.dirs`) with `RF=3` doing the redundancy work.
- **Never use NFS/EFs/network storage** for `log.dirs`.
- **Disable transparent huge pages** if you see latency spikes.
- **Don't run other memory-hungry services on a broker** — you're stealing page cache.

Summary

- Java 17+, fast local SSD, generous file descriptors, **small JVM heap** (~6 GB) because Kafka relies on the OS page cache.
- KRaft requires a one-time `kafka-storage.sh format` with a cluster UUID shared by all nodes in the cluster.
- `/opt/kafka/logs` = broker application logs. `log.dirs` = your data. Change `log.dirs` off `/tmp`.
- On a single node you **must** set `offsets.topic.replication.factor=1` and the two `transaction.state.log.*` settings to 1, or nothing works.
- `advertised.listeners` is the #1 source of Docker/cloud connection problems. Use separate INTERNAL and EXTERNAL listeners so brokers and host clients each get an address they can resolve.
- Use the 3-broker Docker Compose from §5.5 for all exercises — one broker can't demonstrate replication, ISR, or failover.
- Set `auto.create.topics.enable=false`.
- Windows: WSL2 or Docker, not native.

Interview Questions

Q1. Why should a Kafka broker have a small JVM heap? Kafka intentionally keeps almost nothing in the JVM: writes go into the OS page cache and reads are usually served from it, with zero-copy transfer straight from page cache to socket. Free RAM *is* Kafka's cache. A large heap steals memory from the page cache and adds GC pause risk, so it makes Kafka slower. ~6 GB is typical even for large brokers.

Q2. What is `advertised.listeners` and why does it break Docker setups? `listeners` is what the broker binds locally; `advertised.listeners` is the address the broker returns in metadata for clients to connect to. Clients bootstrap once, receive this address, then connect **directly** to partition leaders using it. In Docker, the broker's internal hostname isn't resolvable from the host, so clients connect for bootstrap and then time out on the actual data request. The fix is multiple listeners advertising a network-appropriate address to each audience.

Q3. What does `kafka-storage.sh format` do, and why didn't ZooKeeper mode need it? It initialises the log directory with a `meta.properties` containing the cluster ID and node ID, so the node can participate in the KRaft metadata quorum. In ZooKeeper mode the cluster ID lived in ZooKeeper and was created automatically, so there was nothing to format locally.

Q4. Fresh single-node install; creating a consumer group fails. Likely cause?

`offsets.topic.replication.factor` defaults to 3, and `__consumer_offsets` can't be created with RF=3 on one broker. Same for `transaction.state.log.replication.factor`. Set them to 1 for single-node.

Q5. Why disable `auto.create.topics.enable`? Because a typo in a topic name silently creates a topic with default partitions and RF=1 instead of failing loudly, so your consumer reads an empty topic while you debug. In production it also accumulates unowned, unreplicated topics. Topic creation should be a deliberate, reviewed operation.

Q6. Why not NFS or RAID 5 for Kafka data? Kafka's performance model assumes fast sequential local I/O; network filesystems add latency and have file-locking semantics that break Kafka's assumptions. RAID 5/6 impose a parity write penalty on every write, which is exactly the wrong trade-off for a write-heavy sequential workload. Use local SSDs as JBOD or RAID 10, and let replication factor 3 provide redundancy.

Q7. `log.dirs` vs the `logs/` directory? `log.dirs` holds your topic data (partition directories with `.log/` `.index` segments). `logs/` (or `LOG_DIR`) holds the broker's own log4j application logs. Confusing them leads people to delete data thinking they're clearing logs.

Q8. How many controller nodes and why odd? 3 (tolerates 1 failure) or 5 (tolerates 2). Odd because Raft requires a strict majority; a 4th voter needs 3 for quorum just like 3 does, so it adds cost and latency without improving fault tolerance.

Q9. Combined vs separate `process.roles` in production? Separate. In combined mode heavy data-plane traffic (produce/fetch, replication, GC pauses) can starve the metadata consensus path, and a broker restart also restarts a controller. Dedicated controller nodes are small and stable.

Q10. Why does Kafka need such a high file-descriptor limit? It keeps a file handle open per active log segment per partition — plus the index and timeindex files — plus every client and replication socket. Thousands of partitions easily exceed the default 1024 limit, and the resulting failures look like random unrelated errors.

Mini Quiz

1. Minimum Java version for Kafka 4.x brokers?
2. Which command generates a KRaft cluster ID?
3. Default `log.dirs`, and why is it dangerous?
4. Three configs you must change to 1 on a single-node cluster?
5. Which config makes clients unable to reach brokers in Docker?
6. Recommended broker heap size, and why not more?
7. Where does the broker's log4j output go by default?
8. What does the `/kafka` suffix on `zookeeper.connect` do?
9. Which two listeners does the \$5.5 Compose file define, and who uses each?
10. Can you shrink a topic's partition count by editing config?

ANSWERS

1. Java 17.
2. `kafka-storage.sh random-uuid`.
3. `/tmp/kafka-logs`. Many systems clear `/tmp` on reboot — you'd lose all data.
4. `offsets.topic.replication.factor`, `transaction.state.log.replication.factor`, `transaction.state.log.min.isr` (and `default.replication.factor`).
5. `advertised.listeners`.

6. ~6 GB. Kafka relies on the OS page cache; a bigger heap steals cache and adds GC pauses.
7. `$KAFKA_HOME/logs`, overridable via `LOG_DIR`.
8. It's a chroot — namespaces Kafka's znodes so one ZK ensemble can host multiple Kafka clusters.
9. `INTERNAL` (broker-to-broker, `kafka-N:9092`, resolvable inside the Docker network) and `EXTERNAL` (`localhost:1/2/39092`, resolvable from the host). Plus `CONTROLLER` for the Raft quorum.
10. No. Partition count can only increase, and only via `kafka-topics --alter --partitions N`.

Practice Exercises

1. **Manual install, single node.** Tarball, edit `server.properties` (fix `log.dirs`, the three internal-topic RFs, `auto.create.topics.enable=false`), format, start in the foreground and read the startup log until you find the "Kafka Server started" line.
2. **Break it on purpose.** Skip `kafka-storage.sh format` and start the broker. Read the error. Then format twice and read that error. Then format two nodes with different UUIDs and read *that* error. You'll recognise all three instantly for the rest of your career.
3. **The 3-broker lab.** Bring up the Compose file. Open the UI. Create a topic with RF=3, describe it, note the leaders and ISR.
4. **Prove the listener problem.** Change `KAFKA_ADVERTISED_LISTENERS` so the `EXTERNAL` entry advertises `kafka-1:9092` instead of `localhost:19092`. Try to produce from your host. Observe: connection succeeds, produce times out. Fix it. This is the single most valuable 10 minutes in this Part.
5. **Kill a broker.** With a RF=3 topic and continuous production running, `docker compose stop kafka-2`. Watch the ISR shrink in `--describe`, watch the producer keep working, then restart it and watch the ISR recover.
6. **Hit the internal-topic trap.** Single broker with `offsets.topic.replication.factor=3`. Try to start a consumer group. Read the error, then fix it.
7. **Topic-level override.** Set `retention.ms=60000` on a test topic, produce, wait, and confirm the data disappears — proving retention is independent of consumption.
8. **systemd.** Write and enable the unit file. Reboot the machine and confirm Kafka comes back and your data survived (proving you moved `log.dirs` off `/tmp`).
9. **Look inside `log.dirs`.** Find `meta.properties`, a partition directory, and the `.log/.index/.timeindex` trio. Then run `kafka-dump-log.sh --files <the .log file> --print-data-log` and see your actual records.
10. **JMX.** Set `JMX_PORT=9999`, restart, connect with JConsole, and find `kafka.server:type=BrokerTopicMetrics,name=MessagesInPerSec`. This is the foundation for Part 16.

Common Mistakes

- **Leaving `log.dirs=/tmp/kafka-logs`.** Reboot, data gone, plus `/tmp` is often a small tmpfs.
- **Confusing `log.dirs` (data) with `logs/` (application logs)** and deleting the wrong one.
- **Leaving the internal-topic replication factors at 3 on a single broker.** Everything appears fine until the first consumer group or transaction.
- **Getting `advertised.listeners` wrong.** The signature — connects, then times out — is unmistakable once you've seen it.
- **Huge JVM heap.** Makes Kafka slower, not faster.
- **Default file-descriptor limit (1024).** Fails at scale with confusing errors.
- **Running natively on Windows.** File-locking bugs cause crashes on segment deletion.

- **Storing `log.dirs` on `/mnt/c/` from WSL** — the filesystem bridge is far too slow.
 - **Single-broker "production" clusters.** `RF=1` means one disk failure is permanent data loss. You need three brokers minimum to make any of Kafka's guarantees real.
 - **Sharing a ZooKeeper ensemble between clusters without a chroot.** Two clusters fighting over the same znodes.
 - **kill -9 on a broker instead of `kafka-server-stop.sh`.** Graceful shutdown hands off leadership and flushes cleanly; SIGKILL forces log recovery on restart, which on a large broker takes a long time.
 - **Leaving `auto.create.topics.enable=true`.**
 - **Forgetting `TimeoutStopSec` in systemd,** so systemd SIGKILLS a broker that was politely handing off leadership.
-

Further Reading

- Apache Kafka docs §3 "Configuration" — bookmark the broker config table; you'll return to it constantly.
- Apache Kafka docs §6.6 "Operations — Hardware and OS" — the official filesystem, page cache, and swappiness guidance.
- Apache Kafka docs "KRaft" section — `kafka-storage.sh` and quorum setup.
- Confluent, "*Kafka Listeners — Explained*" by Robin Moffatt. The definitive article on `advertised.listeners`. Read it once and never lose a day to this again.
- `apache/kafka` and `confluentinc/cp-kafka` image docs on Docker Hub for the full `KAFKA_*` environment-variable mapping.
- Confluent, "*Running Kafka in Production*" — deployment sizing and OS tuning checklists.

Part 6 — Running Kafka & the CLI Toolbox

The CLI is not a beginner's crutch — it's how you debug production at 3 a.m. When a consumer group is stuck, you won't open your IDE. You'll run `kafka-consumer-groups --describe`. Learn these commands with your hands, not your eyes.

Convention in this Part: commands are written without the `.sh` suffix (as they appear with Homebrew or on `PATH`). On a tarball install, add `.sh`. Inside Docker, prefix with `docker exec -it kafka-1 /opt/kafka/bin/` and use `--bootstrap-server kafka-1:9092`.

```
# Make life easier – put this in your shell profile
export PATH=$PATH:/opt/kafka/bin
export BS=localhost:19092          # your bootstrap server
alias kt='kafka-topics.sh --bootstrap-server $BS'
alias kcg='kafka-consumer-groups.sh --bootstrap-server $BS'
alias kcp='kafka-console-producer.sh --bootstrap-server $BS'
alias kcc='kafka-console-consumer.sh --bootstrap-server $BS'
```

6.1 Starting and stopping

KRaft (the modern way)

```
# One-time, per cluster
CLUSTER_ID=$(kafka-storage.sh random-uuid)
kafka-storage.sh format -t $CLUSTER_ID -c config/server.properties

# Start
kafka-server-start.sh config/server.properties      # foreground
kafka-server-start.sh -daemon config/server.properties  # background

# Stop – graceful
kafka-server-stop.sh
```

ZooKeeper mode (legacy 3.x)

```
# Order matters: ZK first
zookeeper-server-start.sh -daemon config/zookeeper.properties
sleep 5
kafka-server-start.sh -daemon config/server.properties

# Shutdown in reverse
kafka-server-stop.sh
zookeeper-server-stop.sh
```

Why graceful shutdown matters

`kafka-server-stop.sh` sends `SIGTERM`, which triggers **controlled shutdown**:

SIGTERM

- └─> broker tells the controller "I'm going away"
- └─> controller migrates LEADERSHIP of this broker's partitions to other in-sync replicas ← clients barely notice
- └─> broker flushes and cleanly closes all log segments
- └─> exit

vs. kill -9:

- └─> leadership migration happens only AFTER failure detection (up to `replica.lag.time.max.ms` + election time) → visible errors
- └─> on restart, LOG RECOVERY must scan segments to verify integrity → minutes of downtime on a large broker

Always give shutdown 60–180 seconds (`TimeoutStopSec` in `systemd`). Rolling restarts of a production cluster mean **one broker at a time**, waiting for `UnderReplicatedPartitions` to return to 0 before touching the next one.

Health checks

```
# Can I talk to the cluster?
kafka-broker-api-versions.sh --bootstrap-server $BS | head -5

# Cluster metadata / quorum health (KRaft)
kafka-metadata-quorum.sh --bootstrap-server $BS describe --status
kafka-metadata-quorum.sh --bootstrap-server $BS describe --replication

# Which brokers are alive? (via any topic describe, or the UI)
kafka-cluster.sh cluster-id --bootstrap-server $BS

# Are any partitions unhealthy? ← the single most important operational check
kafka-topics.sh --bootstrap-server $BS --describe --under-replicated-partitions
kafka-topics.sh --bootstrap-server $BS --describe --unavailable-partitions
```

Those last two commands returning **nothing** is what a healthy cluster looks like. Memorise them.

6.2 Topics

```
# CREATE
kafka-topics.sh --bootstrap-server $BS --create \
  --topic orders \
  --partitions 6 \
  --replication-factor 3 \
  --config retention.ms=604800000 \
  --config min.insync.replicas=2 \
  --config compression.type=lz4

# LIST
kafka-topics.sh --bootstrap-server $BS --list
kafka-topics.sh --bootstrap-server $BS --list --exclude-internal

# DESCRIBE ← read this output carefully, it's the core diagnostic
kafka-topics.sh --bootstrap-server $BS --describe --topic orders

# ADD PARTITIONS (can only go up!)
kafka-topics.sh --bootstrap-server $BS --alter --topic orders --partitions 12

# DELETE
kafka-topics.sh --bootstrap-server $BS --delete --topic orders

# Filters for diagnosis
kafka-topics.sh --bootstrap-server $BS --describe --under-replicated-partitions
kafka-topics.sh --bootstrap-server $BS --describe --unavailable-partitions
kafka-topics.sh --bootstrap-server $BS --describe --at-min-isr-partitions
```

Reading `--describe` output

```
Topic: orders  TopicId: nZ8x...  PartitionCount: 3  ReplicationFactor: 3
  Configs: min.insync.replicas=2,retention.ms=604800000

Topic: orders  Partition: 0  Leader: 1  Replicas: 1,2,3  Isr: 1,2,3  Elr: LastKnownElr:
Topic: orders  Partition: 1  Leader: 2  Replicas: 2,3,1  Isr: 2,3,1
Topic: orders  Partition: 2  Leader: 3  Replicas: 3,1,2  Isr: 3,1  ← ▲
                                     ▲
                                broker 2 has fallen out of the ISR
                                for partition 2 (under-replicated)
```

Field	Meaning
Leader	broker currently serving reads/writes for this partition
Replicas	all assigned replicas, in preferred order — the first entry is the <i>preferred leader</i>
Isr	in-sync replicas. <code>Isr ⊂ Replicas</code> .
Elr	Eligible Leader Replicas (KIP-966, newer versions) — replicas known to be safe to elect even after unclean shutdown

Three things to check instantly:

1. **Isr shorter than Replicas** → under-replicated. A broker is down, slow, or GC-thrashing. If `|Isr|` drops below `min.insync.replicas`, producers with `acks=all` start failing.
2. **Leader: none** → partition offline. No ISR member alive. Reads and writes to that partition fail.
3. **Leaders all on one broker** → unbalanced load. Run `kafka-leader-election.sh --election-type preferred --all-topic-partitions`.

Topic configs

```
# Show non-default configs
kafka-configs.sh --bootstrap-server $BS --entity-type topics --entity-name orders --describe

# Show ALL configs including defaults and where they came from
kafka-configs.sh --bootstrap-server $BS --entity-type topics --entity-name orders \
  --describe --all

# Change
kafka-configs.sh --bootstrap-server $BS --entity-type topics --entity-name orders \
  --alter --add-config retention.ms=86400000,cleanup.policy=compact

# Revert to default
kafka-configs.sh --bootstrap-server $BS --entity-type topics --entity-name orders \
  --alter --delete-config retention.ms

# Broker configs (dynamic – no restart needed for many of them!)
kafka-configs.sh --bootstrap-server $BS --entity-type brokers --entity-name 1 --describe
kafka-configs.sh --bootstrap-server $BS --entity-type brokers --entity-name 1 \
  --alter --add-config log.cleaner.threads=2

# Cluster-wide default (entity-default)
kafka-configs.sh --bootstrap-server $BS --entity-type brokers --entity-default \
  --alter --add-config num.replica.fetchers=4
```

Config precedence, highest wins:

```
topic-level config  >  broker dynamic config  >
broker server.properties  >  Kafka default
```

⚠ **Topic deletion is asynchronous.** `--delete` marks the topic for deletion; the actual segment removal happens in the background. And recreating a topic with the same name immediately afterward can confuse consumers whose committed offsets now point into a topic with a different `TopicId`.

6.3 Console producer

```
# Simplest
kafka-console-producer.sh --bootstrap-server $BS --topic orders
> hello
> world
> ^D

# WITH KEYS – you'll use this constantly to test partitioning/ordering
kafka-console-producer.sh --bootstrap-server $BS --topic orders \
  --property parse.key=true \
  --property key.separator=:
> customer-991:{"orderId":8821}
> customer-991:{"orderId":8822}
> customer-772:{"orderId":8823}

# From a file
kafka-console-producer.sh --bootstrap-server $BS --topic orders < events.jsonl

# With headers (Kafka 3.x+)
kafka-console-producer.sh --bootstrap-server $BS --topic orders \
  --property parse.headers=true \
  --property headers.delimiter=$'\t' \
  --property headers.separator=, \
  --property headers.key.separator=:
> traceId:abc123,eventType:OrderPlaced  {"orderId":8821}

# Override producer properties
kafka-console-producer.sh --bootstrap-server $BS --topic orders \
  --producer-property acks=all \
  --producer-property compression.type=lz4 \
  --producer-property linger.ms=100

# Or from a properties file
kafka-console-producer.sh --bootstrap-server $BS --topic orders \
  --producer.config config/producer.properties

# Target a specific partition
kafka-console-producer.sh --bootstrap-server $BS --topic orders --partition 2
```

6.4 Console consumer

```
# Read new messages only (from the current end)
kafka-console-consumer.sh --bootstrap-server $BS --topic orders

# READ EVERYTHING FROM THE START ← the flag you'll use most
kafka-console-consumer.sh --bootstrap-server $BS --topic orders --from-beginning

# SHOW ME EVERYTHING (the debugging incantation – learn this by heart)
kafka-console-consumer.sh --bootstrap-server $BS --topic orders --from-beginning \
  --property print.key=true \
  --property print.value=true \
  --property print.partition=true \
  --property print.offset=true \
  --property print.timestamp=true \
  --property print.headers=true \
  --property key.separator=" | "

# Output looks like:
# CreateTime:1754239320123 | Partition:2 | Offset:41 | traceId:abc | customer-991 | {"orderId":8821}
```

```
# One specific partition, from a specific offset
kafka-console-consumer.sh --bootstrap-server $BS --topic orders \
  --partition 2 --offset 4180

# Latest N messages from a partition (very handy)
kafka-console-consumer.sh --bootstrap-server $BS --topic orders \
  --partition 0 --offset -5          # last 5

# Stop after N messages (otherwise it hangs forever waiting)
kafka-console-consumer.sh --bootstrap-server $BS --topic orders \
  --from-beginning --max-messages 10

# Join a named consumer group (commits offsets!)
kafka-console-consumer.sh --bootstrap-server $BS --topic orders \
  --group my-test-group

# Multiple topics by regex
kafka-console-consumer.sh --bootstrap-server $BS --include 'orders.*' --from-beginning

# Only committed records (skip aborted transactions)
kafka-console-consumer.sh --bootstrap-server $BS --topic orders \
  --isolation-level read_committed

# Deserializers for non-string data
kafka-console-consumer.sh --bootstrap-server $BS --topic metrics --from-beginning \
  --key-deserializer org.apache.kafka.common.serialization.StringDeserializer \
  --value-deserializer org.apache.kafka.common.serialization.LongDeserializer
```

Three gotchas everyone hits

1. **Without `--group`, the console consumer generates a random group ID (`console-consumer-12345`) every run.** So it never remembers where it was, and it litters `__consumer_offsets` with dead groups.
2. **Without `--from-beginning`, you see nothing** if all data was produced before you started. It's not broken. `auto.offset.reset=latest` is the default.
3. **It doesn't exit.** It waits for more data. Use `--max-messages` or `--timeout-ms 5000` in scripts, or Ctrl-C.

Reading internal topics

```
# See raw offset commits – genuinely educational
kafka-console-consumer.sh --bootstrap-server $BS \
  --topic __consumer_offsets --from-beginning \
  --formatter "kafka.coordinator.group.GroupMetadataManager$OffsetsMessageFormatter" \
  --max-messages 20

# Browse the KRaft metadata log
kafka-metadata-shell.sh --snapshot /var/lib/kafka/data/__cluster_metadata-0/00000000000000000000.log
# then: ls / ; cat /brokers/1 ; ls /topics
```

6.5 Consumer groups — the most important tool you own

```
# List all groups
kafka-consumer-groups.sh --bootstrap-server $BS --list

# DESCRIBE – this is the command you run when something is wrong
kafka-consumer-groups.sh --bootstrap-server $BS --describe --group payments
```

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG	CONSUMER-ID	HOST	CLIENT-ID
payments	orders	0	4182	4182	0	consumer-payments-1-a	/10.0.1.5	consumer-1
payments	orders	1	3901	3905	4	consumer-payments-2-b	/10.0.1.6	consumer-2
payments	orders	2	1200	9800	8600	consumer-payments-3-c	/10.0.1.7	consumer-3

▲▲▲▲

partition 2 is 8600 records behind:
a slow/stuck consumer, a hot partition,
or a poison-pill record blocking progress

```
# Group state and coordinator
kafka-consumer-groups.sh --bootstrap-server $BS --describe --group payments --state
# GROUP      COORDINATOR (ID)  ASSIGNMENT-STRATEGY  STATE  #MEMBERS
# payments   kafka-2:9092 (2)         cooperative-sticky    Stable  3

# Members and their partition assignments
kafka-consumer-groups.sh --bootstrap-server $BS --describe --group payments --members --verbose

# Delete a group (must have no active members)
kafka-consumer-groups.sh --bootstrap-server $BS --delete --group old-group

# Delete just the committed offsets for one topic
kafka-consumer-groups.sh --bootstrap-server $BS --delete-offsets \
  --group payments --topic orders
```

Group states, and what each means

State	Meaning
Stable	assignments settled, consuming normally ✓
PreparingRebalance	members are being asked to rejoin
CompletingRebalance	leader is computing/distributing the assignment
Empty	no active members, but committed offsets are retained
Dead	group and its metadata are gone

A group that keeps flipping between `PreparingRebalance` and `Stable` is **rebalance-storming** — usually `max.poll.interval.ms` too low for your processing time, or `session.timeout.ms` too tight, or pods being killed. Part 9.

Resetting offsets — replay, skip, and "oh no" recovery

This is the operational superpower Kafka gives you. **Always run with `--dry-run` first** (which is actually the default if you specify neither `--dry-run` nor `--execute`).

```
# The group MUST have no active members. Stop your consumers first.

# Replay everything
kafka-consumer-groups.sh --bootstrap-server $BS --group payments \
  --reset-offsets --to-earliest --topic orders --execute

# Skip everything (jump to now) – the "unstick a hopeless backlog" button
kafka-consumer-groups.sh --bootstrap-server $BS --group payments \
  --reset-offsets --to-latest --topic orders --execute

# Replay the last 2 hours ← most common real-world reset
kafka-consumer-groups.sh --bootstrap-server $BS --group payments \
  --reset-offsets --by-duration PT2H --topic orders --execute

# From a wall-clock timestamp
kafka-consumer-groups.sh --bootstrap-server $BS --group payments \
  --reset-offsets --to-datetime 2026-08-03T09:00:00.000 --topic orders --execute

# Absolute offset, one partition
kafka-consumer-groups.sh --bootstrap-server $BS --group payments \
  --reset-offsets --to-offset 4000 --topic orders:2 --execute

# Relative: back up 100 records on every partition
kafka-consumer-groups.sh --bootstrap-server $BS --group payments \
  --reset-offsets --shift-by -100 --topic orders --execute

# Skip past a poison-pill record you can't deserialize
kafka-consumer-groups.sh --bootstrap-server $BS --group payments \
  --reset-offsets --shift-by 1 --topic orders:2 --execute

# All topics the group consumes
kafka-consumer-groups.sh --bootstrap-server $BS --group payments \
  --reset-offsets --to-earliest --all-topics --execute

# Export/import for auditability – do this in production
kafka-consumer-groups.sh --bootstrap-server $BS --group payments \
  --reset-offsets --to-earliest --all-topics --dry-run --export > offsets.csv
kafka-consumer-groups.sh --bootstrap-server $BS --group payments \
  --reset-offsets --from-file offsets.csv --execute
```

⚠ **Before you reset to earliest in production, ask: are my consumers idempotent?** Replaying 3 days of `OrderPlaced` through a non-idempotent payment consumer charges every customer again. Resetting offsets is a data-mutation operation in disguise.

Reset options at a glance: `--to-earliest`, `--to-latest`, `--to-offset N`, `--shift-by ±N`, `--to-datetime`, `--by-duration` (ISO-8601 like `PT2H30M`), `--to-current`, `--from-file`.

6.6 Offsets and lag

```
# Log-end offsets (latest)
kafka-get-offsets.sh --bootstrap-server $BS --topic orders --time -1

# Log-start offsets (earliest still retained)
kafka-get-offsets.sh --bootstrap-server $BS --topic orders --time -2

# Offsets as of a timestamp (epoch millis) – "where was the log at 9 a.m.?"
kafka-get-offsets.sh --bootstrap-server $BS --topic orders --time 1754226000000

# Max timestamp per partition
kafka-get-offsets.sh --bootstrap-server $BS --topic orders --time -3

# Count records in a topic (end – start, summed)
kafka-run-class.sh kafka.tools.GetOffsetShell --bootstrap-server $BS --topic orders --time -1 \
| awk -F: '{sum += $3} END {print "latest total:", sum}'
```

LAG = log-end offset – committed offset

Lag is per-partition. Group lag is usually reported as the sum, but the **MAXIMUM** is what matters – one partition at 8600 while others are at 0 means one consumer is stuck, not that the group is 8600/3 behind.

6.7 Inspecting the physical log (great for interviews)

```
# Decode a raw segment file – see batches, offsets, timestamps, compression
kafka-dump-log.sh --files /var/lib/kafka/data/orders-0/00000000000000000000.log \
--print-data-log

# Just the batch headers (compact)
kafka-dump-log.sh --files /var/lib/kafka/data/orders-0/00000000000000000000.log

# The sparse offset index
kafka-dump-log.sh --files /var/lib/kafka/data/orders-0/00000000000000000000.index

# Verify index integrity
kafka-dump-log.sh --files .../00000000000000000000.log --index-sanity-check
```

Sample output, annotated:

```
baseOffset: 0 lastOffset: 2 count: 3 baseSequence: 0 lastSequence: 2
producerId: 1001 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: false
isControl: false position: 0 CreateTime: 1754239320123
size: 234 magic: 2 compresscodec: lz4 crc: 3095174108 invalid: true
▲                                     ▲                                     ▲
3 records in ONE batch                producer ID    compression is per-BATCH,
(batching in action)                  (idempotence) not per-record
| offset: 0 CreateTime: ... keySize: 12 valueSize: 30 headerKeys: [traceId]
| offset: 1 CreateTime: ... keySize: 12 valueSize: 31 headerKeys: [traceId]
| offset: 2 CreateTime: ... keySize: 12 valueSize: 30 headerKeys: [traceId]
```

Running this once teaches you more about Part 7 than any diagram: you can see that records are grouped into **batches**, that compression applies to the batch, and that `producerId / sequence` exist for idempotence.

6.8 Performance testing

```
# Producer throughput
kafka-producer-perf-test.sh \
  --topic perf-test \
  --num-records 1000000 \
  --record-size 1024 \
  --throughput -1 \
  --producer-props bootstrap.servers=$BS acks=all linger.ms=10 \
    compression.type=lz4 batch.size=65536

# Output:
# 1000000 records sent, 187265.9 records/sec (182.88 MB/sec),
# 12.4 ms avg latency, 342.0 ms max latency,
# 8 ms 50th, 31 ms 95th, 78 ms 99th, 201 ms 99.9th.

# Consumer throughput
kafka-consumer-perf-test.sh --bootstrap-server $BS \
  --topic perf-test --messages 1000000 --threads 1

# End-to-end latency
kafka-run-class.sh kafka.tools.EndToEndLatency $BS perf-test 10000 all 1024
```

Use these to *measure* the tuning claims in Part 14 rather than believing them. Change one property at a time and record the numbers.

6.9 Partition reassignment and leader balancing

```
# 1. Describe what you want to move
cat > topics.json <<'EOF'
{"topics":[{"topic":"orders"}], "version":1}
EOF

# 2. Generate a proposed plan for a set of brokers
kafka-reassign-partitions.sh --bootstrap-server $BS \
  --topics-to-move-json-file topics.json \
  --broker-list "1,2,3,4" --generate
# → prints "current" (SAVE THIS as rollback.json) and "proposed"

# 3. Execute, with a throttle so replication doesn't saturate the network
kafka-reassign-partitions.sh --bootstrap-server $BS \
  --reassignment-json-file proposed.json --execute \
  --throttle 50000000 # 50 MB/s

# 4. Verify (this also removes the throttle when complete)
kafka-reassign-partitions.sh --bootstrap-server $BS \
  --reassignment-json-file proposed.json --verify

# Rebalance leadership back to preferred leaders
kafka-leader-election.sh --bootstrap-server $BS \
  --election-type preferred --all-topic-partitions

# Emergency only: elect an out-of-sync replica (ACCEPTS DATA LOSS)
kafka-leader-election.sh --bootstrap-server $BS \
  --election-type unclean --topic orders --partition 2
```

Always use `--throttle`. An unthrottled reassignment of large partitions will saturate your network, starve normal replication, cause ISR shrinkage, and turn a maintenance task into an outage. And **always save the "current" plan** as your rollback.

6.10 Other tools worth knowing

```
# ACLs (Part 15)
kafka-acls.sh --bootstrap-server $BS --list
kafka-acls.sh --bootstrap-server $BS --add \
  --allow-principal User:order-service --operation Write --topic orders

# Quotas (throttle a noisy client)
kafka-configs.sh --bootstrap-server $BS --alter \
  --add-config 'producer_byte_rate=1048576,consumer_byte_rate=2097152' \
  --entity-type clients --entity-name order-service

# Delete records below an offset (GDPR / free space; does NOT delete the topic)
cat > delete.json <<'EOF'
{"partitions":[{"topic":"orders","partition":0,"offset":5000}], "version":1}
EOF
kafka-delete-records.sh --bootstrap-server $BS --offset-json-file delete.json

# Verify replica consistency across brokers
kafka-replica-verification.sh --broker-list $BS --topic-white-list orders

# Transactions
kafka-transactions.sh --bootstrap-server $BS list
kafka-transactions.sh --bootstrap-server $BS describe --transactional-id my-txn-1

# Console consumer/producer for Avro (Confluent distribution)
kafka-avro-console-consumer --bootstrap-server $BS --topic orders \
  --property schema.registry.url=http://localhost:8081 --from-beginning
```

6.11 A 10-minute hands-on walkthrough

Do this now, with the Docker Compose cluster from Part 5. Four terminals.

```
# — Terminal 1: create and inspect _____
export BS=localhost:19092
kafka-topics.sh --bootstrap-server $BS --create --topic orders \
  --partitions 3 --replication-factor 3 --config min.insync.replicas=2
kafka-topics.sh --bootstrap-server $BS --describe --topic orders
# Note the Leader and Isr for each of the 3 partitions.

# — Terminal 2: consume, showing everything _____
kafka-console-consumer.sh --bootstrap-server $BS --topic orders \
  --from-beginning --group payments \
  --property print.key=true --property print.partition=true \
  --property print.offset=true --property key.separator=" | "

# — Terminal 3: produce with keys _____
kafka-console-producer.sh --bootstrap-server $BS --topic orders \
  --property parse.key=true --property key.separator=:
> customer-991:order-1
> customer-991:order-2
> customer-772:order-3
> customer-991:order-4
> customer-555:order-5
```

Observe in terminal 2: all `customer-991` records land in the *same* partition, in order. `customer-772` and `customer-555` may land elsewhere. You have now seen per-key ordering with your own eyes.

```
# — Terminal 4: watch the group _____
watch -n 1 'kafka-consumer-groups.sh --bootstrap-server localhost:19092 \
  --describe --group payments'
```

Now run the experiments:

```
# 1. Scale the group: start a 2nd and 3rd console consumer with --group payments
#   → watch partitions redistribute in terminal 4 (a REBALANCE)
# 2. Start a 4TH consumer → it gets nothing. Partitions cap parallelism.
# 3. Ctrl-C a consumer → watch its partitions get reassigned
# 4. Kill a broker: docker compose stop kafka-2
#   → re-describe the topic; watch the ISR shrink and leaders move
# 5. Reset and replay:
kafka-consumer-groups.sh --bootstrap-server $BS --group payments \
  --reset-offsets --to-earliest --topic orders --dry-run
#   stop all consumers, then --execute, restart a consumer, watch it re-read
# 6. Second group sees everything independently:
kafka-console-consumer.sh --bootstrap-server $BS --topic orders \
  --from-beginning --group analytics
#   → proves pub/sub across groups
# 7. Look at the bytes:
docker exec kafka-1 /opt/kafka/bin/kafka-dump-log.sh \
  --files /var/lib/kafka/data/orders-0/00000000000000000000.log --print-data-log
```

If you do all seven, Parts 4, 7, 9 and 12 will feel like review rather than new material.

Summary

- `kafka-server-stop.sh` (SIGTERM) triggers **controlled shutdown**: leadership migrates first, segments close cleanly. `kill -9` forces slow log recovery on restart and visible client errors. Roll restarts one broker at a time, waiting for `UnderReplicatedPartitions = 0`.

- `kafka-topics --describe` is your primary diagnostic: `Isr` shorter than `Replicas` = under-replicated; `Leader: none` = offline partition; the first entry in `Replicas` is the *preferred* leader.
- The two health checks worth memorising: `--describe --under-replicated-partitions` and `--unavailable-partitions`. Both should output nothing.
- Console consumer without `--from-beginning` shows nothing (default `auto.offset.reset=latest`); without `--group` it uses a throwaway random group.
- `kafka-consumer-groups --describe` gives you per-partition lag. Look at the **maximum** partition lag, not the sum.
- `--reset-offsets` gives replay, skip-ahead, and poison-pill skipping. It requires an inactive group, defaults to dry-run, and is only safe if your consumers are idempotent.
- `kafka-dump-log` shows you the real on-disk format — batches, producer IDs, per-batch compression.
- Always `--throttle` a partition reassignment, and always save the "current" plan as rollback.

Interview Questions

Q1. How do you check if a Kafka cluster is healthy from the CLI? `kafka-topics --describe --under-replicated-partitions` and `--unavailable-partitions` should both return nothing. Then `kafka-metadata-quorum describe --status` for controller quorum health, and `kafka-consumer-groups --describe` per critical group to check lag and that state is `Stable`. Broker-side, watch the `UnderReplicatedPartitions`, `OfflinePartitionsCount` and `ActiveControllerCount` JMX metrics (Part 16).

Q2. Your consumer group has 50,000 lag on one partition and 0 on the others. Diagnose. It's not a capacity problem — it's one partition or one consumer. Candidates: (a) a **hot partition** because one key dominates (e.g. everything keyed to the same tenant); (b) the consumer owning that partition is stuck — a blocking call, a deadlock, or an infinite retry loop; (c) a **poison-pill record** that fails deserialization or processing and is retried forever; (d) that consumer's pod is throttled or its downstream dependency is slow. Check `--describe --members` to see which host owns it, look at that instance's logs and thread dumps, and inspect the record at the stuck offset with `kafka-console-consumer --partition N --offset M --max-messages 1`.

Q3. How do you replay the last 6 hours of a topic for one consumer group? Stop all consumers in the group (reset requires an inactive group), then: `kafka-consumer-groups --group G --reset-offsets --by-duration PT6H --topic T --dry-run`, review the output, re-run with `--execute`, restart consumers. Verify the consumers are idempotent first — replay reprocesses side effects.

Q4. A consumer can't deserialize one record and is stuck. Fastest fix? Short-term: skip it with `--reset-offsets --shift-by 1 --topic T:partition --execute` on the stopped group. Real fix: an `ErrorHandlingDeserializer` (Spring) or a try/catch around deserialization that routes bad records to a dead-letter topic so a single malformed record can't block a partition indefinitely.

Q5. Why does `--reset-offsets` require the group to have no active members? Because active members own their partitions and hold in-memory positions; they'd immediately commit over your reset. Kafka refuses rather than silently losing the change.

Q6. Difference between `kafka-server-stop.sh` and `kill -9`? `kafka-server-stop.sh` sends `SIGTERM`, which triggers controlled shutdown: the broker notifies the controller, leadership for its partitions migrates to in-sync replicas, and log segments are closed cleanly. `kill -9` skips all of that — clients see errors until failure detection and election complete, and on restart the broker must run log recovery, scanning segments to verify integrity, which can take many minutes on a large broker.

Q7. How do you safely add a broker and rebalance data onto it? Start the broker (it joins but hosts no existing partitions — Kafka does not auto-move data). Then `kafka-reassign-partitions --generate` with the new broker list, **save the "current" output as rollback**, `--execute` with a `--throttle` sized to leave headroom for normal replication, and `--verify` until complete (which also clears the throttle). Finish with

`kafka-leader-election --election-type preferred` to spread leadership. Or use Cruise Control (Part 17) to do it continuously.

Q8. How do you delete specific data from a topic without deleting the topic? `kafka-delete-records` with a JSON file specifying a per-partition offset — everything *below* that offset is removed. It's offset-based, not record-based, so you can't delete one record from the middle. For key-based deletion you need a compacted topic and a tombstone (`value=null`). Design GDPR requirements in up front: either keep PII out of Kafka, use short retention, or use compaction with tombstones.

Q9. Why does the console consumer show nothing when the topic clearly has data? Default `auto.offset.reset=latest` — it starts at the end and waits for new records. Add `--from-beginning`. (Or, if you passed `--group` and that group already has committed offsets at the end, it resumes there — `--from-beginning` is ignored for a group with existing offsets.)

Q10. What does `kafka-dump-log` teach you? That records are stored in **batches** with a shared header, that compression is applied per batch (not per record), that each batch carries a `producerId`, `producerEpoch` and sequence range (the basis of idempotence), and that the `.index` file maps offsets to byte positions sparsely. It's the fastest way to make Part 7's internals concrete.

Mini Quiz

1. Flag to read a topic from offset 0 with the console consumer?
2. Command to see per-partition consumer lag?
3. What does `Isr: 3,1` with `Replicas: 3,1,2` tell you?
4. What does `Leader: none` mean?
5. Can you reduce partitions with `--alter`?
6. Which flag prevents an offset reset from actually happening?
7. Group state that means "consuming normally"?
8. What must be true before you can reset a group's offsets?
9. What does the *first* broker ID in `Replicas` signify?
10. Which two `kafka-topics --describe` filters should return nothing on a healthy cluster?
11. Why must you throttle a partition reassignment?
12. How do you count the records in a topic?

ANSWERS

1. `--from-beginning` (and it's ignored if `--group` already has committed offsets).
2. `kafka-consumer-groups --bootstrap-server ... --describe --group G`.
3. Broker 2 has fallen out of the ISR — the partition is under-replicated. If `min.insync.replicas=2`, you are one failure from rejecting writes.
4. No replica in the ISR is alive, so the partition is offline: reads and writes to it fail until an ISR member returns (or you accept unclean election).
5. No. Only increase.
6. `--dry-run` (also the default when neither `--dry-run` nor `--execute` is given).
7. `Stable`.
8. The group must have **no active members** — stop all consumers.
9. It's the **preferred leader** — the replica Kafka will hand leadership back to during preferred-leader rebalancing.
10. `--under-replicated-partitions` and `--unavailable-partitions`.

11. An unthrottled reassignment saturates the network, starves normal replication, shrinks ISRs and can cause an outage.
12. Sum `--time -1` (log-end) minus `--time -2` (log-start) across partitions via `kafka-get-offsets`. Note this counts offsets, which over-counts if there are transaction markers or compaction gaps.

Practice Exercises

1. Do the full §6.11 walkthrough, all seven experiments. Non-negotiable.
2. **Prove pub/sub.** Two groups on one topic. Advance one group to the end, then start the second with `--from-beginning`. Confirm it gets everything.
3. **Prove per-key ordering.** Produce 50 records alternating between two keys to a 3-partition topic. Consume with `print.partition=true`. Verify each key stays in one partition, in order.
4. **Break ordering.** Same test with no key. Observe the spread.
5. **Cause a rebalance storm.** Start and stop consumers rapidly while watching `--describe --state`. Watch the group thrash between `PreparingRebalance` and `Stable`, and note that no records are processed during a rebalance.
6. **Hit `min.insync.replicas`.** RF=3, `min.insync.replicas=2`. Produce with `--producer-property acks=all` while stopping two brokers. Capture the exact exception.
7. **Poison pill.** Produce a record whose value breaks your consumer's deserializer. Watch the partition stall. Fix it with `--shift-by 1`. Then fix it *properly* with error-handling deserialization (Part 19).
8. **Retention.** Topic with `retention.ms=30000` and `segment.ms=10000`. Produce, wait 60 s, then `--from-beginning`. Confirm the data is gone even though nobody consumed it.
9. **Compaction.** Topic with `cleanup.policy=compact`. Produce `(A,1) (A,2) (A,3)`, force a segment roll, wait for the cleaner, read from the beginning. Then produce `(A,null)` and watch `A` disappear.
10. **Perf test with one variable.** Run `kafka-producer-perf-test` with `linger.ms=0` and then `linger.ms=100`, everything else identical. Record throughput and p99 latency for both. You now own a real data point for Part 14.
11. **Reassignment drill.** Add a 4th broker to the Compose file, generate a reassignment plan, save the rollback, execute with a throttle, verify, then run preferred-leader election.
12. **Write a lag alert.** A shell script that fails (exit 1) if any group's maximum partition lag exceeds a threshold. This is the seed of real monitoring.

Common Mistakes

- **Forgetting `--from-beginning`** and concluding Kafka is broken.
- **Not passing `--group`**, so every console-consumer run starts fresh and pollutes `__consumer_offsets` with dead groups.
- **Running `--reset-offsets --to-earliest` in production without checking idempotency.** You just re-charged everyone.
- **Resetting offsets while consumers are running** and being confused when it fails (or when the running consumers overwrite it).
- **kill -9 on brokers**, then wondering why restarts take 20 minutes (log recovery) and why clients errored.
- **Restarting all brokers at once.** Roll one at a time and wait for `UnderReplicatedPartitions = 0`.
- **Unthrottled reassignment.** A maintenance task becomes an incident.

- **Not saving the "current" plan** from `--generate`, leaving no rollback.
 - **Increasing partitions on a keyed topic without thinking**, silently breaking per-key ordering and any keyed consumer state.
 - **Recreating a deleted topic immediately**. Deletion is async; the new topic gets a different `TopicId` and consumers' committed offsets become invalid.
 - **Reading total group lag instead of maximum partition lag**. Hides the one stuck consumer, which is almost always the actual problem.
 - **Using `kafka-console-producer` for load tests**. It's single-threaded with poor defaults; use `kafka-producer-perf-test`.
 - **Leaving a `watch` on `--describe` for a huge cluster**. Each call is a real cluster request; on thousands of partitions it's not free.
-

Further Reading

- Apache Kafka docs §6 "Operations" — the canonical reference for all of these tools, plus rolling-restart and reassignment procedures.
- `kafka-topics.sh --help`, `kafka-consumer-groups.sh --help` etc. — genuinely the best documentation for flags; read them once each.
- Confluent docs, "CLI Tools for Confluent Platform" — covers the extra Avro and Schema Registry console tools.
- KIP-122: "Add Reset Consumer Group Offsets tooling" — the design rationale for `--reset-offsets` and every option it supports.
- `kafka-dump-log.sh --help` and Kafka's `RecordBatch` javadoc — pair these with Part 7.